Bank Churn Prediction

Problem Statement

Context

Businesses like banks which provide service have to worry about problem of 'Customer Churn' i.e. customers leaving and joining another service provider. It is important to understand which aspects of the service influence a customer's decision in this regard. Management can concentrate efforts on improvement of service, keeping in mind these priorities.

Objective

You as a Data scientist with the bank need to build a neural network based classifier that can determine whether a customer will leave the bank or not in the next 6 months.

Data Dictionary

- CustomerId: Unique ID which is assigned to each customer
- Surname: Last name of the customer
- CreditScore: It defines the credit history of the customer.
- Geography: A customer's location
- Gender: It defines the Gender of the customer
- Age: Age of the customer
- Tenure: Number of years for which the customer has been with the bank
- NumOfProducts: refers to the number of products that a customer has purchased through the bank.
- Balance: Account balance
- HasCrCard: It is a categorical variable which decides whether the customer has credit card or not.
- EstimatedSalary: Estimated salary
- isActiveMember: Is is a categorical variable which decides whether the customer is active member of the bank or not (Active member in the sense, using bank products regularly, making transactions etc)
- Exited : whether or not the customer left the bank within six month. It can take two values

** 0=No (Customer did not leave the bank) ** 1=Yes (Customer left the bank)

Keep Alive Code

```
#import time
#while True:
#       print("Keeping Colab Active...")
#       time.sleep(500)  # Wait for 500 seconds before printing again
```

Importing necessary libraries

```
# Libraries to help with reading and manipulating data
import pandas as pd
import numpy as np

import time
import random

# libaries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Library to suppress warnings or deprecation notes
import warnings
warnings.filterwarnings("ignore")

# To suppress scientific notations
pd.set_option("display.float_format", lambda x: "%.3f" % x)

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)

# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 200)
```

```
# to split the data into train and test
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
# importing SMOTE
from imblearn.over_sampling import SMOTE

# importing different functions to build models
import tensorflow as tf
from tensorflow import keras
from keras import backend
from keras.models import Sequential
from keras.layers import Dense, Dropout, Input
from keras.utils import plot_model
from tensorflow.keras.metrics import Precision, Recall
from tensorflow.keras.callbacks import EarlyStopping
```

```
# importing metrics
from sklearn import metrics
# To get diferent metric scores
from sklearn.metrics import (
    f1_score,
    accuracy_score,
    recall_score,
    precision_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    make_scorer,
    classification_report,
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)
```

```
# Library to avoid the warnings
import warnings
warnings.filterwarnings("ignore")
```

```
# Set the seed using keras.utils.set_random_seed. This will set:
# 1) `numpy` seed
# 2) backend random seed
# 3) `python` random seed
keras.utils.set_random_seed(812)

# If using TensorFlow, this will make GPU ops as deterministic as possible,
# but it will affect the overall performance, so be mindful of that.
tf.config.experimental.enable_op_determinism()
```

Loading the dataset

```
# This is specific to my google colab environment
from google.colab import drive

drive.mount('/content/drive',force_remount = True)

FOLDER_PATH = '/content/drive/MyDrive/Colab_Notebooks/Bank_Churn_Project/'

data = pd.read_csv(FOLDER_PATH+"bank-1.csv")
```

```
Mounted at /content/drive
```

Data Overview

Displaying Head & Tail of the dataset

```
#Head and Tail of the data
display(data.head())
display(data.tail())
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	1	15634602	Hargrave	619	France	Female	42	2	0.000	1	1	1	101348.880	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.860	1	0	1	112542.580	0
2	3	15619304	Onio	502	France	Female	42	8	159660.800	3	1	0	113931.570	1
3	4	15701354	Boni	699	France	Female	39	1	0.000	2	0	0	93826.630	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.820	1	1	1	79084.100	0

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
9995	9996	15606229	Obijiaku	771	France	Male	39	5	0.000	2	1	0	96270.640	0
9996	9997	15569892	Johnstone	516	France	Male	35	10	57369.610	1	1	1	101699.770	0
9997	9998	15584532	Liu	709	France	Female	36	7	0.000	1	0	1	42085.580	1
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3	75075.310	2	1	0	92888.520	1
9999	10000	15628319	Walker	792	France	Female	28	4	130142.790	1	1	0	38190.780	0

Displaying the shape of the dataset

```
#Shape of the data
print(f"There are {data.shape[0]} rows and {data.shape[1]} columns.")
```

```
There are 10000 rows and 14 columns.
```

- We see that the dataset has 10,000 Rows and 14 Columns

Checking 10 random rows in the dataset

```
data.sample(n=10, random_state=1)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
9953	9954	15655952	Burke	550	France	Male	47	2	0.000	2	1	1	97057.280	0
3850	3851	15775293	Stephenson	680	France	Male	34	3	143292.950	1	1	0	66526.010	0
4962	4963	15665088	Gordon	531	France	Female	42	2	0.000	2	0	1	90537.470	0
3886	3887	15720941	Tien	710	Germany	Male	34	8	147833.300	2	0	1	1561.580	0
5437	5438	15733476	Gonzalez	543	Germany	Male	30	6	73481.050	1	1	1	176692.650	0
8517	8518	15671800	Robinson	688	France	Male	20	8	137624.400	2	1	1	197582.790	0
2041	2042	15709846	Yeh	840	France	Female	39	1	94968.970	1	1	0	84487.620	0
1989	1990	15622454	Zaitsev	695	Spain	Male	28	0	96020.860	1	1	1	57992.490	0
1933	1934	15815560	Bogle	666	Germany	Male	74	7	105102.500	1	1	1	46172.470	0
9984	9985	15696175	Echezonachukwu	602	Germany	Male	35	7	90602.420	2	1	1	51695.410	0

Checking data types present in the dataset

```
df = data.copy()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

- Three columns are of the "Object" type, two are of type "float" and the remaining nine are of type "Int".

Statistical summary of the dataset

```
df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
RowNumber	10000.000	NaN	NaN	NaN	5000.500	2886.896	1.000	2500.750	5000.500	7500.250	10000.000
CustomerId	10000.000	NaN	NaN	NaN	15690940.569	71936.186	15565701.000	15628528.250	15690738.000	15753233.750	15815690.000
Surname	10000	2932	Smith	32	NaN	NaN	NaN	NaN	NaN	NaN	NaN
CreditScore	10000.000	NaN	NaN	NaN	650.529	96.653	350.000	584.000	652.000	718.000	850.000
Geography	10000	3	France	5014	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Gender	10000	2	Male	5457	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Age	10000.000	NaN	NaN	NaN	38.922	10.488	18.000	32.000	37.000	44.000	92.000
Tenure	10000.000	NaN	NaN	NaN	5.013	2.892	0.000	3.000	5.000	7.000	10.000
Balance	10000.000	NaN	NaN	NaN	76485.889	62397.405	0.000	0.000	97198.540	127644.240	250898.090
NumOfProducts	10000.000	NaN	NaN	NaN	1.530	0.582	1.000	1.000	1.000	2.000	4.000
HasCrCard	10000.000	NaN	NaN	NaN	0.706	0.456	0.000	0.000	1.000	1.000	1.000
IsActiveMember	10000.000	NaN	NaN	NaN	0.515	0.500	0.000	0.000	1.000	1.000	1.000
EstimatedSalary	10000.000	NaN	NaN	NaN	100090.240	57510.493	11.580	51002.110	100193.915	149388.247	199992.480
Exited	10000.000	NaN	NaN	NaN	0.204	0.403	0.000	0.000	0.000	0.000	1.000

Key Customer Characteristics:

- **Credit Score:** Ranges widely from 350 to 850, with an average score of 650.5. Half the customers have scores between 584 and 718.
- **Age:** Customers range from 18 to 92 years old, with an average age of approximately 39 years. The middle 50% of customers are between 32 and 44 years old.
- **Tenure:** Customers have been with the bank between 0 and 10 years, with an average tenure of 5 years.
- **Account Balance:** Balances vary significantly, from 0 up to nearly 251,000.

At least 25% of customers have a zero balance, while the median balance is $97,199, indicating a skew potentially due to the zero-balance accounts.

- **Number of Products:** Customers typically hold 1 or 2 bank products (with an average of 1.53). More than half of the customers hold only one product.
- **Estimated Salary:** Estimated salaries are distributed across a wide range (approx. $11.58 to 200,000). The median is close to the mean and distribution relatively uniform.
- **Credit Card Ownership:** About 70.6% of customers have a credit card with the bank.
- **Active Members:** The customer base is nearly evenly split, with 51.5% identified as active members.
- **Exited Status (Churn):** Approximately 20.4% of the customers in this dataset have closed their accounts (exited).

Checking for missing values

```
df.isnull().sum()
```

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0

dtype: int64

Checking for unique values

```
df.nunique()
```

	0
RowNumber	10000
CustomerId	10000
Surname	2932
CreditScore	460
Geography	3
Gender	2
Age	70
Tenure	11
Balance	6382
NumOfProducts	4
HasCrCard	2
IsActiveMember	2
EstimatedSalary	9999
Exited	2

dtype: int64

```
# Print the uniques values in Exited
print("Unique values in Exited:", np.sort(df['Exited'].unique()))

# Print the unique values in Geography
print("Unique values in Geography:", df['Geography'].unique())

# Print the unique values in Gender
print("Unique values in Gender:", df['Gender'].unique())

# Print the unique values in NumOfProducts
print("Unique values in NumOfProducts:", np.sort(df['NumOfProducts'].unique()))

# Print the unique values in Tenure
print("Unique values in Tenure:", np.sort(df['Tenure'].unique()))
```

```
Unique values in Exited: [0 1]
Unique values in Geography: ['France' 'Spain' 'Germany']
Unique values in Gender: ['Female' 'Male']
Unique values in NumOfProducts: [1 2 3 4]
Unique values in Tenure: [ 0  1  2  3  4  5  6  7  8  9 10]
```

- There are quite a few unique columns
- Exited has two unique values - 0, 1
- Geography has 3 unique values - France, Germany, Spain
- Gender has 2 unique values - Male, Female
- NumOfProducts has 4 unique values

Checking for duplicates

```
df.duplicated().sum()
```

```
np.int64(0)
```

- No duplicate rows found in the dataset

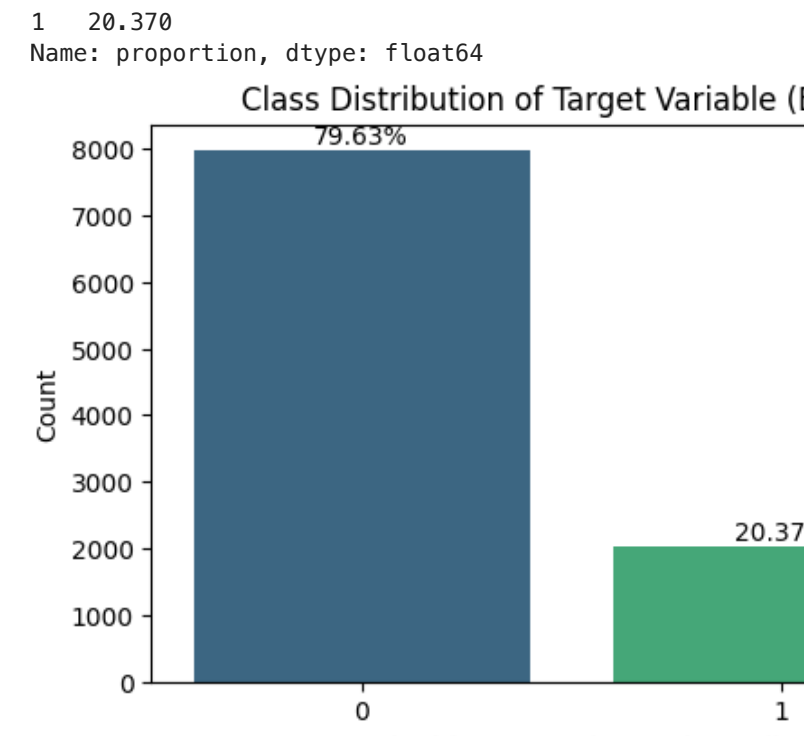
Dropping columns

```
df = df.drop(['RowNumber','CustomerId','Surname'], axis=1)
```

Checking for data imbalance

```
# 1. Check value counts
class_counts = df['Exited'].value_counts()
print("Class Distribution:")
print(class_counts)

# 2. Percentage distribution
class_percentages = df['Exited'].value_counts(normalize=True) * 100
print("\nPercentage Distribution:")
print(class_percentages)

# 3. Plot class distribution
plt.figure(figsize=(6,4))
sns.barplot(x=class_counts.index, y=class_counts.values, palette="viridis")
plt.title("Class Distribution of Target Variable (Exited)")
plt.xlabel("Exited (0 = Stayed, 1 = Churned)")
plt.ylabel("Count")
for i, val in enumerate(class_counts.values):
    plt.text(i, val + 100, f'{class_percentages[i]:.2f}%', ha='center')
plt.show()
```

```
Class Distribution:
Exited
0    7963
1    2037
Name: count, dtype: int64

Percentage Distribution:
Exited
0   79.630
1   20.370
Name: proportion, dtype: float64
```



- Data is imbalanced: almost an 80-20 split between customers who stayed and exited.
- If this imbalance is not handled then, we might get poor recall or F1-scores when the model is trained.
- The model might bias towards predicting customers who stayed as it seems more of those.

Exploratory Data Analysis

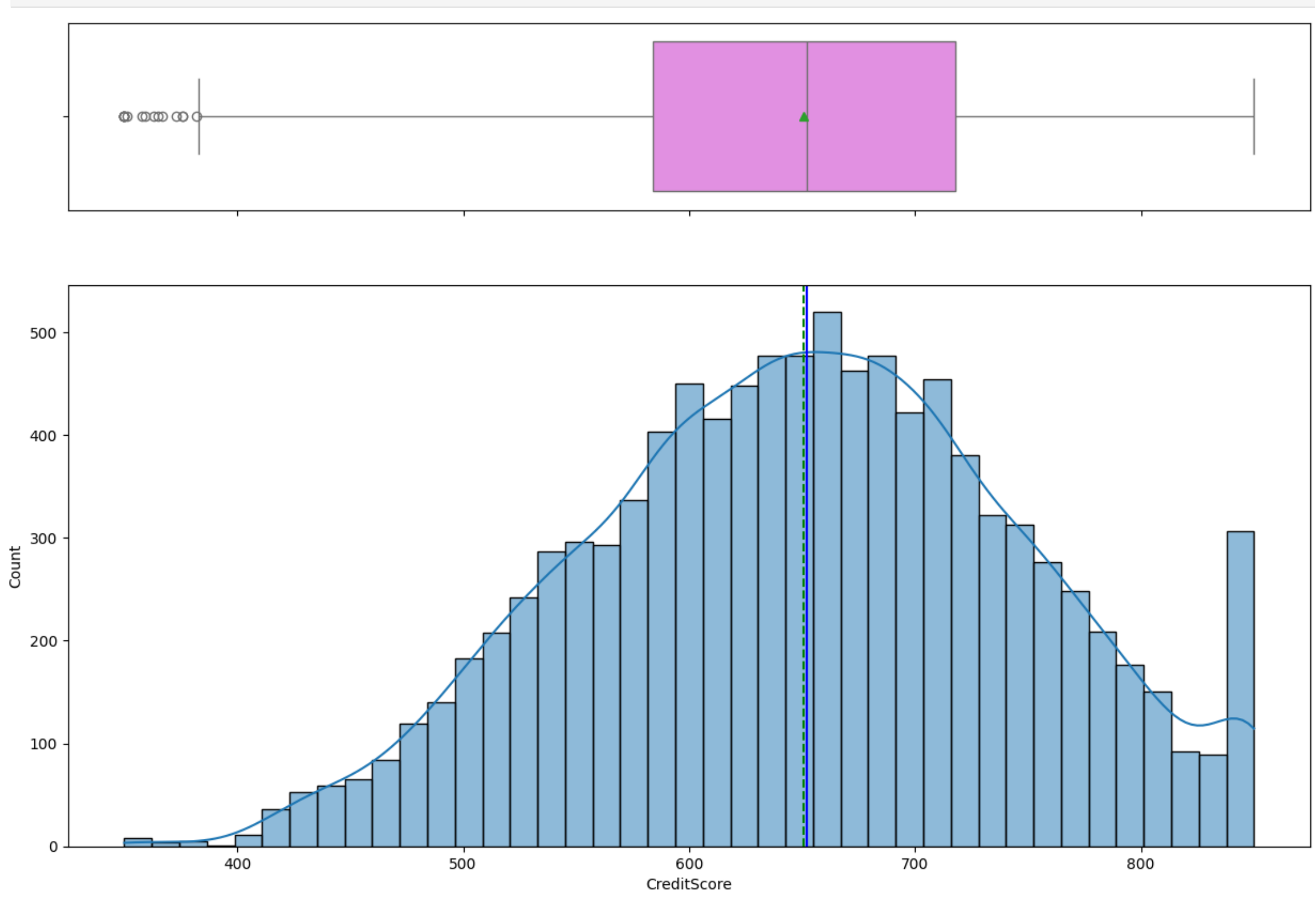
Univariate Analysis

```
#Function to plot both histogram and boxplot

def histogram_boxplot(data, feature, figsize=(15, 10), kde=False, bins=None):
    """
    Boxplot and histogram combined

    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (15,10))
    kde: whether to show the density curve (default False)
    bins: number of bins for histogram (default None)
    """
    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2,  # Number of rows of the subplot grid= 2
        sharex=True,  # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    )  # creating the 2 subplots
    sns.boxplot(
        data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
    )  # boxplot will be created and a triangle will indicate the mean value of the column
    sns.histplot(
        data=data, x=feature, kde=True, ax=ax_hist2, bins=bins
    ) if bins else sns.histplot(
        data=data, x=feature, kde=True, ax=ax_hist2
    )  # For histogram

    # Calculate and plot mean and median only if the feature is numeric
    if pd.api.types.is_numeric_dtype(data[feature]):
      ax_hist2.axvline(
          data[feature].mean(), color="green", linestyle="--"
      )  # Add mean to the histogram
      ax_hist2.axvline(
          data[feature].median(), color="blue", linestyle="-"
      )  # Add median to the histogram
    else:
      #add mode
      ax_hist2.axvline(
          data[feature].mode()[0], color="blue", linestyle="-"
      )
```

```
# function to create labeled barplots


def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is False)
    n: displays the top n category levels (default is None, i.e., display all levels)
    """

    total = len(data[feature])  # length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 2, 6))
    else:
        plt.figure(figsize=(n + 2, 6))

    plt.xticks(rotation=90, fontsize=15)
    ax = sns.countplot(
        data=data,
        x=feature,
        palette="Paired",
        order=data[feature].value_counts().index[:n],
    )

    for p in ax.patches:
        if perc == True:
            label = "{:.1f}%".format(
                100 * p.get_height() / total
            )  # percentage of each class of the category
        else:
            label = p.get_height()  # count of each level of the category

        x = p.get_x() + p.get_width() / 2  # width of the plot
        y = p.get_height()  # height of the plot

        ax.annotate(
            label,
            (x, y),
            ha="center",
            va="center",
            size=12,
            xytext=(0, 5),
            textcoords="offset points",
        )  # annotate the percentage

    plt.show()  # show the plot
```

Observations on CreditScore

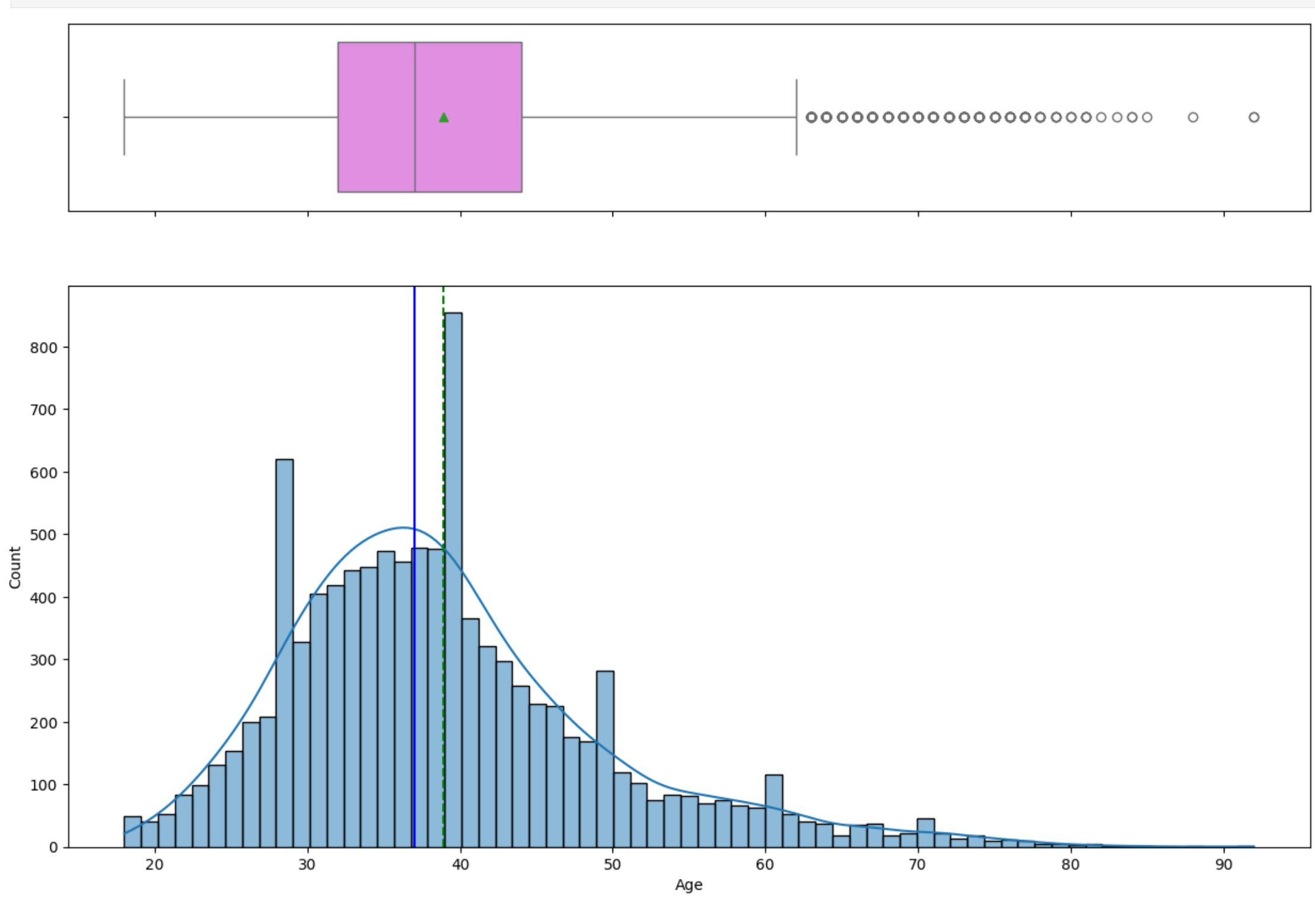
```
histogram_boxplot(df, 'CreditScore')
```



- The distribution is approximately bell-shaped (with a slight left skew).
- The bulk of the credit scores are around the middle range.
- The credit scores range between around 350 to 850.
- The Interquartile Range (IQR), shows that the middle 50% of customers have credit scores roughly between ~580 and ~720
- The boxplot clearly identifies several outliers with very low credit scores (below approximately 400). These are scores that fall significantly below the typical range for the majority of customers.
- There is a noticeable spike in credit scores that are exactly 850, indicating that a significant number of customers have top score.

Observations on Age

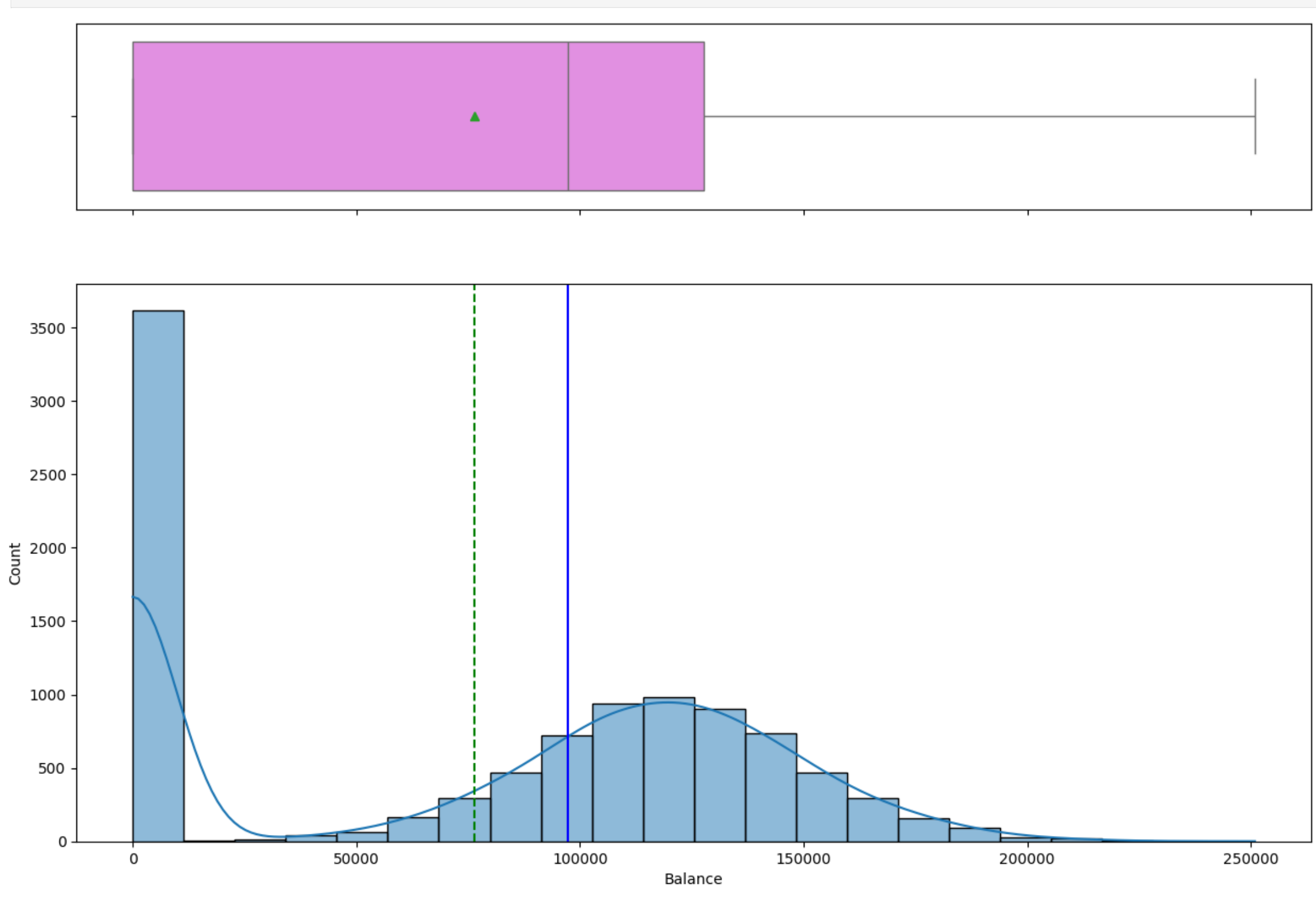
```
histogram_boxplot(df, 'Age')
```



- The distribution is right skewed. This means a significant portion of customers are in the younger to middle age range.
- The customer age ranges from 18 to over 90 years
- IQR, indicates that middle 50% of customers are aged between approximately 32 and 44 years.
- The boxplot indicates several outliers on the higher end of the age scale.

Observations on Balance

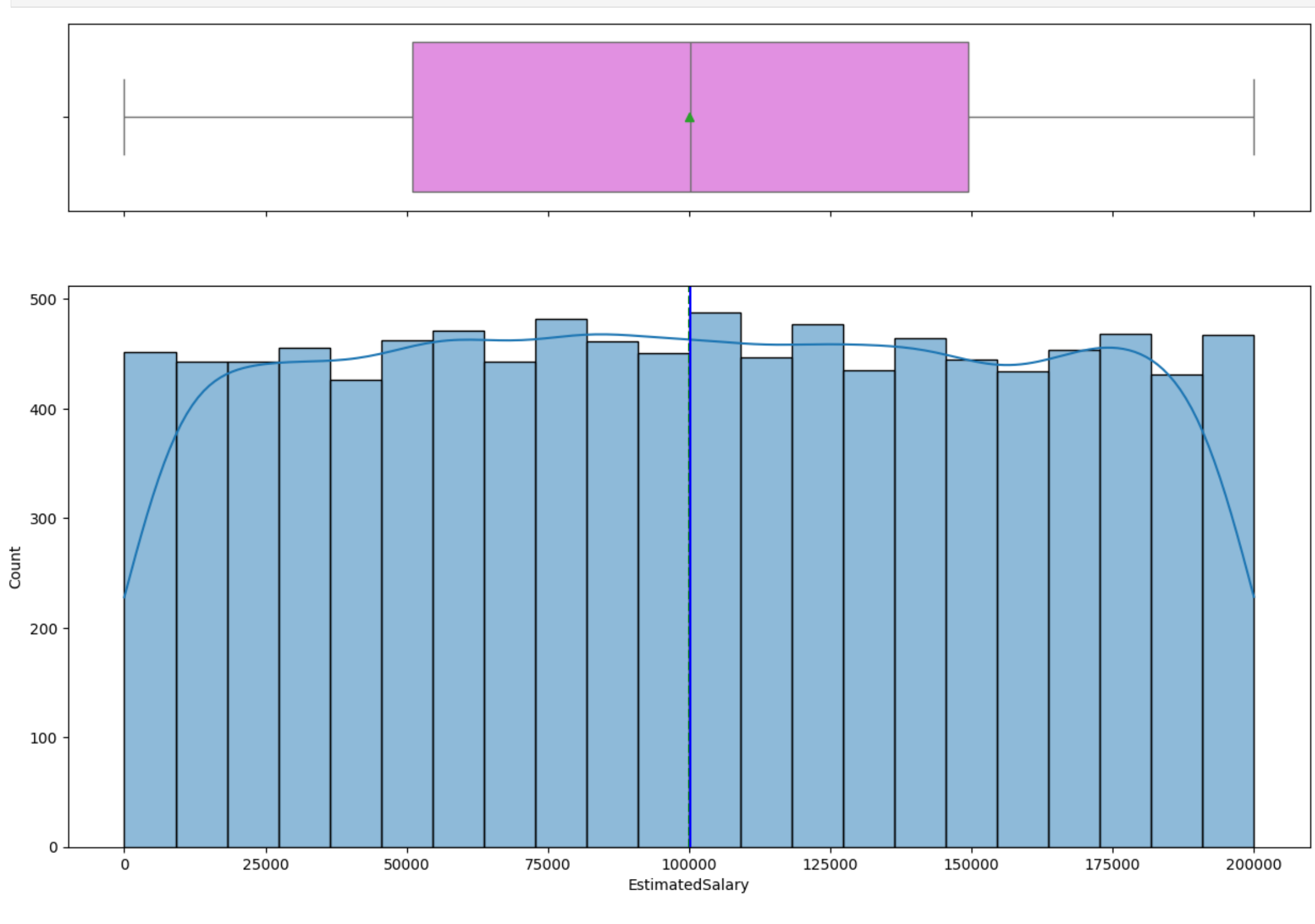
```
histogram_boxplot(df, 'Balance')
```



- The distribution is right skewed.
- There is a huge spike near zero, indicating that many accounts have very low or no balance.
- There are two peaks - the one near zero and another between approximately 120,000 & 130,000.
- The mean is pulled up high due to some high balance values (mean > median).
- No visible outliers observed.

Observations on Estimated Salary

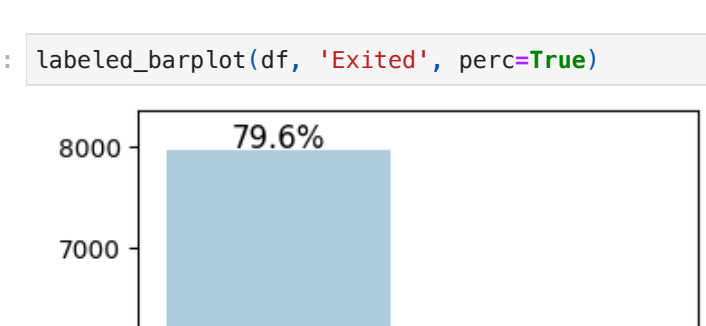
```
histogram_boxplot(df, 'EstimatedSalary')
```

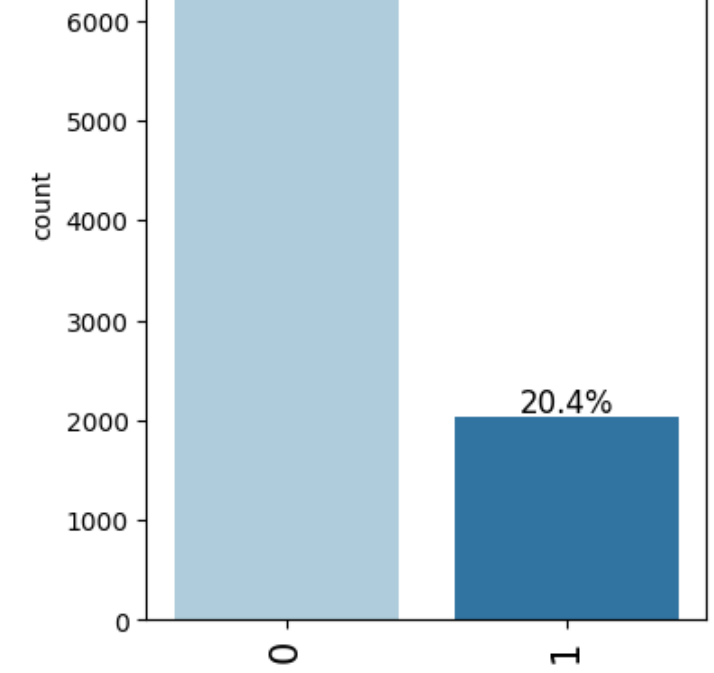


- The distribution is uniform and symmetric between (0 and 200,000)
- The mean and median are almost identical around 100,000

Observations on Exited

```
labeled_barplot(df, 'Exited', perc=True)
```

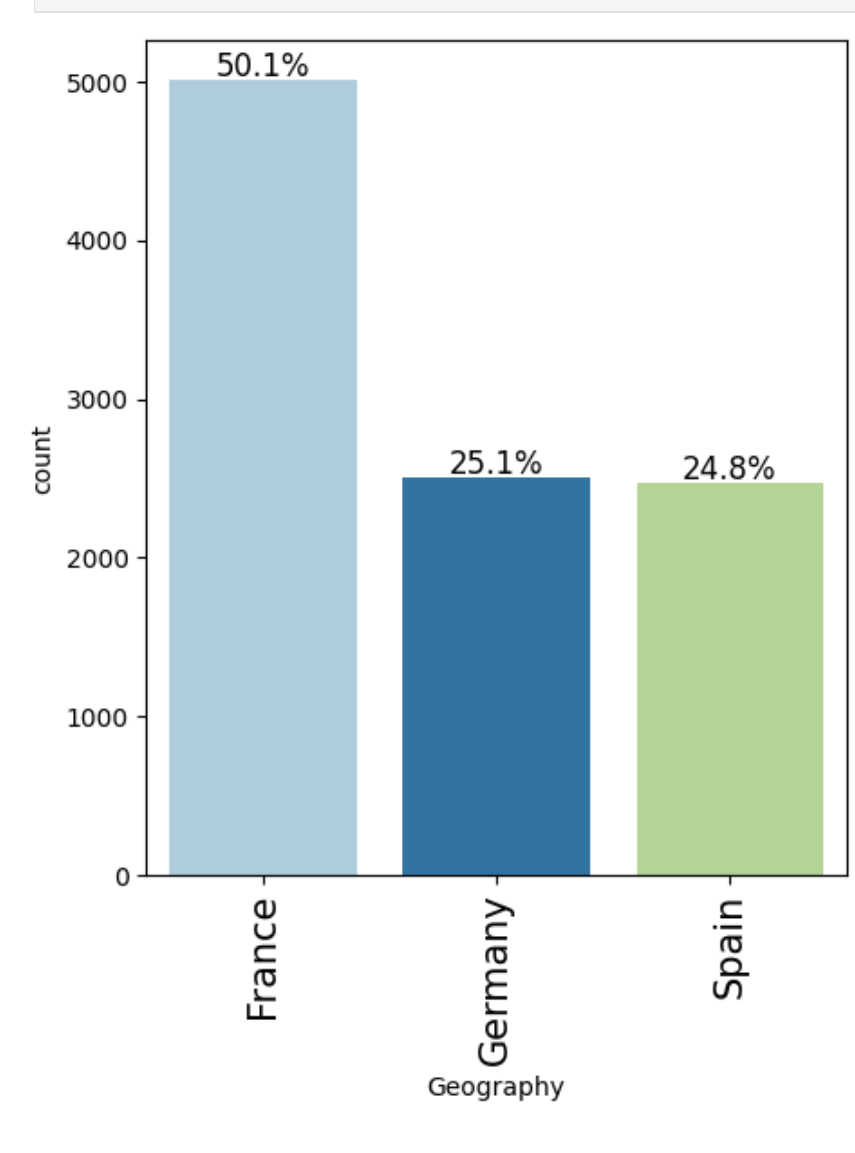




- 20% of the customer show that they have exited and needs more analysis.

Observations on Geography

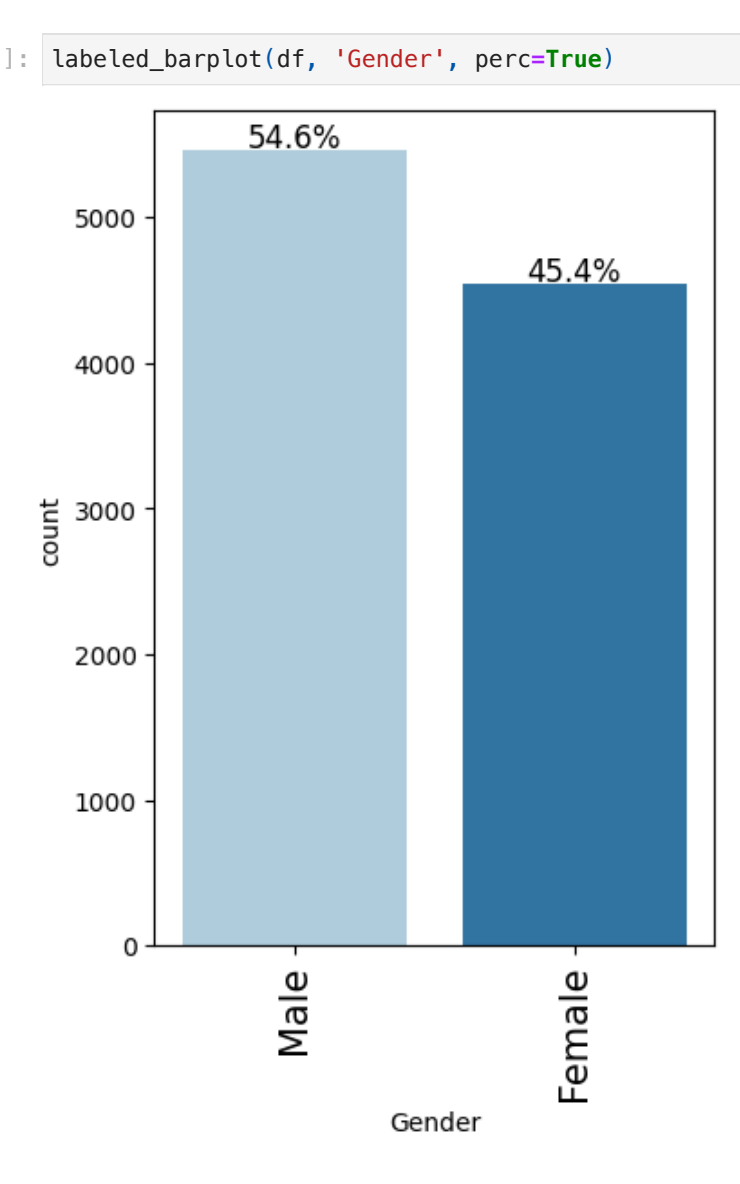
```
In [ ]: labeled_barplot(df, 'Geography', perc=True)
```



- Approximately 50% of the customers are from France, 25% from Germany and Spain each.

Observations on Gender

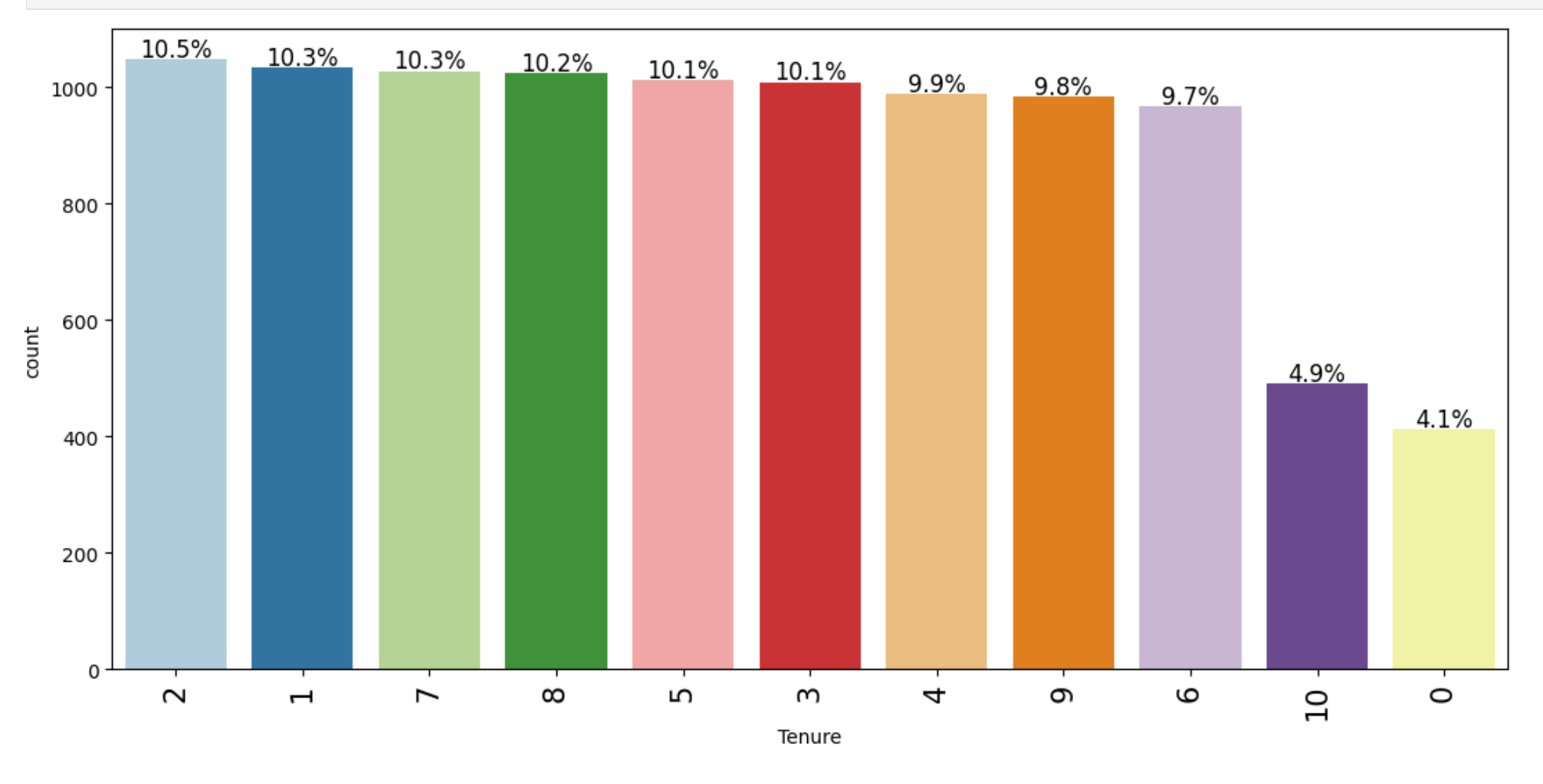
```
In [ ]: labeled_barplot(df, 'Gender', perc=True)
```



- 54.6% of customers are males and 45.4% are females.

Observations on Tenure

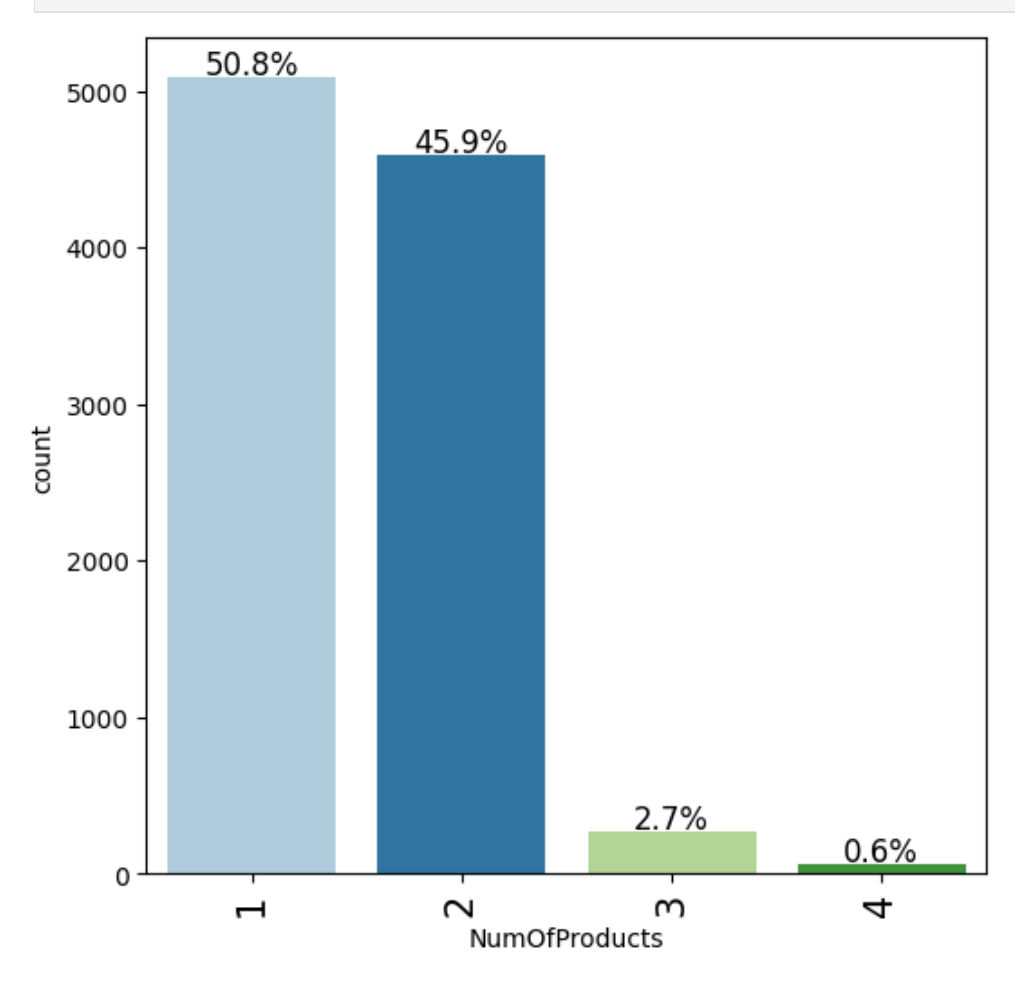
```
In [ ]: labeled_barplot(df, 'Tenure', perc=True)
```



- 4.1% of customers either did not take loans or finished repaying.
- Tenures of 1 to 9 has almost equal number of customers.

Observations on Number of Products

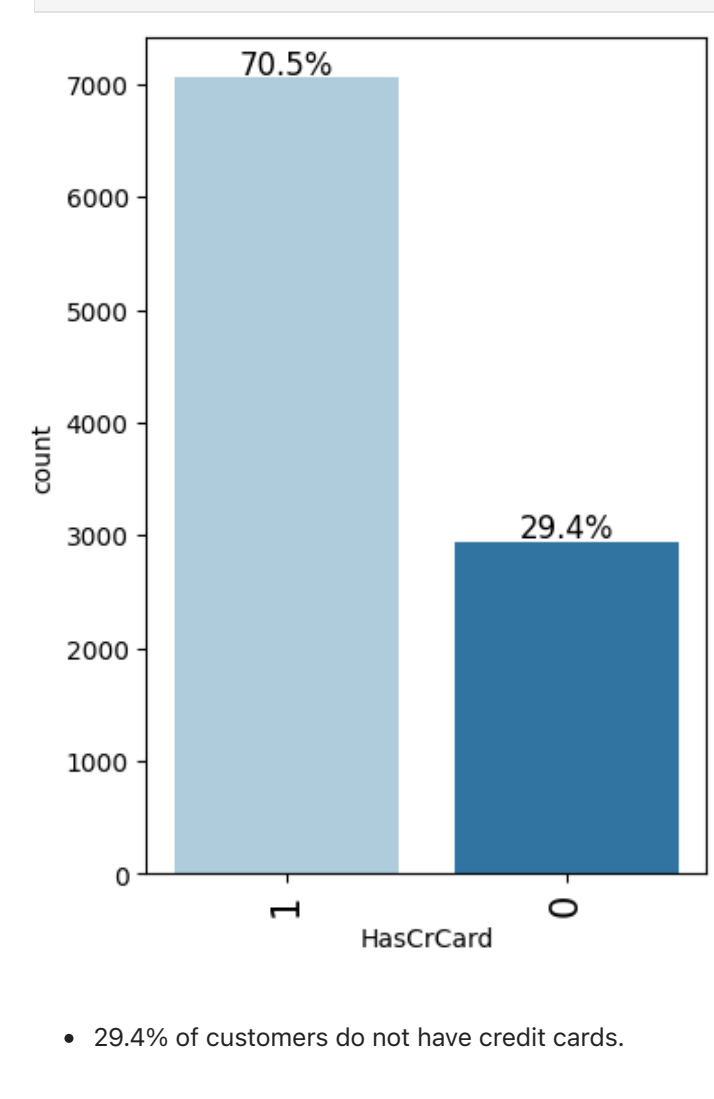
```
In [ ]: labeled_barplot(df, 'NumOfProducts', perc=True)
```



- 50.8% of customers have availed 1 product and 45.9% availed 2 products.
- The number of customers that have availed 3 or 4 products.

Observations on has Credit Card

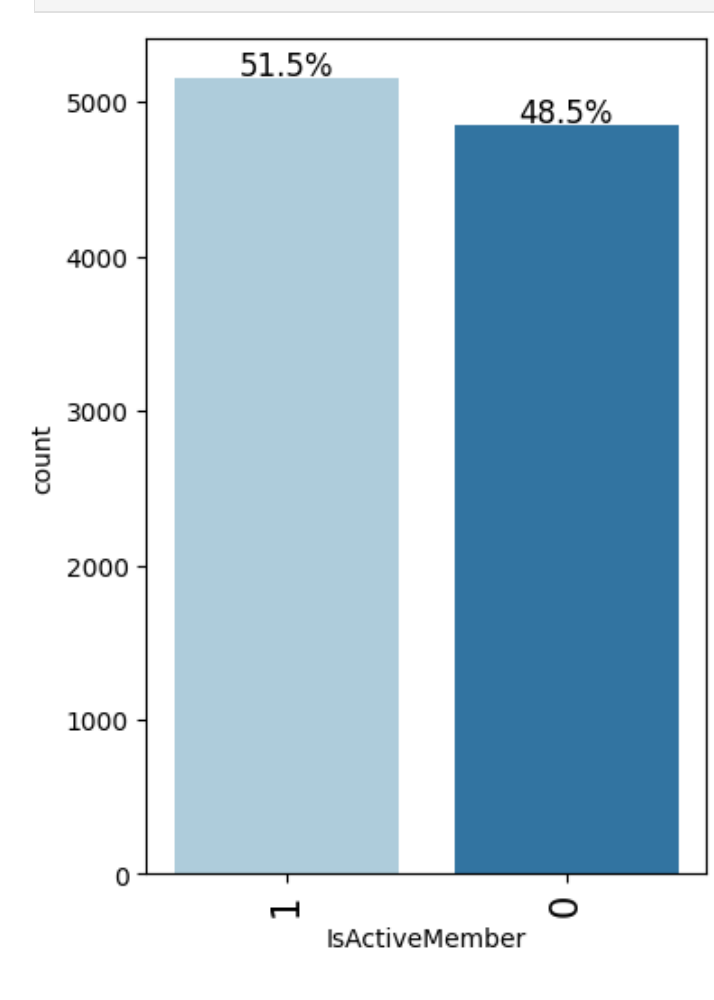
```
In [ ]: labeled_barplot(df, 'HasCrCard', perc=True)
```



- 29.4% of customers do not have credit cards.

Observations on Active Member

```
In [ ]: labeled_barplot(df, 'IsActiveMember', perc=True)
```



- Almost 50% customers are not active members.

Bivariate Analysis

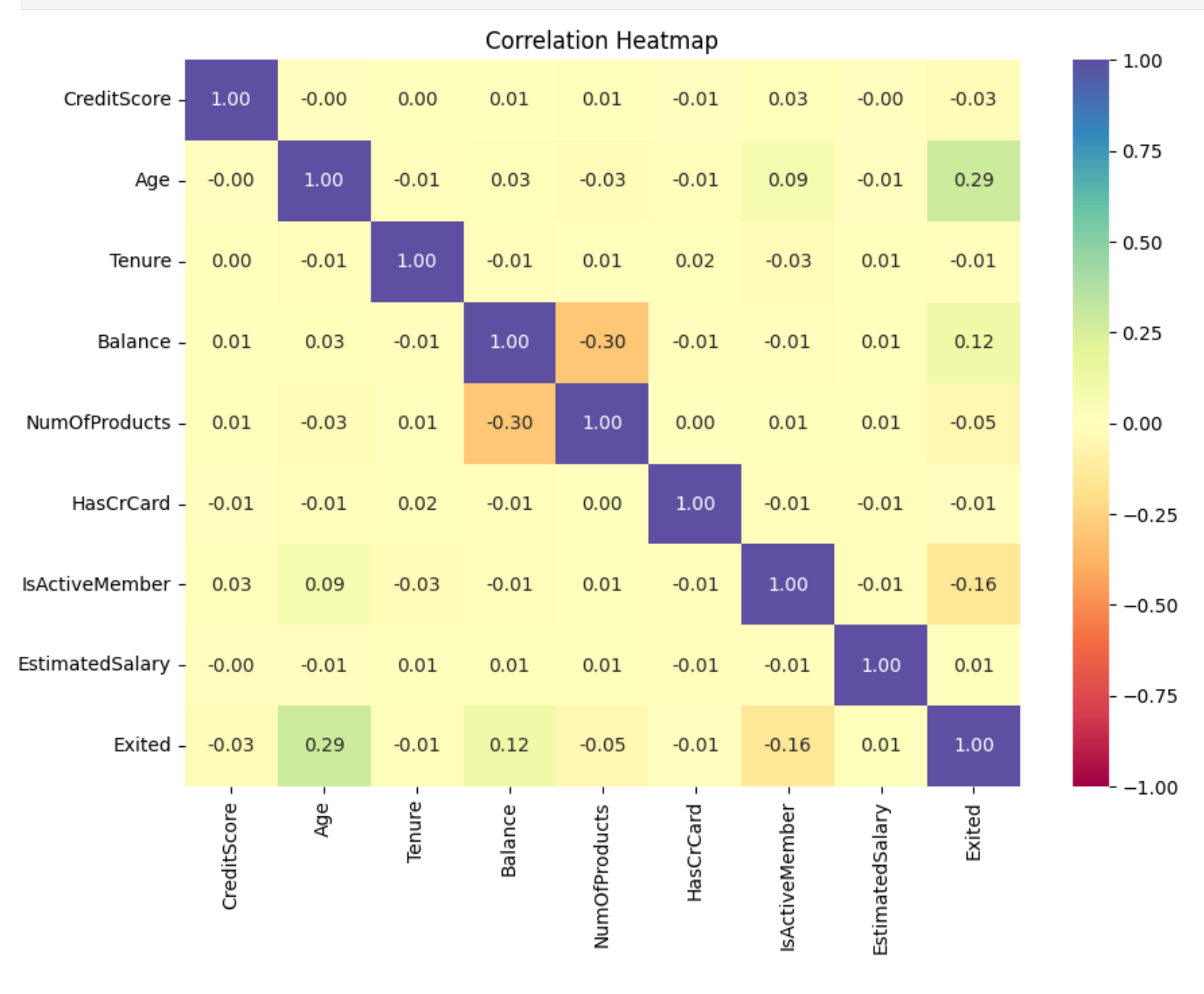
```
In [ ]: ### function to plot distributions wrt target


        def distribution_plot_wrt_target(data, predictor, target):

            fig, axs = plt.subplots(2, 2, figsize=(12, 10))

            target_uniq = data[target].unique()

            axs[0, 0].set_title("Distribution of target for target=" + str(target_uniq[0]))
            sns.histplot(
                data=data[data[target] == target_uniq[0]],
                x=predictor,
                kde=True,
                ax=axs[0, 0],
                color="teal",
                stat="density",
            )

            axs[0, 1].set_title("Distribution of target for target=" + str(target_uniq[1]))
            sns.histplot(
                data=data[data[target] == target_uniq[1]],
                x=predictor,
                kde=True,
                ax=axs[0, 1],
                color="orange",
                stat="density",
            )

            axs[1, 0].set_title("Boxplot w.r.t target")
            sns.boxplot(data=data, x=target, y=predictor, ax=axs[1, 0], palette="gist_rainbow")

            axs[1, 1].set_title("Boxplot (without outliers) w.r.t target")
            sns.boxplot(
                data=data,
                x=target,
                y=predictor,
                ax=axs[1, 1],
                showfliers=False,
                palette="gist_rainbow",
            )

            plt.tight_layout()
            plt.show()
```

```
In [ ]: ### function to boxplot distributions wrt target

        def distribution_boxplot_wrt_target(data, predictor, target):

            fig, axs = plt.subplots(1,2 , figsize=(12, 5))

            axs[0].set_title("Boxplot w.r.t target")
            sns.boxplot(data=data, x=target, y=predictor, ax=axs[0], palette="gist_rainbow")

            axs[1].set_title("Boxplot (without outliers) w.r.t target")
            sns.boxplot(
                data=data,
                x=target,
                y=predictor,
                ax=axs[1],
                showfliers=False,
                palette="gist_rainbow",
            )

            plt.tight_layout()
            plt.show()
```

```
In [ ]: def stacked_barplot(data, predictor, target):
            """
            Print the category counts and plot a stacked bar chart

            data: dataframe
            predictor: independent variable
            target: target variable
            """
            count = data[predictor].nunique()
            sorter = data[target].value_counts().index[-1]
            tab1 = pd.crosstab(data[predictor], data[target], margins=True).sort_values(
                by=sorter, ascending=False
            )
            print(tab1)
            print("-" * 120)
            tab = pd.crosstab(data[predictor], data[target], normalize="index").sort_values(
                by=sorter, ascending=False
            )
            tab.plot(kind="bar", stacked=True, figsize=(count + 5, 5))
            plt.legend(
                loc="lower left", frameon=False,
            )
            plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
            plt.show()
```

Correlation (Heat) Map

```
In [ ]: # plot correlation on numeric columns
        corr =  df.select_dtypes(include=['number']).corr()

        plt.figure(figsize=(10,7))

        sns.heatmap(corr, annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral")

        plt.title("Correlation Heatmap")
        plt.show()
```

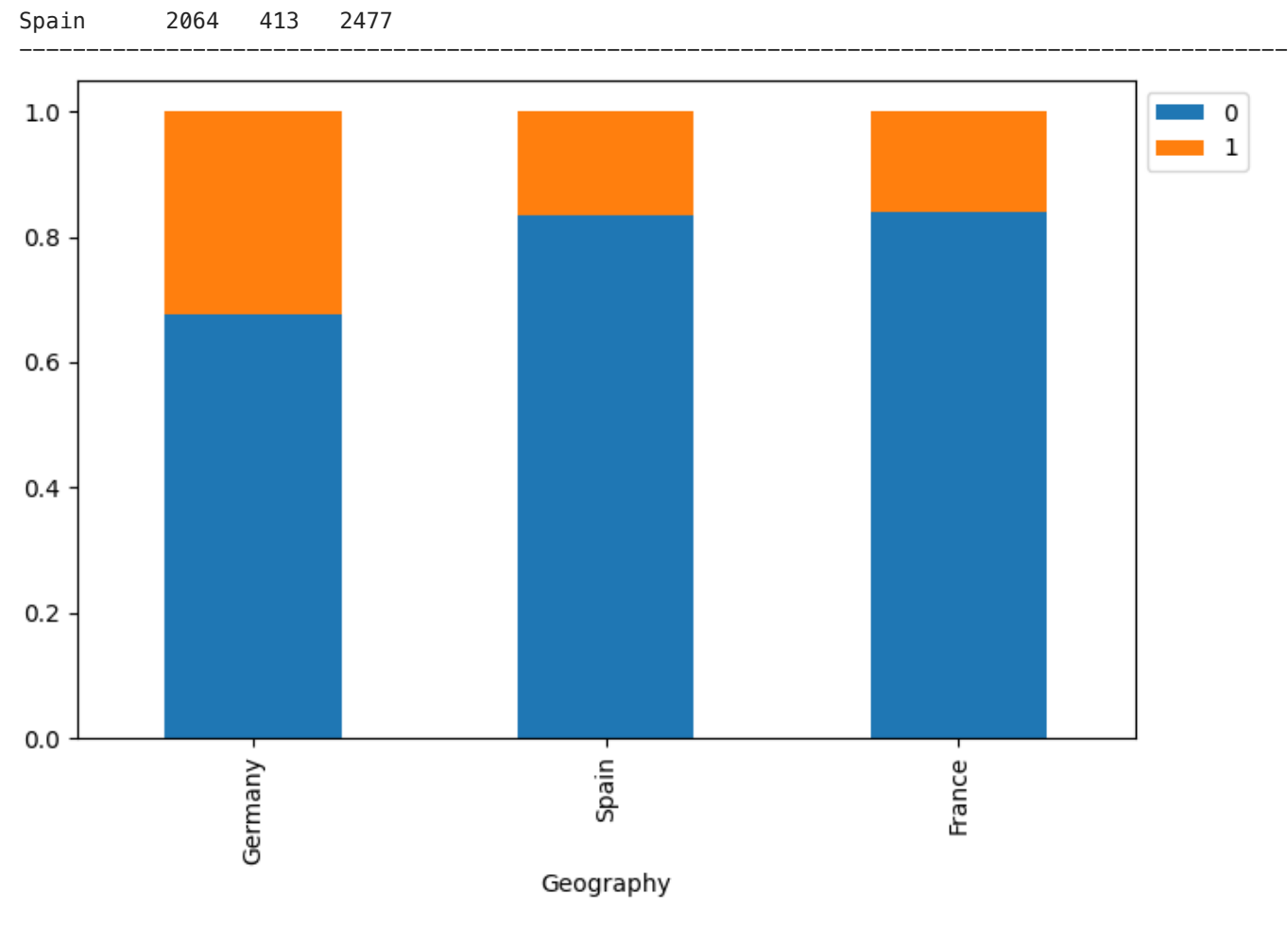


- No strong correlation is observed in the dataset.
- Age is observed to have 0.29 correlation with Exited

Exited vs Geography

```
In [ ]: stacked_barplot(df, 'Geography', 'Exited')
```

```
Exited        0     1    All
Geography
All        7963  2037  10000
Germany    1695   814   2509
France     4204   810   5014
Spain      2064   413   2477
------------------------------------------------------------------------------------------------------------------------
```

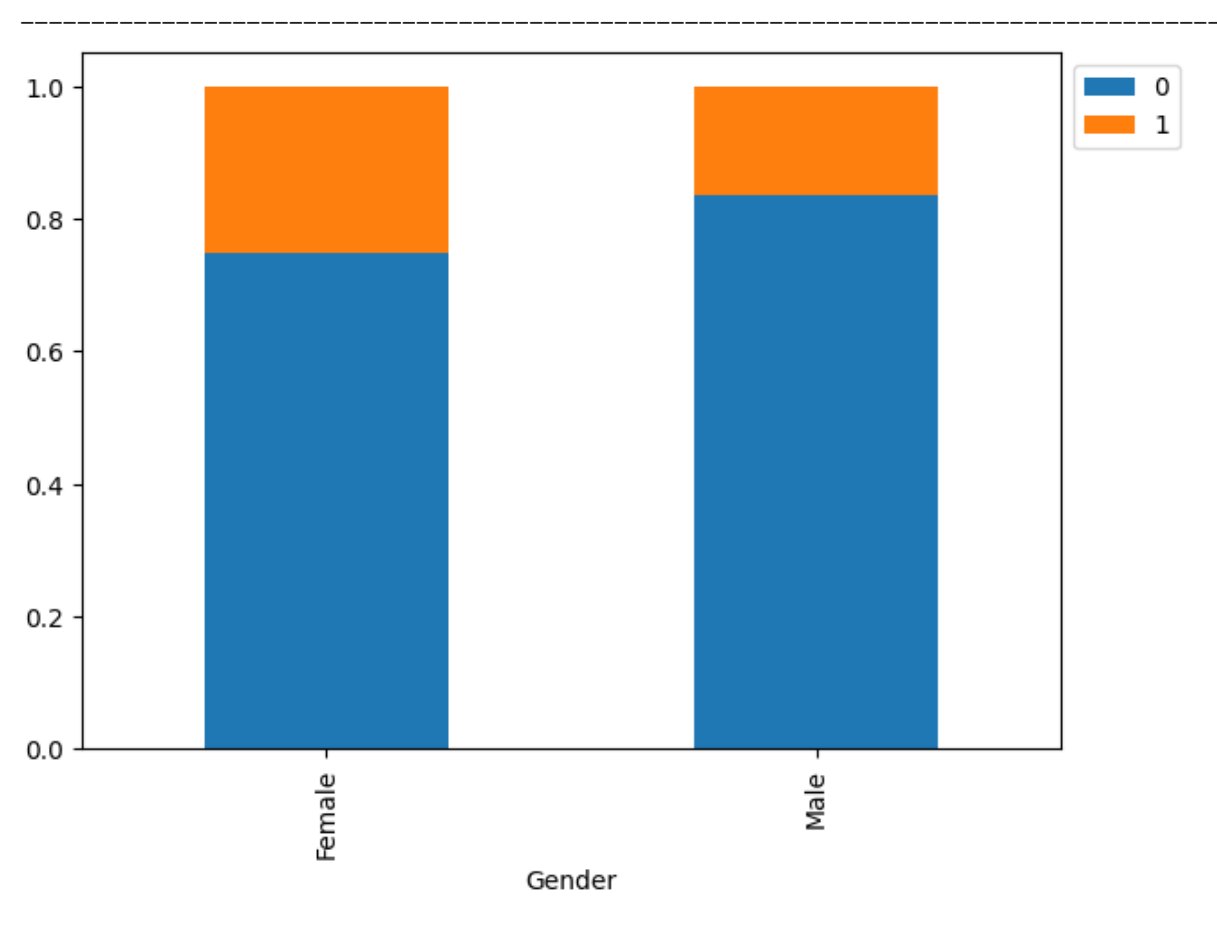


- Germany has more number of customers that have exited (almost 35%)
- France & Spain have very similar numbers for customers that have exited (almost 15%)

Exited vs Gender

```
In [ ]: stacked_barplot(df, 'Gender', 'Exited')
```

```
Exited     0     1    All
Gender
All     7963  2037  10000
Female  3404  1139   4543
Male    4559   898   5457
------------------------------------------------------------------------------------------------------------------------
```

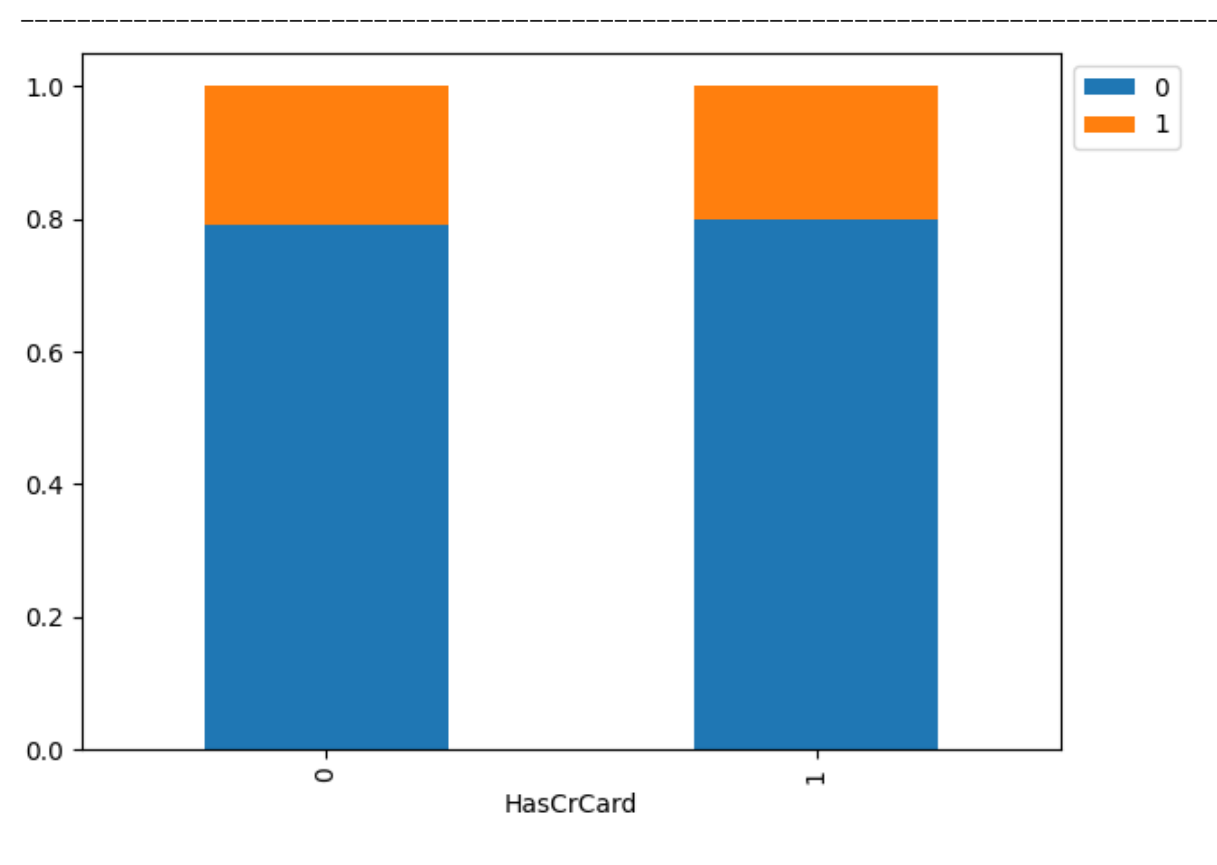


- Female customers (approx. 25%) seem to have exited more than Male customers (approx. 15%)

Exited vs has Credit Card

```
In [ ]: stacked_barplot(df, 'HasCrCard', 'Exited')
```

```
Exited        0     1    All
HasCrCard
All        7963  2037  10000
1          5631  1424   7055
0          2332   613   2945
------------------------------------------------------------------------------------------------------------------------
```

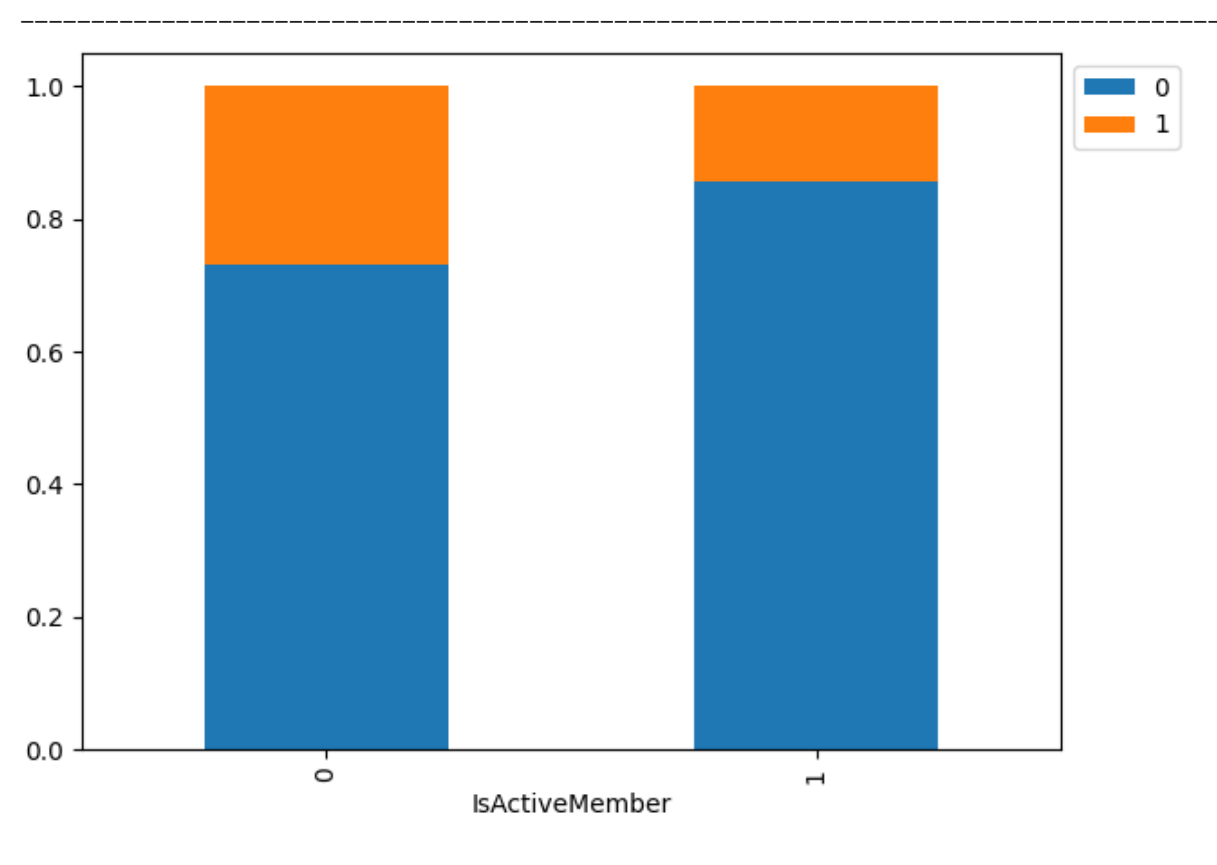


- The percentage of customers (almost 20%) that exited were the same irrespective of having a credit card.

Exited vs Is Active Member

```
In [ ]: stacked_barplot(df, 'IsActiveMember', 'Exited')
```

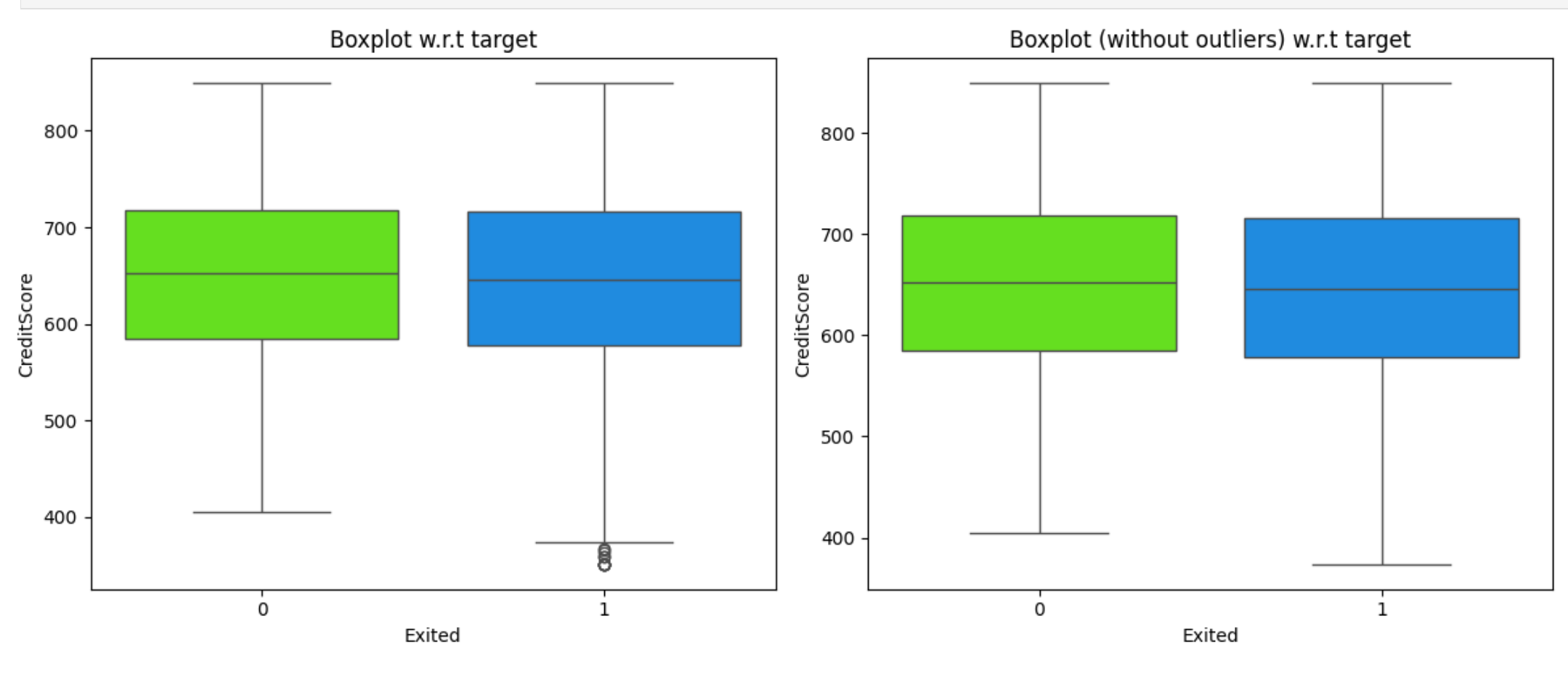
```
Exited             0     1    All
IsActiveMember
All             7963  2037  10000
0               3547  1302   4849
1               4416   735   5151
------------------------------------------------------------------------------------------------------------------------
```



- Inactive customers had more exits (almost 25%) than active customers (almost 15%)

Exited vs Credit Score

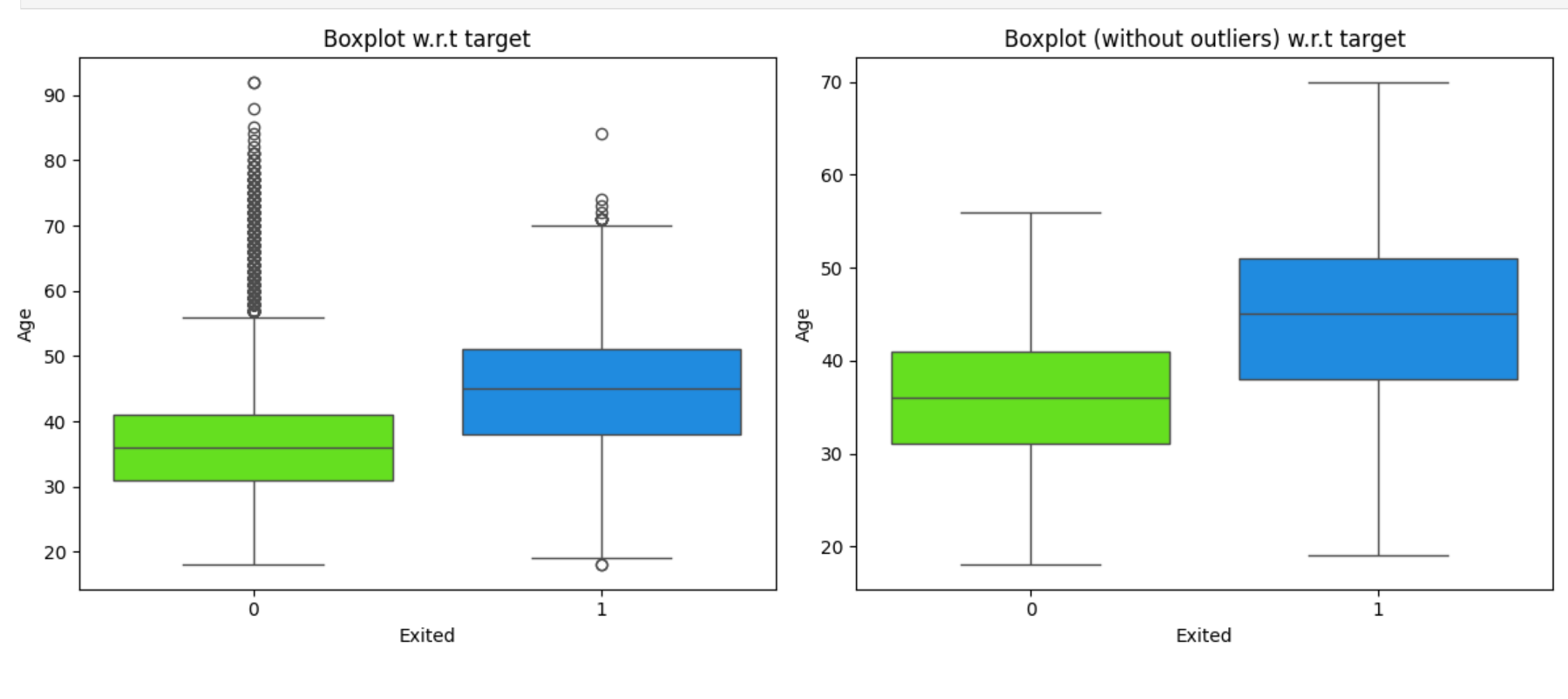
```
In [ ]: distribution_boxplot_wrt_target(df, 'CreditScore', 'Exited')
```



- IQR is similar for bothe groups
- Outliers are present in the exited group (left plot) - a cluster of very low credit scores
- With the outliers removed, the visual difference is minimal between the plots. This might suggest that credit score alone might not be a strong predictor of exit.

Exited vs Age

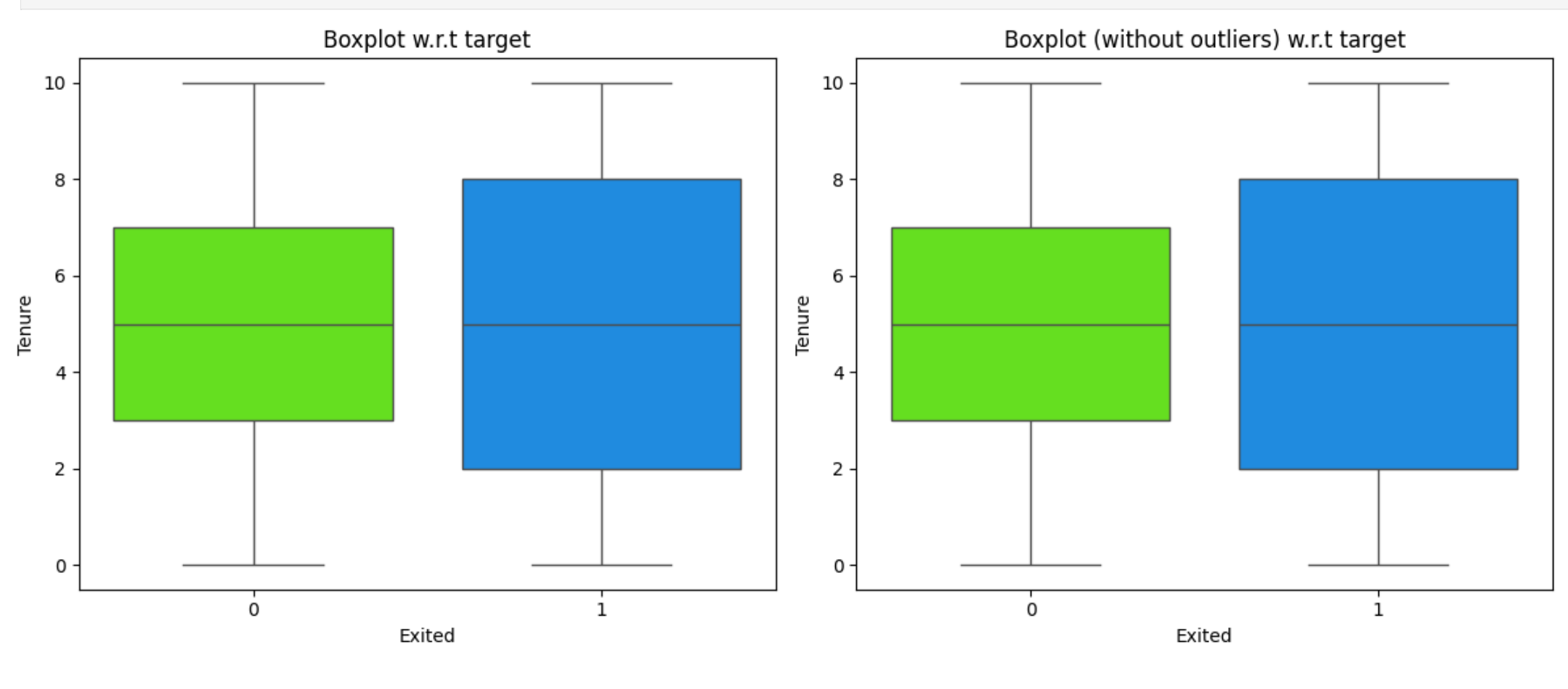
```
In [ ]: distribution_boxplot_wrt_target(df, 'Age', 'Exited')
```



- The median age of customers who exited is higher than those that stayed in both plots - with or without outliers.
- Large numbers of outliers are present in the non-exited customer group (between the ages of 60 and 90+).
- With the outliers removed, the median age is more pronounced.
- The customers who stayed clustered around the age range of 30s to 40s, while the customers who exited clustered around the age range of 40s to 50s.
- Age can be seen to have an impact on exiting - with older customers more likely to exit.

Exited vs Tenure

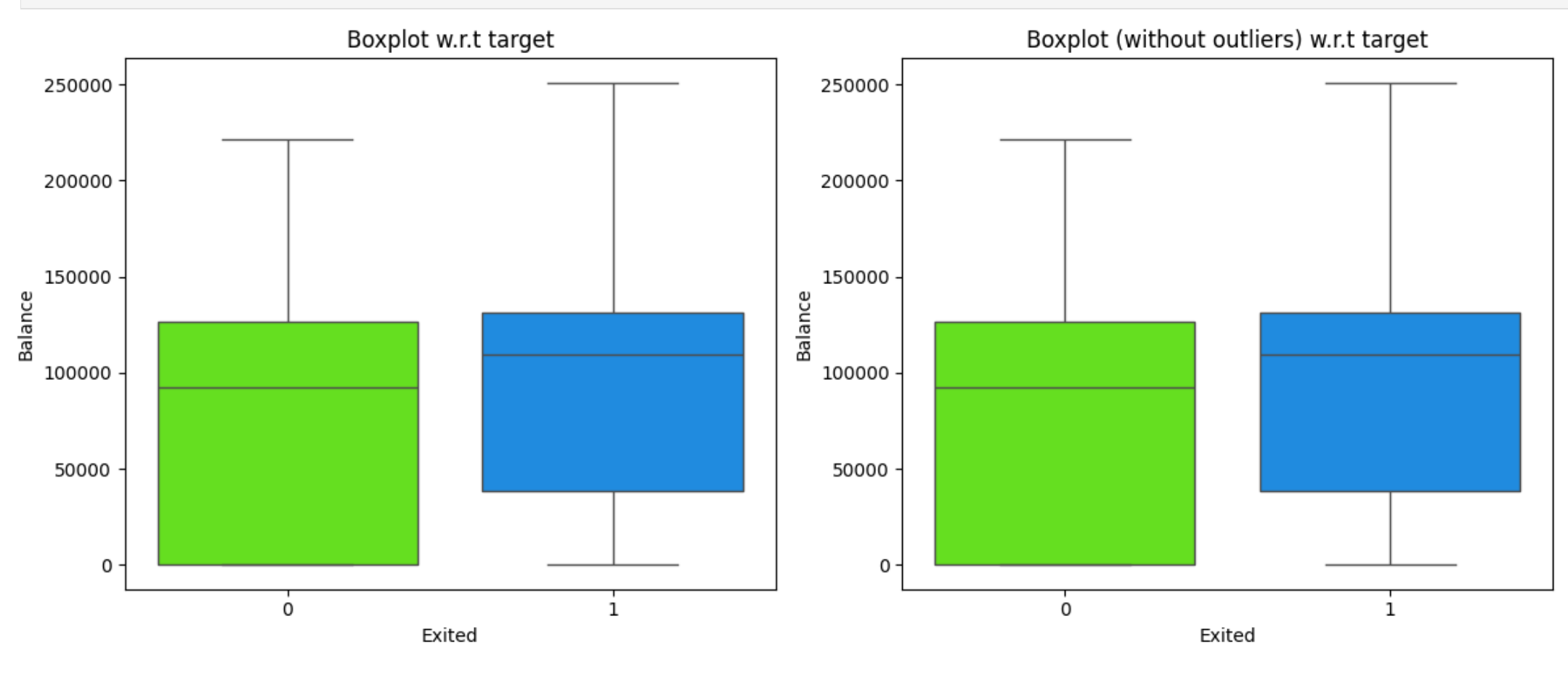
```
In [ ]: distribution_boxplot_wrt_target(df, 'Tenure', 'Exited')
```



- There is a lack of outliers observed in the plots.
- Tenure does not show a strong signal in indicating the customers exit.

Exited vs Balance

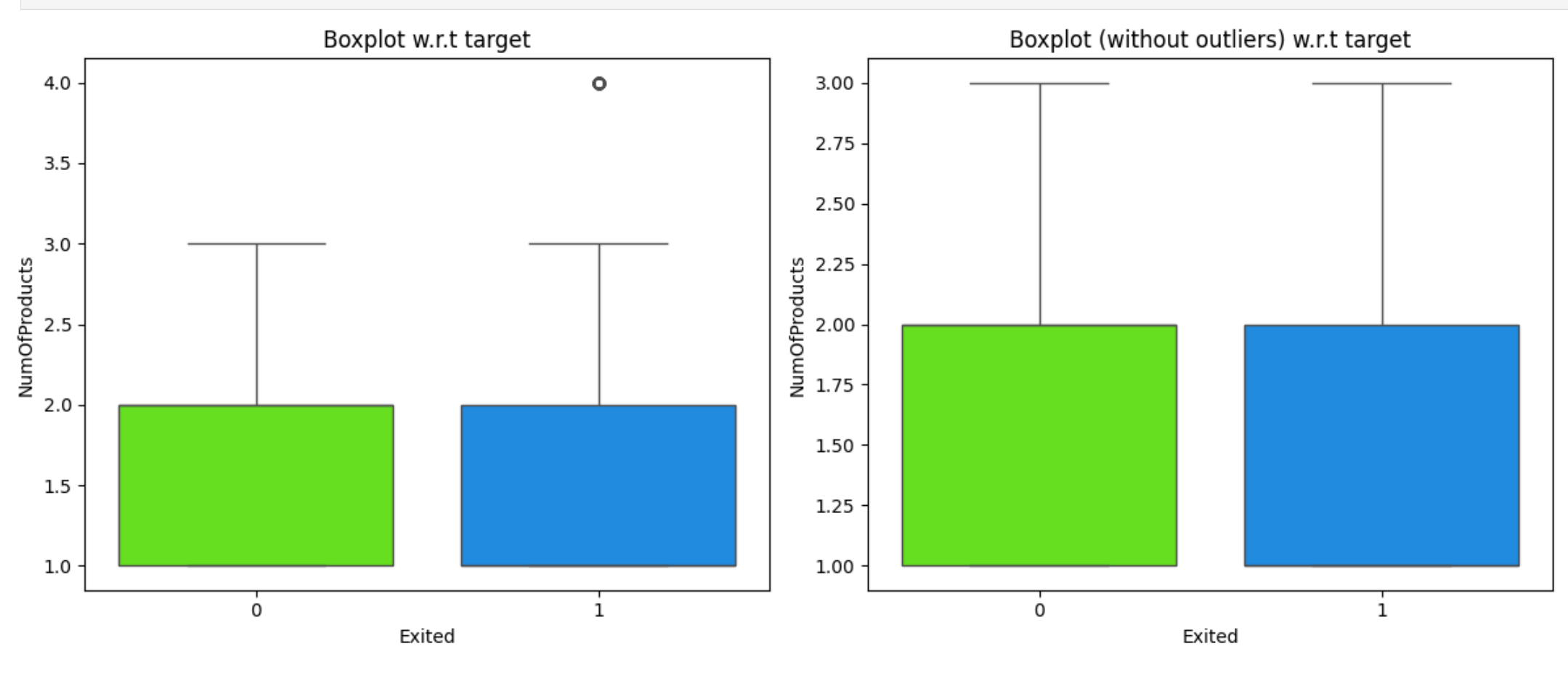
```
In [ ]: distribution_boxplot_wrt_target(df, 'Balance', 'Exited')
```



- There is a lack of outliers observed in the plots.
- Balance does not show a strong signal in indicating the customers exit.

Exited vs Number of Products

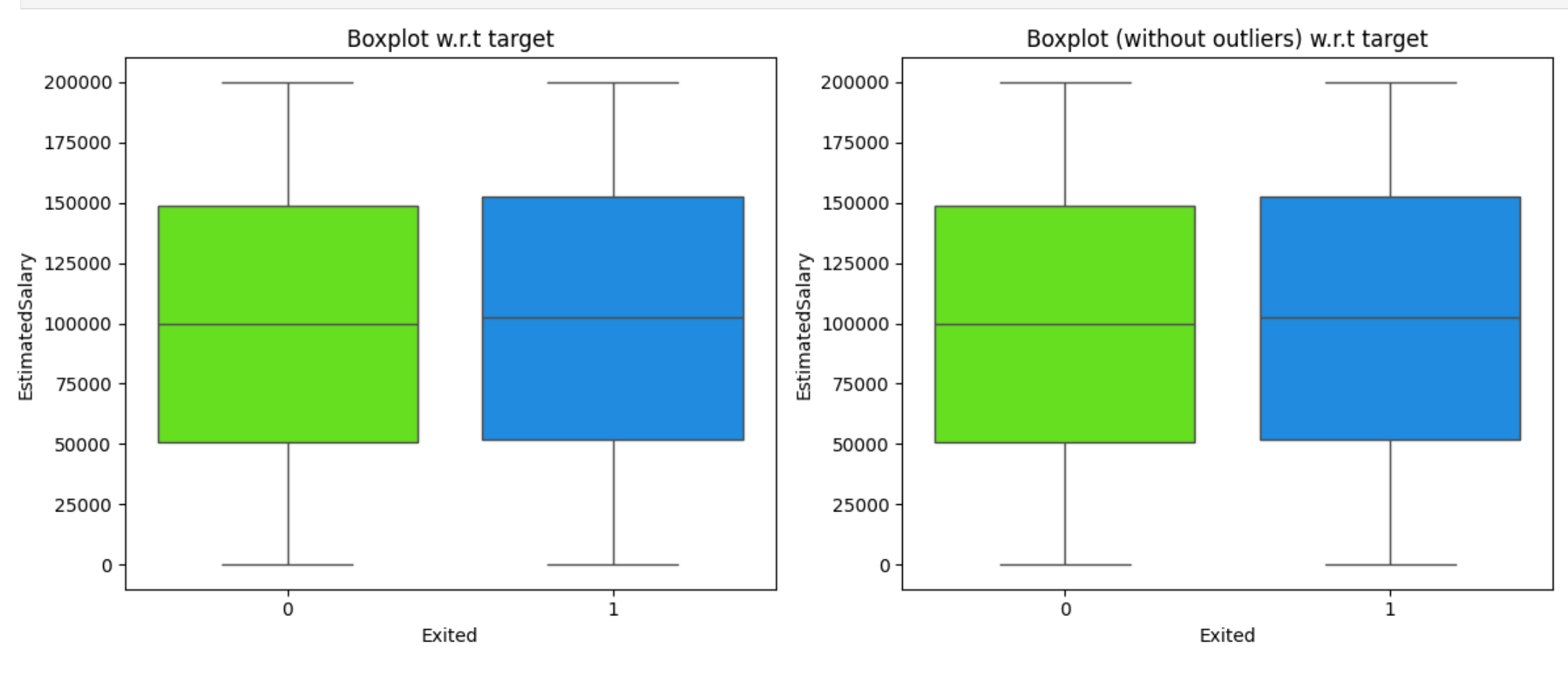
```
In [ ]: distribution_boxplot_wrt_target(df, 'NumOfProducts', 'Exited')
```



- There is an outlier observed in the plot.
- Number of products does not show a strong signal in indicating the customers exit.

Exited vs Estimated Salary

```
In [ ]: distribution_boxplot_wrt_target(df, 'EstimatedSalary', 'Exited')
```



- There is a lack of outliers observed in the plots.
- Estimated Salary does not show a strong signal in indicating the customers exit.

Data Preprocessing

Splitting into Train, Validation and Test

```
In [ ]: # Drop the Exited column
        X = df.drop(['Exited'], axis = 1)
        y = df['Exited']
```

```
In [ ]: # define RANDOM STATE
```

```
RANDOM_STATE = 42

# Splitting data into training (60%), validation (20%) and test (20%) set:

# first we split data into 2 parts, say temporary (80%) and test (20%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

# then we split the temporary set into train (75%) and validation (25%)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=RANDOM_STATE, stratify=y_temp
)
```

Shape of the split dataset

```
In [ ]: # Create a dictionary for dataset shapes
        shape_data = {
            "Dataset": ["Training Set", "Validation Set", "Testing Set"],
            "Shape": [X_train.shape, X_val.shape, X_test.shape]
        }

        # Create a DataFrame for class distributions
        class_dist = {
            "Training Set": y_train.value_counts(normalize=True).round(4),
            "Validation Set": y_val.value_counts(normalize=True).round(4),
            "Testing Set": y_test.value_counts(normalize=True).round(4)
        }
        class_dist_df = pd.DataFrame(class_dist)

        # Convert shape tuple to a string for display
        shape_df = pd.DataFrame(shape_data)
        shape_df["Shape"] = shape_df["Shape"].astype(str)

        # Transpose class distribution to match the shape table format
        class_dist_df = class_dist_df.T.reset_index().rename(columns={"index": "Dataset"})

        # Merge both tables
        final_df = pd.merge(shape_df, class_dist_df, on="Dataset", how="left")

        # Display the final table
        print("\nSplit Dataset Overview:")
        print(final_df.to_string(index=False))
```

```
Split Dataset Overview:
       Dataset       Shape     0     1
  Training Set  (6000, 10) 0.796 0.204
Validation Set  (2000, 10) 0.796 0.203
   Testing Set  (2000, 10) 0.796 0.203
```

Dummy Variable Creation

```
In [ ]: #List the object type variables in df
        cat_cols = X.select_dtypes(include=['object']).columns
        cat_cols
```

```
Out[ ]: Index(['Geography', 'Gender'], dtype='object')
```

```
In [ ]: # Encode 'Geography' and 'Gender' in Training, Validation and Testing splits
        # And convert them to float.
        X_train = pd.get_dummies(X_train, columns=cat_cols, drop_first=True).astype(float)

        X_val = pd.get_dummies(X_val, columns=cat_cols, drop_first=True).astype(float)

        X_test = pd.get_dummies(X_test, columns=cat_cols, drop_first=True).astype(float)

        print(X_train.shape, X_val.shape, X_test.shape)

        #print(X_train)
```

```
(6000, 11) (2000, 11) (2000, 11)
```

Data Normalization

- Scale all the numeric values to bring them to the same scale

```
In [ ]: #List all the numeric columns
        num_cols = ["CreditScore","Age","Tenure","Balance","EstimatedSalary"]

        #Instance of the standard scaler
        sc = StandardScaler()

        X_train[num_cols] = sc.fit_transform(X_train[num_cols])
        X_val[num_cols] = sc.transform(X_val[num_cols])
        X_test[num_cols] = sc.transform(X_test[num_cols])
```

Missing value treatment

- Previously it was observed that no values were missing and hence we can skip this step.

Model Building

Model Evaluation Criterion

- **F1-score** would have been the best metric to be evaluated.
  - **False negatives** (FN) could be expensive and might lose a retention chance.
  - **False positives** (FP) might lead to unnecessary efforts, but less costly.
  - As the models were built, F1-scores were seen to be flat lines since it is a harmonic mean. We will choose a different metric.
- Customer churn is often imbalanced and hence Accuracy would be the wrong metric.
- Precision and Recall too could be considered as other candidates.
- In our case **we will choose Recall** (to measure true positives and minimize costlier false negatives)

Function for plotting confusion matrix

```
In [ ]: def make_confusion_matrix(actual_targets, predicted_targets):
            """
            To plot the confusion_matrix with percentages

            actual_targets: actual target (dependent) variable values
            predicted_targets: predicted target (dependent) variable values
            """
            cm = confusion_matrix(actual_targets, predicted_targets)
            labels = np.asarray(
                [
                    ["{0:0.0f}".format(item) + "\n{0:.2%}".format(item / cm.flatten().sum())]
                    for item in cm.flatten()
                ]
            ).reshape(cm.shape[0], cm.shape[1])

            plt.figure(figsize=(6, 4))
            sns.heatmap(cm, annot=labels, fmt="")
            plt.ylabel("True label")
            plt.xlabel("Predicted label")
```

```
In [ ]: # Blank dataframes to store Recall Scores
        train_scores_df = pd.DataFrame(columns=["Recall"])
        val_scores_df = pd.DataFrame(columns=["Recall"])
        test_scores_df = pd.DataFrame(columns=["Recall"])
```

Neural Network with SGD Optimizer

```
In [ ]: # Initialization
        backend.clear_session()
        np.random.seed(RANDOM_STATE)
        random.seed(RANDOM_STATE)
        tf.random.set_seed(RANDOM_STATE)
```

```
In [ ]: # Initialize the neural network with SGD Optimizer
        model_sgd = Sequential()

        # Add an input layer with 128 neurons and Relu as the activation function
        model_sgd.add(Dense(units=128, activation='relu', input_shape=(X_train.shape[1],)))

        # Add an hidden layer with 64 neurons and Relu as the activation function
        model_sgd.add(Dense(units=64, activation='relu'))

        # Add an output layer with 1 neuron and Sigmoid as the activation function
        # as this is a binary classification problem
        model_sgd.add(Dense(units=1, activation='sigmoid'))
```

```
In [ ]: # Use SGD as the optimizer
        optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)

        #Choose Precision, Recall and F1 score as the metrics
        metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]

        # Compile the model with loss function of binary cross entropy and F1score as the metric
        model_sgd.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

```
In [ ]: model_sgd.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,536
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 9,857 (38.50 KB)
Trainable params: 9,857 (38.50 KB)
Non-trainable params: 0 (0.00 B)

```
In [ ]: start = time.time()

        history_sgd = model_sgd.fit(
        X_train, y_train,
        batch_size=32,
        validation_data=(X_val,y_val),
        epochs=100,
        verbose=1
        )

        end=time.time()
```

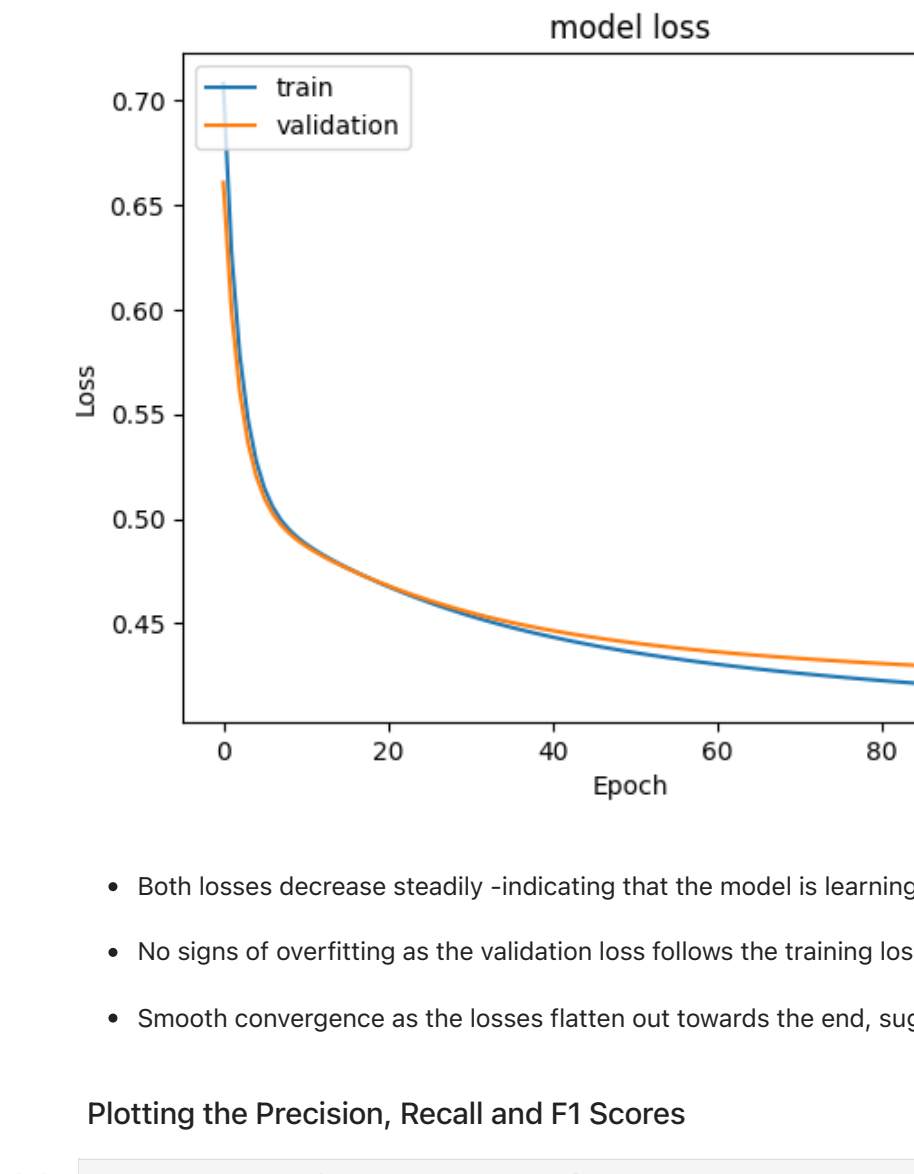
```
Epoch 1/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.3364 - loss: 0.7325 - precision: 0.1609 - recall: 0.5011 - val_f1_score: 0.3382 - val_loss: 0.6607 - val_precision: 0.1626 - val_recall: 0.1474
Epoch 2/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.6402 - precision: 0.1810 - recall: 0.0726 - val_f1_score: 0.3382 - val_loss: 0.5990 - val_precision: 0.4000 - val_recall: 0.0049
Epoch 3/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.5853 - precision: 0.7231 - recall: 0.0025 - val_f1_score: 0.3382 - val_loss: 0.5607 - val_precision: 0.0000e+00 - val_recall: 0.000
0e+00
Epoch 4/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.5508 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.5362 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 5/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.5286 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.5203 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 6/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.5141 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.5097 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 7/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.5043 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.5024 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 8/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.4974 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4971 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 9/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4924 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4930 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 10/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4886 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4898 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 11/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4854 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4870 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 12/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4827 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4845 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 13/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4803 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4823 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 14/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4781 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4802 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 15/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4761 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4782 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 16/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4741 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4763 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 17/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4723 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4746 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 18/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4705 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4728 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 19/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.4688 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4712 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 20/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.4671 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4696 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 21/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4655 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4680 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 22/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4640 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4665 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 23/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.3364 - loss: 0.4625 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4651 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 24/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4611 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4637 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 25/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4597 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4623 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 26/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4583 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4610 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 27/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4570 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4598 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 28/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4557 - precision: 0.0000e+00 - recall: 0.0000e+00 - val_f1_score: 0.3382 - val_loss: 0.4586 - val_precision: 0.0000e+00 - val_recal
l: 0.0000e+00
Epoch 29/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4545 - precision: 0.2222 - recall: 4.7365e-04 - val_f1_score: 0.3382 - val_loss: 0.4574 - val_precision: 0.0000e+00 - val_recall:
0.0000e+00
Epoch 30/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.4533 - precision: 0.2222 - recall: 4.7365e-04 - val_f1_score: 0.3382 - val_loss: 0.4563 - val_precision: 0.0000e+00 - val_recall:
0.0000e+00
Epoch 31/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4521 - precision: 0.2222 - recall: 4.7365e-04 - val_f1_score: 0.3382 - val_loss: 0.4552 - val_precision: 0.0000e+00 - val_recall:
0.0000e+00
Epoch 32/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4510 - precision: 0.3889 - recall: 0.0013 - val_f1_score: 0.3382 - val_loss: 0.4542 - val_precision: 0.0000e+00 - val_recall: 0.000
0e+00
Epoch 33/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4500 - precision: 0.3889 - recall: 0.0013 - val_f1_score: 0.3382 - val_loss: 0.4532 - val_precision: 0.0000e+00 - val_recall: 0.000
0e+00
Epoch 34/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4489 - precision: 0.7478 - recall: 0.0046 - val_f1_score: 0.3382 - val_loss: 0.4522 - val_precision: 0.0000e+00 - val_recall: 0.000
0e+00
Epoch 35/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4480 - precision: 0.7022 - recall: 0.0061 - val_f1_score: 0.3382 - val_loss: 0.4513 - val_precision: 0.0000e+00 - val_recall: 0.000
0e+00
Epoch 36/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4470 - precision: 0.7393 - recall: 0.0070 - val_f1_score: 0.3382 - val_loss: 0.4504 - val_precision: 0.7500 - val_recall: 0.0074
Epoch 37/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4461 - precision: 0.7092 - recall: 0.0071 - val_f1_score: 0.3382 - val_loss: 0.4496 - val_precision: 0.5000 - val_recall: 0.0074
Epoch 38/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4451 - precision: 0.5382 - recall: 0.0085 - val_f1_score: 0.3382 - val_loss: 0.4488 - val_precision: 0.3750 - val_recall: 0.0074
Epoch 39/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4443 - precision: 0.5319 - recall: 0.0096 - val_f1_score: 0.3382 - val_loss: 0.4480 - val_precision: 0.4545 - val_recall: 0.0123
Epoch 40/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.4434 - precision: 0.4843 - recall: 0.0117 - val_f1_score: 0.3382 - val_loss: 0.4472 - val_precision: 0.4615 - val_recall: 0.0147
Epoch 41/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.4426 - precision: 0.5595 - recall: 0.0156 - val_f1_score: 0.3382 - val_loss: 0.4465 - val_precision: 0.4286 - val_recall: 0.0147
Epoch 42/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4418 - precision: 0.5572 - recall: 0.0164 - val_f1_score: 0.3382 - val_loss: 0.4458 - val_precision: 0.5789 - val_recall: 0.0270
Epoch 43/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4410 - precision: 0.5937 - recall: 0.0243 - val_f1_score: 0.3382 - val_loss: 0.4452 - val_precision: 0.5789 - val_recall: 0.0270
Epoch 44/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4403 - precision: 0.6676 - recall: 0.0349 - val_f1_score: 0.3382 - val_loss: 0.4445 - val_precision: 0.5909 - val_recall: 0.0319
Epoch 45/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4396 - precision: 0.6201 - recall: 0.0359 - val_f1_score: 0.3382 - val_loss: 0.4439 - val_precision: 0.5652 - val_recall: 0.0319
Epoch 46/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4388 - precision: 0.6531 - recall: 0.0403 - val_f1_score: 0.3382 - val_loss: 0.4433 - val_precision: 0.6296 - val_recall: 0.0418
Epoch 47/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4382 - precision: 0.6657 - recall: 0.0421 - val_f1_score: 0.3382 - val_loss: 0.4427 - val_precision: 0.6071 - val_recall: 0.0418
Epoch 48/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4375 - precision: 0.6840 - recall: 0.0458 - val_f1_score: 0.3382 - val_loss: 0.4421 - val_precision: 0.5667 - val_recall: 0.0418
Epoch 49/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4368 - precision: 0.6867 - recall: 0.0490 - val_f1_score: 0.3382 - val_loss: 0.4416 - val_precision: 0.5625 - val_recall: 0.0442
Epoch 50/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4362 - precision: 0.6862 - recall: 0.0514 - val_f1_score: 0.3382 - val_loss: 0.4411 - val_precision: 0.5714 - val_recall: 0.0491
Epoch 51/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4356 - precision: 0.6893 - recall: 0.0614 - val_f1_score: 0.3382 - val_loss: 0.4406 - val_precision: 0.5946 - val_recall: 0.0541
Epoch 52/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.4350 - precision: 0.6905 - recall: 0.0693 - val_f1_score: 0.3382 - val_loss: 0.4401 - val_precision: 0.5789 - val_recall: 0.0541
Epoch 53/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4344 - precision: 0.6789 - recall: 0.0709 - val_f1_score: 0.3382 - val_loss: 0.4396 - val_precision: 0.5897 - val_recall: 0.0565
Epoch 54/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4339 - precision: 0.6677 - recall: 0.0716 - val_f1_score: 0.3382 - val_loss: 0.4392 - val_precision: 0.6190 - val_recall: 0.0639
Epoch 55/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4333 - precision: 0.6793 - recall: 0.0751 - val_f1_score: 0.3382 - val_loss: 0.4387 - val_precision: 0.5778 - val_recall: 0.0639
Epoch 56/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4328 - precision: 0.6739 - recall: 0.0806 - val_f1_score: 0.3382 - val_loss: 0.4383 - val_precision: 0.5870 - val_recall: 0.0663
Epoch 57/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4323 - precision: 0.6748 - recall: 0.0837 - val_f1_score: 0.3382 - val_loss: 0.4379 - val_precision: 0.5833 - val_recall: 0.0688
Epoch 58/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4318 - precision: 0.6775 - recall: 0.0867 - val_f1_score: 0.3382 - val_loss: 0.4375 - val_precision: 0.6000 - val_recall: 0.0737
Epoch 59/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4313 - precision: 0.6879 - recall: 0.0907 - val_f1_score: 0.3382 - val_loss: 0.4371 - val_precision: 0.6038 - val_recall: 0.0786
Epoch 60/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4308 - precision: 0.6905 - recall: 0.0942 - val_f1_score: 0.3382 - val_loss: 0.4367 - val_precision: 0.6250 - val_recall: 0.0860
Epoch 61/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4303 - precision: 0.6848 - recall: 0.0984 - val_f1_score: 0.3382 - val_loss: 0.4364 - val_precision: 0.6271 - val_recall: 0.0909
Epoch 62/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.4299 - precision: 0.6877 - recall: 0.1042 - val_f1_score: 0.3382 - val_loss: 0.4360 - val_precision: 0.6349 - val_recall: 0.0983
Epoch 63/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4294 - precision: 0.6789 - recall: 0.1085 - val_f1_score: 0.3382 - val_loss: 0.4357 - val_precision: 0.6406 - val_recall: 0.1007
Epoch 64/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4290 - precision: 0.6880 - recall: 0.1130 - val_f1_score: 0.3382 - val_loss: 0.4353 - val_precision: 0.6462 - val_recall: 0.1032
Epoch 65/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4285 - precision: 0.7035 - recall: 0.1199 - val_f1_score: 0.3382 - val_loss: 0.4350 - val_precision: 0.6567 - val_recall: 0.1081
Epoch 66/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4281 - precision: 0.7022 - recall: 0.1209 - val_f1_score: 0.3382 - val_loss: 0.4347 - val_precision: 0.6522 - val_recall: 0.1106
Epoch 67/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4277 - precision: 0.7029 - recall: 0.1244 - val_f1_score: 0.3382 - val_loss: 0.4344 - val_precision: 0.6575 - val_recall: 0.1179
Epoch 68/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4273 - precision: 0.7095 - recall: 0.1319 - val_f1_score: 0.3382 - val_loss: 0.4341 - val_precision: 0.6533 - val_recall: 0.1204
Epoch 69/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4269 - precision: 0.7074 - recall: 0.1361 - val_f1_score: 0.3382 - val_loss: 0.4338 - val_precision: 0.6579 - val_recall: 0.1229
Epoch 70/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4265 - precision: 0.7124 - recall: 0.1400 - val_f1_score: 0.3382 - val_loss: 0.4335 - val_precision: 0.6579 - val_recall: 0.1229
Epoch 71/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4261 - precision: 0.7107 - recall: 0.1420 - val_f1_score: 0.3382 - val_loss: 0.4332 - val_precision: 0.6456 - val_recall: 0.1253
Epoch 72/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4258 - precision: 0.7127 - recall: 0.1445 - val_f1_score: 0.3382 - val_loss: 0.4330 - val_precision: 0.6625 - val_recall: 0.1302
Epoch 73/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.4254 - precision: 0.7239 - recall: 0.1459 - val_f1_score: 0.3382 - val_loss: 0.4327 - val_precision: 0.6667 - val_recall: 0.1327
Epoch 74/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.3364 - loss: 0.4251 - precision: 0.7170 - recall: 0.1501 - val_f1_score: 0.3382 - val_loss: 0.4324 - val_precision: 0.6627 - val_recall: 0.1351
Epoch 75/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4247 - precision: 0.7286 - recall: 0.1595 - val_f1_score: 0.3382 - val_loss: 0.4322 - val_precision: 0.6588 - val_recall: 0.1376
Epoch 76/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4244 - precision: 0.7177 - recall: 0.1595 - val_f1_score: 0.3382 - val_loss: 0.4319 - val_precision: 0.6552 - val_recall: 0.1400
Epoch 77/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4240 - precision: 0.7162 - recall: 0.1612 - val_f1_score: 0.3382 - val_loss: 0.4317 - val_precision: 0.6552 - val_recall: 0.1400
Epoch 78/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4237 - precision: 0.7226 - recall: 0.1636 - val_f1_score: 0.3382 - val_loss: 0.4315 - val_precision: 0.6552 - val_recall: 0.1400
Epoch 79/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4234 - precision: 0.7167 - recall: 0.1636 - val_f1_score: 0.3382 - val_loss: 0.4312 - val_precision: 0.6591 - val_recall: 0.1425
Epoch 80/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4230 - precision: 0.7236 - recall: 0.1693 - val_f1_score: 0.3382 - val_loss: 0.4310 - val_precision: 0.6591 - val_recall: 0.1425
Epoch 81/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4227 - precision: 0.7140 - recall: 0.1709 - val_f1_score: 0.3382 - val_loss: 0.4308 - val_precision: 0.6667 - val_recall: 0.1474
Epoch 82/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.4224 - precision: 0.7131 - recall: 0.1733 - val_f1_score: 0.3382 - val_loss: 0.4306 - val_precision: 0.6703 - val_recall: 0.1499
Epoch 83/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.4221 - precision: 0.7108 - recall: 0.1759 - val_f1_score: 0.3382 - val_loss: 0.4304 - val_precision: 0.6739 - val_recall: 0.1523
Epoch 84/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.4218 - precision: 0.7143 - recall: 0.1789 - val_f1_score: 0.3382 - val_loss: 0.4302 - val_precision: 0.6774 - val_recall: 0.1548
Epoch 85/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3364 - loss: 0.4215 - precision: 0.7162 - recall: 0.1804 - val_f1_score: 0.3382 - val_loss: 0.4300 - val_precision: 0.6774 - val_recall: 0.1548
Epoch 86/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4212 - precision: 0.7161 - recall: 0.1839 - val_f1_score: 0.3382 - val_loss: 0.4298 - val_precision: 0.6809 - val_recall: 0.1572
Epoch 87/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4210 - precision: 0.7111 - recall: 0.1874 - val_f1_score: 0.3382 - val_loss: 0.4296 - val_precision: 0.6875 - val_recall: 0.1622
Epoch 88/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4207 - precision: 0.7091 - recall: 0.1911 - val_f1_score: 0.3382 - val_loss: 0.4294 - val_precision: 0.7000 - val_recall: 0.1720
Epoch 89/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4204 - precision: 0.7154 - recall: 0.1915 - val_f1_score: 0.3382 - val_loss: 0.4292 - val_precision: 0.7059 - val_recall: 0.1769
Epoch 90/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4201 - precision: 0.7150 - recall: 0.1916 - val_f1_score: 0.3382 - val_loss: 0.4290 - val_precision: 0.7059 - val_recall: 0.1769
Epoch 91/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4199 - precision: 0.7137 - recall: 0.1932 - val_f1_score: 0.3382 - val_loss: 0.4289 - val_precision: 0.7129 - val_recall: 0.1769
Epoch 92/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4196 - precision: 0.7142 - recall: 0.1936 - val_f1_score: 0.3382 - val_loss: 0.4287 - val_precision: 0.7228 - val_recall: 0.1794
Epoch 93/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4193 - precision: 0.7187 - recall: 0.1944 - val_f1_score: 0.3382 - val_loss: 0.4285 - val_precision: 0.7228 - val_recall: 0.1794
Epoch 94/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.4191 - precision: 0.7206 - recall: 0.1962 - val_f1_score: 0.3382 - val_loss: 0.4284 - val_precision: 0.7184 - val_recall: 0.1818
Epoch 95/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.3364 - loss: 0.4188 - precision: 0.7182 - recall: 0.1981 - val_f1_score: 0.3382 - val_loss: 0.4282 - val_precision: 0.7143 - val_recall: 0.1843
Epoch 96/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4186 - precision: 0.7165 - recall: 0.2014 - val_f1_score: 0.3382 - val_loss: 0.4280 - val_precision: 0.7170 - val_recall: 0.1867
Epoch 97/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4183 - precision: 0.7107 - recall: 0.2024 - val_f1_score: 0.3382 - val_loss: 0.4279 - val_precision: 0.7170 - val_recall: 0.1867
Epoch 98/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4181 - precision: 0.7110 - recall: 0.2026 - val_f1_score: 0.3382 - val_loss: 0.4277 - val_precision: 0.7238 - val_recall: 0.1867
Epoch 99/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4179 - precision: 0.7116 - recall: 0.2030 - val_f1_score: 0.3382 - val_loss: 0.4276 - val_precision: 0.7308 - val_recall: 0.1867
Epoch 100/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.3364 - loss: 0.4176 - precision: 0.6930 - recall: 0.2045 - val_f1_score: 0.3382 - val_loss: 0.4274 - val_precision: 0.7308 - val_recall: 0.1867
```

```
In [ ]: print("Time taken in seconds ",end-start)
```

```
Time taken in seconds  127.89241433143616
```

Plotting Training & Validation Loss

```
In [ ]: #Plotting Train Loss vs Validation Loss
        plt.plot(history_sgd.history['loss'])
        plt.plot(history_sgd.history['val_loss'])
        plt.title('model loss')
        plt.ylabel('Loss')
        plt.xlabel('Epoch')
        plt.legend(['train', 'validation'], loc='upper left')
        plt.show()
```

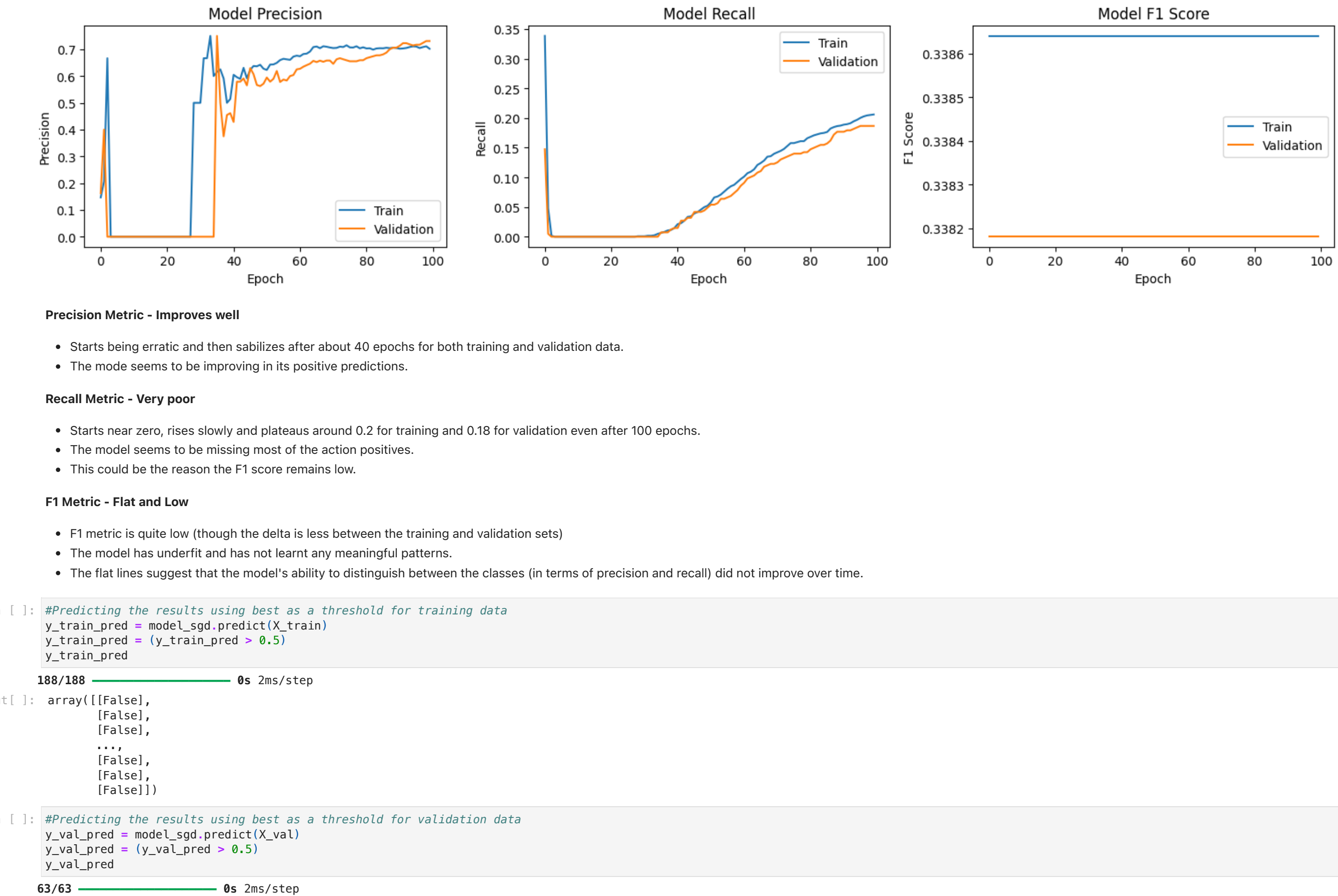


- Both losses decrease steadily -indicating that the model is learning over time — the optimizer is effectively minimizing the loss.
- No signs of overfitting as the validation loss follows the training loss closely, without diverging.
- Smooth convergence as the losses flatten out towards the end, suggesting the model has nearly converged.

Plotting the Precision, Recall and F1 Scores

```
In [ ]: def plot_metrics(history, metrics=["precision", "recall", "f1_score"]):
            """
            Plots training and validation metrics over epochs.

            Parameters:
            - history: Keras History object (e.g., history.history)
            - metrics: List of metric names to plot (default: ["precision", "recall", "f1_score"])
            """
            num_metrics = len(metrics)
            plt.figure(figsize=(5 * num_metrics, 4))

            for i, metric in enumerate(metrics):
                plt.subplot(1, num_metrics, i + 1)
                plt.plot(history.history[metric], label='Train')
                plt.plot(history.history[f'val_{metric}'], label='Validation')
                plt.title(f'Model {metric.replace("_", " ").title()}')
                plt.xlabel('Epoch')
                plt.ylabel(metric.replace("_", " ").title())
                plt.legend()

            plt.suptitle('Training and Validation Metrics over Epochs', fontsize=14)
            plt.tight_layout(rect=[0, 0, 1, 0.95])
            plt.show()
```

```
In [ ]: plot_metrics(history_sgd, metrics=["precision", "recall", "f1_score"])
```



Precision Metric - Improves well

- Starts being erratic and then stabilizes after about 40 epochs for both training and validation data.
- The mode seems to be improving in its positive predictions.

Recall Metric - Very poor

- Starts near zero, rises slowly and plateaus around 0.2 for training and 0.18 for validation even after 100 epochs.
- The model seems to be missing most of the action positives.
- This could be the reason the F1 score remains low.

F1 Metric - Flat and Low

- F1 metric is quite low (though the delta is less between the training and validation sets)
- The model has underfit and has not learnt any meaningful patterns.
- The flat lines suggest that the model's ability to distinguish between the classes (in terms of precision and recall) did not improve over time.

```
In [ ]: #Predicting the results using best as a threshold for training data
        y_train_pred = model_sgd.predict(X_train)
        y_train_pred = (y_train_pred > 0.5)
        y_train_pred
```

```
188/188 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
Out[ ]: array([[False],
               [False],
               [False],
               ...,
               [False],
               [False],
               [False]])
```

```
In [ ]: #Predicting the results using best as a threshold for validation data
        y_val_pred = model_sgd.predict(X_val)
        y_val_pred = (y_val_pred > 0.5)
        y_val_pred
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
Out[ ]: array([[False],
               [False],
               [False],
               ...,
               [False],
               [False],
               [False]])
```

```
In [ ]: model_name = "NN with SGD"

        train_scores_df.loc[model_name] = recall_score(y_train, y_train_pred)
        val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

Classification Report

```
In [ ]: #Training classification report
        cr = classification_report(y_train, y_train_pred)
        print(cr)
```

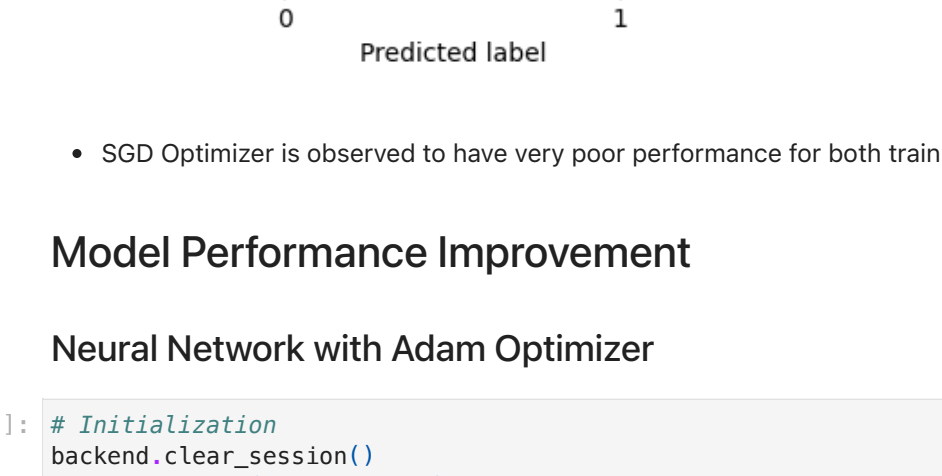
```
              precision    recall  f1-score   support

           0       0.83      0.98      0.90      4777
           1       0.70      0.21      0.32      1223

    accuracy                           0.82      6000
   macro avg       0.76      0.59      0.61      6000
weighted avg       0.80      0.82      0.78      6000
```

```
In [ ]: #Validation classification report
        cr = classification_report(y_val, y_val_pred)
        print(cr)
```

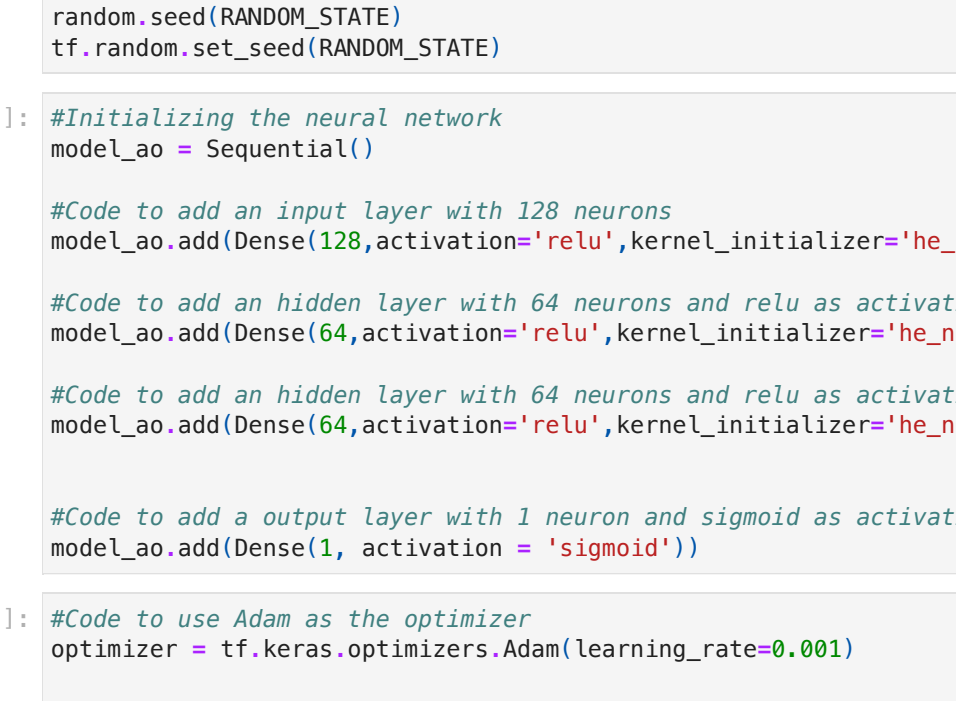
```
              precision    recall  f1-score   support

           0       0.83      0.98      0.90      1593
           1       0.73      0.19      0.30       407

    accuracy                           0.82      2000
   macro avg       0.78      0.58      0.60      2000
weighted avg       0.81      0.82      0.78      2000
```

Confusion Matrix

```
In [ ]: # Confusion matrix for training
        make_confusion_matrix(y_train, y_train_pred)
```



```
In [ ]: # Confusion matrix for validation
        make_confusion_matrix(y_val, y_val_pred)
```



- SGD Optimizer is observed to have very poor performance for both training and validation datasets.

Model Performance Improvement

Neural Network with Adam Optimizer

```
In [ ]: # Initialization
        backend.clear_session()
        np.random.seed(RANDOM_STATE)
        random.seed(RANDOM_STATE)
        tf.random.set_seed(RANDOM_STATE)
```

```
In [ ]: #Initializing the neural network
        model_ao = Sequential()

        #Code to add an input layer with 128 neurons
        model_ao.add(Dense(128,activation='relu',kernel_initializer='he_normal', input_dim = X_train.shape[1]))

        #Code to add an hidden layer with 64 neurons and relu as activation function
        model_ao.add(Dense(64,activation='relu',kernel_initializer='he_normal'))

        #Code to add an hidden layer with 64 neurons and relu as activation function
        model_ao.add(Dense(64,activation='relu',kernel_initializer='he_normal'))


        #Code to add a output layer with 1 neuron and sigmoid as activation function
        model_ao.add(Dense(1, activation = 'sigmoid'))
```

```
In [ ]: #Code to use Adam as the optimizer
        optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

        #Choose Precision, Recall and F1 score as the metric
        metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]

        # Compile the model with loss function of binary cross entropy and F1score as the metric
        model_ao.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

```
In [ ]: model_ao.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,536
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 1)	65

Total params: 14,017 (54.75 KB)
Trainable params: 14,017 (54.75 KB)
Non-trainable params: 0 (0.00 B)

```
In [ ]: start = time.time()

        history_ao = model_ao.fit(
        X_train, y_train,
        batch_size=32,
        validation_data=(X_val,y_val),
        epochs=100,
        verbose=1
        )

        end=time.time()
```

```
Epoch 1/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - f1_score: 0.3364 - loss: 0.7133 - precision: 0.3032 - recall: 0.2236 - val_f1_score: 0.3382 - val_loss: 0.4205 - val_precision: 0.6369 - val_recall: 0.2629
Epoch 2/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.4019 - precision: 0.6600 - recall: 0.3082 - val_f1_score: 0.3382 - val_loss: 0.4078 - val_precision: 0.6814 - val_recall: 0.3415
Epoch 3/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.3834 - precision: 0.6767 - recall: 0.3759 - val_f1_score: 0.3382 - val_loss: 0.3956 - val_precision: 0.6996 - val_recall: 0.3833
Epoch 4/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.3682 - precision: 0.7089 - recall: 0.4069 - val_f1_score: 0.3382 - val_loss: 0.3886 - val_precision: 0.6947 - val_recall: 0.3857
Epoch 5/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.3580 - precision: 0.7238 - recall: 0.4425 - val_f1_score: 0.3382 - val_loss: 0.3826 - val_precision: 0.7149 - val_recall: 0.4005
Epoch 6/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3484 - precision: 0.7231 - recall: 0.4603 - val_f1_score: 0.3382 - val_loss: 0.3810 - val_precision: 0.7232 - val_recall: 0.3980
Epoch 7/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3396 - precision: 0.7309 - recall: 0.4764 - val_f1_score: 0.3382 - val_loss: 0.3805 - val_precision: 0.7277 - val_recall: 0.4005
Epoch 8/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.3364 - loss: 0.3321 - precision: 0.7208 - recall: 0.4828 - val_f1_score: 0.3382 - val_loss: 0.3801 - val_precision: 0.7321 - val_recall: 0.4029
Epoch 9/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.3255 - precision: 0.7219 - recall: 0.4895 - val_f1_score: 0.3382 - val_loss: 0.3787 - val_precision: 0.7249 - val_recall: 0.4079
Epoch 10/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3188 - precision: 0.7390 - recall: 0.4946 - val_f1_score: 0.3382 - val_loss: 0.3796 - val_precision: 0.7031 - val_recall: 0.3956
Epoch 11/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3364 - loss: 0.3134 - precision: 0.7430 - recall: 0.4965 - val_f1_score: 0.3382 - val_loss: 0.3796 - val_precision: 0.7277 - val_recall: 0.4005
Epoch 12/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3066 - precision: 0.7535 - recall: 0.5099 - val_f1_score: 0.3382 - val_loss: 0.3804 - val_precision: 0.7237 - val_recall: 0.4054
Epoch 13/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3011 - precision: 0.7633 - recall: 0.5261 - val_f1_score: 0.3382 - val_loss: 0.3815 - val_precision: 0.7249 - val_recall: 0.4079
Epoch 14/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.2950 - precision: 0.7735 - recall: 0.5279 - val_f1_score: 0.3382 - val_loss: 0.3833 - val_precision: 0.7137 - val_recall: 0.4103
Epoch 15/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.2893 - precision: 0.7768 - recall: 0.5365 - val_f1_score: 0.3382 - val_loss: 0.3862 - val_precision: 0.7179 - val_recall: 0.4128
Epoch 16/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - f1_score: 0.3364 - loss: 0.2836 - precision: 0.7876 - recall: 0.5516 - val_f1_score: 0.3382 - val_loss: 0.3881 - val_precision: 0.7095 - val_recall: 0.4201
Epoch 17/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3364 - loss: 0.2774 - precision: 0.7971 - recall: 0.5672 - val_f1_score: 0.3382 - val_loss: 0.3904 - val_precision: 0.7097 - val_recall: 0.4324
Epoch 18/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.2718 - precision: 0.8003 - recall: 0.5768 - val_f1_score: 0.3382 - val_loss: 0.3943 - val_precision: 0.7085 - val_recall: 0.4300
Epoch 19/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3364 - loss: 0.2660 - precision: 0.7952 - recall: 0.5899 - val_f1_score: 0.3382 - val_loss: 0.3986 - val_precision: 0.7028 - val_recall: 0.4300
Epoch 20/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.2594 - precision: 0.8009 - recall: 0.5963 - val_f1_score: 0.3382 - val_loss: 0.4011 - val_precision: 0.7056 - val_recall: 0.4300
Epoch 21/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.2532 - precision: 0.8179 - recall: 0.6053 - val_f1_score: 0.3382 - val_loss: 0.4038 - val_precision: 0.6914 - val_recall: 0.4349
Epoch 22/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.2478 - precision: 0.8192 - recall: 0.6108 - val_f1_score: 0.3382 - val_loss: 0.4073 - val_precision: 0.6832 - val_recall: 0.4398
Epoch 23/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2415 - precision: 0.8291 - recall: 0.6298 - val_f1_score: 0.3382 - val_loss: 0.4124 - val_precision: 0.6716 - val_recall: 0.4423
Epoch 24/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - f1_score: 0.3364 - loss: 0.2350 - precision: 0.8322 - recall: 0.6489 - val_f1_score: 0.3382 - val_loss: 0.4182 - val_precision: 0.6605 - val_recall: 0.4398
Epoch 25/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3364 - loss: 0.2312 - precision: 0.8357 - recall: 0.6594 - val_f1_score: 0.3382 - val_loss: 0.4240 - val_precision: 0.6489 - val_recall: 0.4496
Epoch 26/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.2255 - precision: 0.8448 - recall: 0.6625 - val_f1_score: 0.3382 - val_loss: 0.4268 - val_precision: 0.6512 - val_recall: 0.4496
Epoch 27/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.2195 - precision: 0.8557 - recall: 0.6814 - val_f1_score: 0.3382 - val_loss: 0.4344 - val_precision: 0.6477 - val_recall: 0.4472
Epoch 28/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3364 - loss: 0.2138 - precision: 0.8620 - recall: 0.6898 - val_f1_score: 0.3382 - val_loss: 0.4422 - val_precision: 0.6500 - val_recall: 0.4472
Epoch 29/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.2090 - precision: 0.8614 - recall: 0.6937 - val_f1_score: 0.3382 - val_loss: 0.4480 - val_precision: 0.6389 - val_recall: 0.4521
Epoch 30/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.2036 - precision: 0.8638 - recall: 0.7044 - val_f1_score: 0.3382 - val_loss: 0.4552 - val_precision: 0.6399 - val_recall: 0.4496
Epoch 31/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.1988 - precision: 0.8689 - recall: 0.7217 - val_f1_score: 0.3382 - val_loss: 0.4614 - val_precision: 0.6446 - val_recall: 0.4545
Epoch 32/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.1932 - precision: 0.8660 - recall: 0.7265 - val_f1_score: 0.3382 - val_loss: 0.4679 - val_precision: 0.6477 - val_recall: 0.4472
Epoch 33/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.1879 - precision: 0.8804 - recall: 0.7342 - val_f1_score: 0.3382 - val_loss: 0.4758 - val_precision: 0.6309 - val_recall: 0.4619
Epoch 34/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.1831 - precision: 0.8821 - recall: 0.7418 - val_f1_score: 0.3382 - val_loss: 0.4844 - val_precision: 0.6230 - val_recall: 0.4668
Epoch 35/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.1783 - precision: 0.8857 - recall: 0.7429 - val_f1_score: 0.3382 - val_loss: 0.4914 - val_precision: 0.6325 - val_recall: 0.4693
Epoch 36/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.1740 - precision: 0.8871 - recall: 0.7539 - val_f1_score: 0.3382 - val_loss: 0.4990 - val_precision: 0.6266 - val_recall: 0.4742
Epoch 37/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.1691 - precision: 0.8922 - recall: 0.7611 - val_f1_score: 0.3382 - val_loss: 0.5104 - val_precision: 0.6109 - val_recall: 0.4668
Epoch 38/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - f1_score: 0.3364 - loss: 0.1659 - precision: 0.8924 - recall: 0.7588 - val_f1_score: 0.3382 - val_loss: 0.5157 - val_precision: 0.6136 - val_recall: 0.4644
Epoch 39/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.1598 - precision: 0.8971 - recall: 0.7674 - val_f1_score: 0.3382 - val_loss: 0.5252 - val_precision: 0.6065 - val_recall: 0.4619
Epoch 40/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.1560 - precision: 0.9014 - recall: 0.7782 - val_f1_score: 0.3382 - val_loss: 0.5374 - val_precision: 0.6054 - val_recall: 0.4447
Epoch 41/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3364 - loss: 0.1503 - precision: 0.9045 - recall: 0.7887 - val_f1_score: 0.3382 - val_loss: 0.5553 - val_precision: 0.5937 - val_recall: 0.4595
Epoch 42/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3364 - loss: 0.1469 - precision: 0.9067 - recall: 0.8024 - val_f1_score: 0.3382 - val_loss: 0.5593 - val_precision: 0.6097 - val_recall: 0.4644
Epoch 43/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3366 - loss: 0.1443 - precision: 0.9200 - recall: 0.8091 - val_f1_score: 0.3382 - val_loss: 0.5711 - val_precision: 0.5924 - val_recall: 0.4570
Epoch 44/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3366 - loss: 0.1409 - precision: 0.9111 - recall: 0.8225 - val_f1_score: 0.3382 - val_loss: 0.5855 - val_precision: 0.5877 - val_recall: 0.4447
Epoch 45/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3367 - loss: 0.1362 - precision: 0.9171 - recall: 0.8272 - val_f1_score: 0.3382 - val_loss: 0.6048 - val_precision: 0.5755 - val_recall: 0.4496
Epoch 46/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3367 - loss: 0.1324 - precision: 0.9221 - recall: 0.8400 - val_f1_score: 0.3382 - val_loss: 0.6127 - val_precision: 0.5692 - val_recall: 0.4447
Epoch 47/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3367 - loss: 0.1294 - precision: 0.9324 - recall: 0.8473 - val_f1_score: 0.3383 - val_loss: 0.6172 - val_precision: 0.5723 - val_recall: 0.4373
Epoch 48/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3368 - loss: 0.1243 - precision: 0.9426 - recall: 0.8562 - val_f1_score: 0.3385 - val_loss: 0.6370 - val_precision: 0.5762 - val_recall: 0.4275
Epoch 49/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3369 - loss: 0.1219 - precision: 0.9380 - recall: 0.8514 - val_f1_score: 0.3385 - val_loss: 0.6440 - val_precision: 0.5627 - val_recall: 0.4300
Epoch 50/100
```

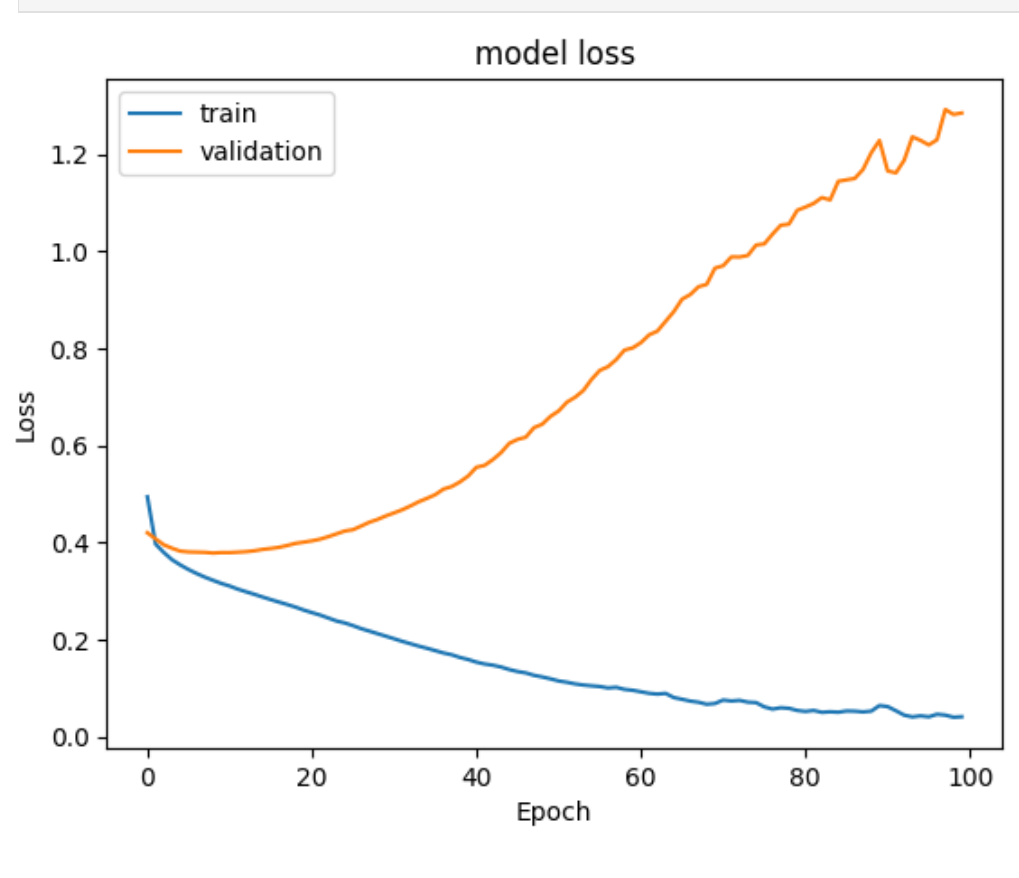
```
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3372 - loss: 0.1175 - precision: 0.9412 - recall: 0.8678 - val_f1_score: 0.3387 - val_loss: 0.6602 - val_precision: 0.5587 - val_recall: 0.4324
Epoch 51/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3373 - loss: 0.1137 - precision: 0.9389 - recall: 0.8707 - val_f1_score: 0.3387 - val_loss: 0.6713 - val_precision: 0.5690 - val_recall: 0.4152
Epoch 52/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.3377 - loss: 0.1098 - precision: 0.9485 - recall: 0.8864 - val_f1_score: 0.3389 - val_loss: 0.6894 - val_precision: 0.5570 - val_recall: 0.4324
Epoch 53/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3381 - loss: 0.1068 - precision: 0.9528 - recall: 0.8873 - val_f1_score: 0.3390 - val_loss: 0.6996 - val_precision: 0.5676 - val_recall: 0.4128
Epoch 54/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3381 - loss: 0.1036 - precision: 0.9570 - recall: 0.8912 - val_f1_score: 0.3392 - val_loss: 0.7134 - val_precision: 0.5482 - val_recall: 0.4054
Epoch 55/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3383 - loss: 0.1017 - precision: 0.9452 - recall: 0.8992 - val_f1_score: 0.3390 - val_loss: 0.7364 - val_precision: 0.5488 - val_recall: 0.4005
Epoch 56/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3386 - loss: 0.1001 - precision: 0.9532 - recall: 0.9031 - val_f1_score: 0.3385 - val_loss: 0.7549 - val_precision: 0.5827 - val_recall: 0.3980
Epoch 57/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3385 - loss: 0.0976 - precision: 0.9551 - recall: 0.9032 - val_f1_score: 0.3390 - val_loss: 0.7625 - val_precision: 0.5534 - val_recall: 0.4201
Epoch 58/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.3387 - loss: 0.0987 - precision: 0.9414 - recall: 0.8875 - val_f1_score: 0.3395 - val_loss: 0.7772 - val_precision: 0.6015 - val_recall: 0.4005
Epoch 59/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - f1_score: 0.3392 - loss: 0.0971 - precision: 0.9448 - recall: 0.8959 - val_f1_score: 0.3402 - val_loss: 0.7967 - val_precision: 0.5784 - val_recall: 0.3808
Epoch 60/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3393 - loss: 0.0944 - precision: 0.9522 - recall: 0.8868 - val_f1_score: 0.3400 - val_loss: 0.8013 - val_precision: 0.5571 - val_recall: 0.3833
Epoch 61/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3397 - loss: 0.0936 - precision: 0.9513 - recall: 0.8897 - val_f1_score: 0.3402 - val_loss: 0.8120 - val_precision: 0.5510 - val_recall: 0.3980
Epoch 62/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3399 - loss: 0.0902 - precision: 0.9603 - recall: 0.9026 - val_f1_score: 0.3407 - val_loss: 0.8281 - val_precision: 0.5680 - val_recall: 0.4103
Epoch 63/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3399 - loss: 0.0872 - precision: 0.9593 - recall: 0.8982 - val_f1_score: 0.3407 - val_loss: 0.8357 - val_precision: 0.5487 - val_recall: 0.4152
Epoch 64/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3407 - loss: 0.0875 - precision: 0.9590 - recall: 0.9061 - val_f1_score: 0.3412 - val_loss: 0.8559 - val_precision: 0.5306 - val_recall: 0.4472
Epoch 65/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - f1_score: 0.3400 - loss: 0.0817 - precision: 0.9790 - recall: 0.9115 - val_f1_score: 0.3415 - val_loss: 0.8760 - val_precision: 0.5364 - val_recall: 0.4349
Epoch 66/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3409 - loss: 0.0782 - precision: 0.9742 - recall: 0.9069 - val_f1_score: 0.3416 - val_loss: 0.9015 - val_precision: 0.5392 - val_recall: 0.4398
Epoch 67/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3406 - loss: 0.0735 - precision: 0.9739 - recall: 0.9230 - val_f1_score: 0.3418 - val_loss: 0.9111 - val_precision: 0.5378 - val_recall: 0.4373
Epoch 68/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3415 - loss: 0.0716 - precision: 0.9751 - recall: 0.9270 - val_f1_score: 0.3427 - val_loss: 0.9272 - val_precision: 0.5411 - val_recall: 0.4201
Epoch 69/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3421 - loss: 0.0683 - precision: 0.9780 - recall: 0.9330 - val_f1_score: 0.3425 - val_loss: 0.9321 - val_precision: 0.5449 - val_recall: 0.4619
Epoch 70/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3425 - loss: 0.0680 - precision: 0.9719 - recall: 0.9413 - val_f1_score: 0.3441 - val_loss: 0.9656 - val_precision: 0.5488 - val_recall: 0.4423
Epoch 71/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3432 - loss: 0.0729 - precision: 0.9653 - recall: 0.9288 - val_f1_score: 0.3431 - val_loss: 0.9705 - val_precision: 0.5122 - val_recall: 0.4644
Epoch 72/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3433 - loss: 0.0780 - precision: 0.9565 - recall: 0.9120 - val_f1_score: 0.3425 - val_loss: 0.9888 - val_precision: 0.4903 - val_recall: 0.4963
Epoch 73/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3435 - loss: 0.0740 - precision: 0.9547 - recall: 0.9298 - val_f1_score: 0.3450 - val_loss: 0.9884 - val_precision: 0.4987 - val_recall: 0.4717
Epoch 74/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3442 - loss: 0.0667 - precision: 0.9667 - recall: 0.9368 - val_f1_score: 0.3440 - val_loss: 0.9917 - val_precision: 0.4987 - val_recall: 0.4816
Epoch 75/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3447 - loss: 0.0735 - precision: 0.9599 - recall: 0.9236 - val_f1_score: 0.3443 - val_loss: 1.0131 - val_precision: 0.4879 - val_recall: 0.4939
Epoch 76/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3445 - loss: 0.0613 - precision: 0.9734 - recall: 0.9496 - val_f1_score: 0.3436 - val_loss: 1.0155 - val_precision: 0.5105 - val_recall: 0.4791
Epoch 77/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3456 - loss: 0.0561 - precision: 0.9737 - recall: 0.9485 - val_f1_score: 0.3445 - val_loss: 1.0356 - val_precision: 0.5051 - val_recall: 0.4889
Epoch 78/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3463 - loss: 0.0551 - precision: 0.9700 - recall: 0.9472 - val_f1_score: 0.3455 - val_loss: 1.0539 - val_precision: 0.5137 - val_recall: 0.4619
Epoch 79/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3474 - loss: 0.0591 - precision: 0.9669 - recall: 0.9563 - val_f1_score: 0.3467 - val_loss: 1.0562 - val_precision: 0.4976 - val_recall: 0.5160
Epoch 80/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3481 - loss: 0.0538 - precision: 0.9673 - recall: 0.9553 - val_f1_score: 0.3457 - val_loss: 1.0847 - val_precision: 0.4877 - val_recall: 0.4865
Epoch 81/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3474 - loss: 0.0507 - precision: 0.9797 - recall: 0.9554 - val_f1_score: 0.3475 - val_loss: 1.0913 - val_precision: 0.4880 - val_recall: 0.5012
Epoch 82/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3493 - loss: 0.0509 - precision: 0.9679 - recall: 0.9541 - val_f1_score: 0.3456 - val_loss: 1.0988 - val_precision: 0.5067 - val_recall: 0.4668
Epoch 83/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3489 - loss: 0.0507 - precision: 0.9705 - recall: 0.9539 - val_f1_score: 0.3467 - val_loss: 1.1109 - val_precision: 0.5052 - val_recall: 0.4816
Epoch 84/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3507 - loss: 0.0490 - precision: 0.9708 - recall: 0.9606 - val_f1_score: 0.3482 - val_loss: 1.1060 - val_precision: 0.5130 - val_recall: 0.4840
Epoch 85/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3521 - loss: 0.0470 - precision: 0.9750 - recall: 0.9607 - val_f1_score: 0.3478 - val_loss: 1.1445 - val_precision: 0.4948 - val_recall: 0.4693
Epoch 86/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.3516 - loss: 0.0507 - precision: 0.9743 - recall: 0.9527 - val_f1_score: 0.3486 - val_loss: 1.1474 - val_precision: 0.5025 - val_recall: 0.4988
Epoch 87/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3518 - loss: 0.0474 - precision: 0.9684 - recall: 0.9646 - val_f1_score: 0.3465 - val_loss: 1.1503 - val_precision: 0.4796 - val_recall: 0.4914
Epoch 88/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3522 - loss: 0.0448 - precision: 0.9712 - recall: 0.9638 - val_f1_score: 0.3494 - val_loss: 1.1687 - val_precision: 0.4835 - val_recall: 0.4693
Epoch 89/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3540 - loss: 0.0559 - precision: 0.9561 - recall: 0.9486 - val_f1_score: 0.3548 - val_loss: 1.2031 - val_precision: 0.5382 - val_recall: 0.4496
Epoch 90/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3556 - loss: 0.0562 - precision: 0.9655 - recall: 0.9427 - val_f1_score: 0.3523 - val_loss: 1.2285 - val_precision: 0.5489 - val_recall: 0.4275
Epoch 91/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3551 - loss: 0.0646 - precision: 0.9463 - recall: 0.9358 - val_f1_score: 0.3529 - val_loss: 1.1657 - val_precision: 0.5335 - val_recall: 0.4693
Epoch 92/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3544 - loss: 0.0516 - precision: 0.9727 - recall: 0.9469 - val_f1_score: 0.3525 - val_loss: 1.1618 - val_precision: 0.5463 - val_recall: 0.4496
Epoch 93/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3539 - loss: 0.0436 - precision: 0.9726 - recall: 0.9622 - val_f1_score: 0.3498 - val_loss: 1.1876 - val_precision: 0.5092 - val_recall: 0.4742
Epoch 94/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - f1_score: 0.3544 - loss: 0.0369 - precision: 0.9835 - recall: 0.9692 - val_f1_score: 0.3504 - val_loss: 1.2360 - val_precision: 0.4952 - val_recall: 0.5111
Epoch 95/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3562 - loss: 0.0392 - precision: 0.9812 - recall: 0.9709 - val_f1_score: 0.3502 - val_loss: 1.2286 - val_precision: 0.4920 - val_recall: 0.5307
Epoch 96/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - f1_score: 0.3558 - loss: 0.0375 - precision: 0.9785 - recall: 0.9696 - val_f1_score: 0.3535 - val_loss: 1.2193 - val_precision: 0.5102 - val_recall: 0.4939
Epoch 97/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3584 - loss: 0.0399 - precision: 0.9790 - recall: 0.9654 - val_f1_score: 0.3523 - val_loss: 1.2300 - val_precision: 0.4977 - val_recall: 0.5405
Epoch 98/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.3592 - loss: 0.0426 - precision: 0.9789 - recall: 0.9707 - val_f1_score: 0.3583 - val_loss: 1.2923 - val_precision: 0.5378 - val_recall: 0.4373
Epoch 99/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3619 - loss: 0.0411 - precision: 0.9742 - recall: 0.9602 - val_f1_score: 0.3626 - val_loss: 1.2821 - val_precision: 0.5358 - val_recall: 0.4595
Epoch 100/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3623 - loss: 0.0409 - precision: 0.9736 - recall: 0.9564 - val_f1_score: 0.3601 - val_loss: 1.2851 - val_precision: 0.5123 - val_recall: 0.4595
```

In [ ]:
```python
print("Time taken in seconds ",end-start)
```

```
Time taken in seconds  180.23850798606873
```

Plotting Training and Validation Loss

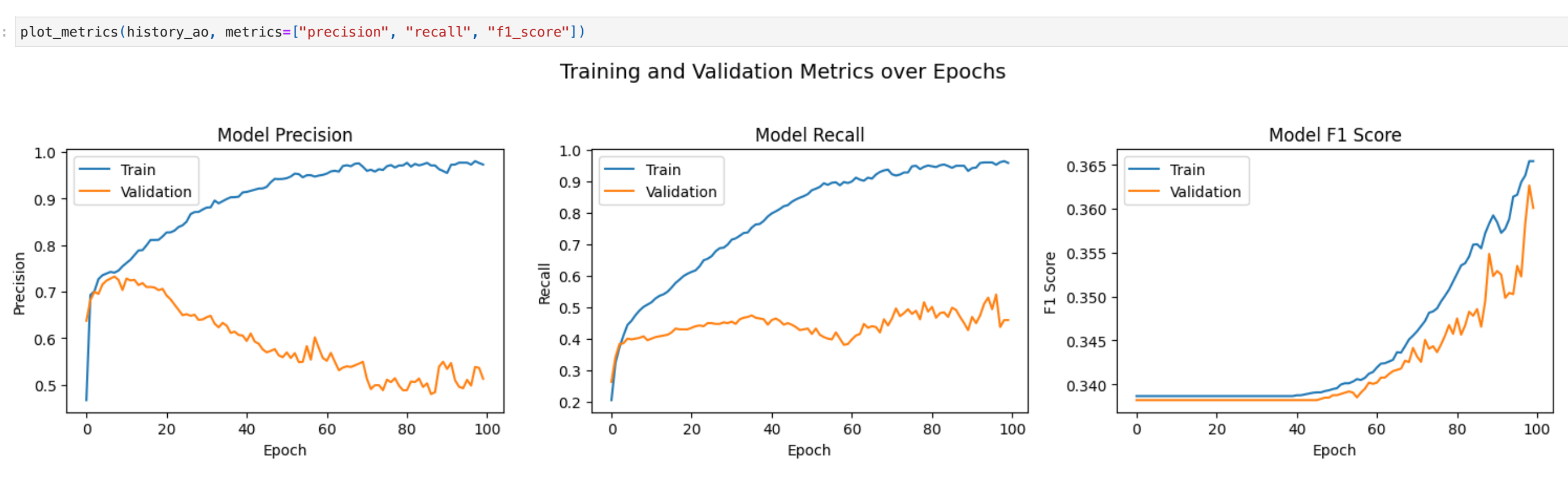
In [ ]:
```python
#Plotting Train Loss vs Validation Loss
plt.plot(history_ao.history['loss'])
plt.plot(history_ao.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```



- Training loss decreases smoothly and approaches zero - indicating that the model is fitting the training data well.
- Validation loss initially drop slightly and then increases to a higher value - indicating overfitting too early.

Plotting the Precision, Recall and F1 Scores

In [ ]:
```python
plot_metrics(history_ao, metrics=["precision", "recall", "f1_score"])
```



Precision Metric - Maybe overfitting

- Training precision steadily approaches 1, while, the validation precision peaks and then declines before stabilizing a little.
- Model seems to be learning,but, generalization is dropping for validation data (maybe due to overfitting)

Recall Metric - Poor recall on validation

- Training recall steadily approaches 1, while the, validation recall plateaus around 0.5.
- Limited generalization is seen with probable overfitting

F1 Metric - Low generalization

- Training and Validation F1 scores both show increases after about 50 epochs.
- The gap between the two F1 scores indicate that the model is not generalizing well.

In [ ]:
```python
#Predicting the results using best as a threshold for training data
y_train_pred = model_ao.predict(X_train)
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

```
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

Out[ ]:
```
array([[False],
       [False],
       [False],
       ...,
       [ True],
       [False],
       [ True]])
```

In [ ]:
```python
#Predicting the results using best as a threshold for validation data
y_val_pred = model_ao.predict(X_val)
y_val_pred = (y_val_pred > 0.5)
y_val_pred
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

Out[ ]:
```
array([[False],
       [ True],
       [ True],
       ...,
       [False],
       [False],
       [ True]])
```

In [ ]:
```python
model_name = "NN with Adam"

train_scores_df.loc[model_name] = recall_score(y_train, y_train_pred)
val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

Classification Report

In [ ]:
```python
#Training classification report
cr = classification_report(y_train, y_train_pred)
print(cr)
```

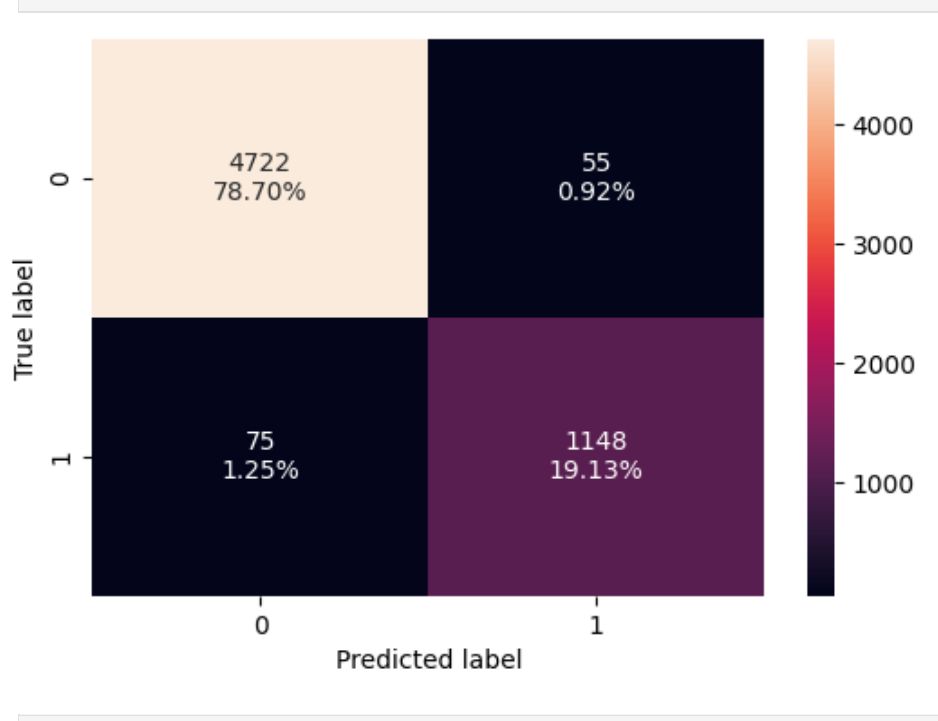
```
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      4777
           1       0.95      0.94      0.95      1223

    accuracy                           0.98      6000
   macro avg       0.97      0.96      0.97      6000
weighted avg       0.98      0.98      0.98      6000
```

In [ ]:
```python
#Validation classification report
cr = classification_report(y_val, y_val_pred)
print(cr)
```

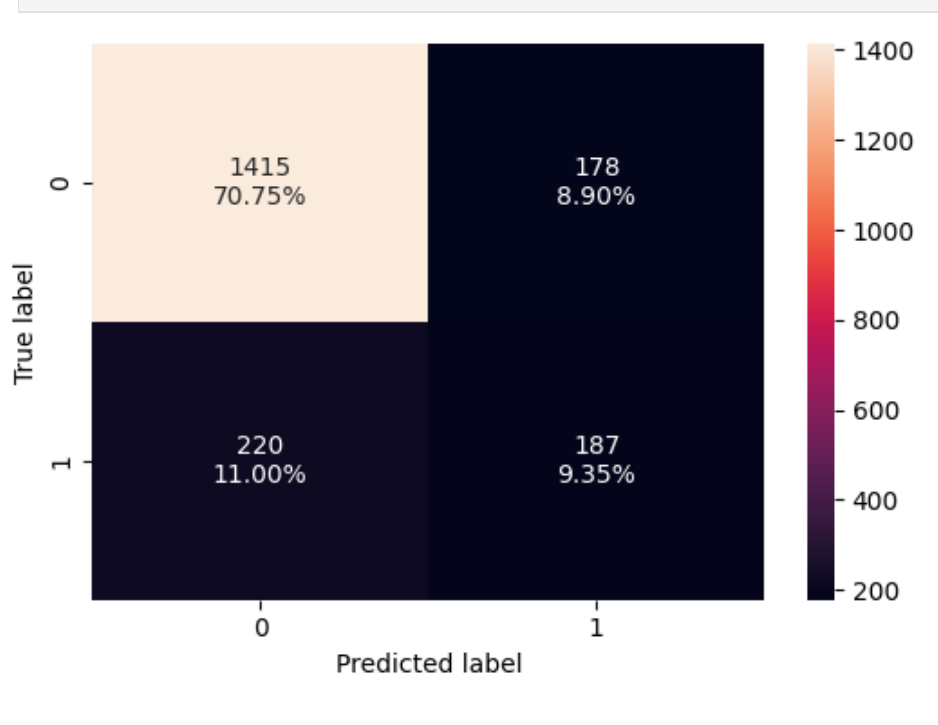
```
              precision    recall  f1-score   support

           0       0.87      0.89      0.88      1593
           1       0.51      0.46      0.48       407

    accuracy                           0.80      2000
   macro avg       0.69      0.67      0.68      2000
weighted avg       0.79      0.80      0.80      2000
```

Confusion Matrix

In [ ]:
```python
# Confusion matrix for training
make_confusion_matrix(y_train, y_train_pred)
```



In [ ]:
```python
# Confusion matrix for validation
make_confusion_matrix(y_val, y_val_pred)
```



- Validation metrics are not that great for Adam Optimizer.
- The model is learning training data too well.

Neural Network with Adam Optimizer and Dropout

In [ ]:
```python
# Initialization
backend.clear_session()
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
```

In [ ]:
```python
# Initialize the neural network with SGD Optimizer
model_ao_do = Sequential()

# Add an input layer with 64 neurons and Relu as the activation function
model_ao_do.add(Dense(units=64, activation='relu', input_shape=(X_train.shape[1],)))

# Add an hidden layer with 32 neurons and Relu as the activation function
model_ao_do.add(Dense(units=32, activation='relu'))

# Add a dropout with ratio of 0.2
model_ao_do.add(Dropout(0.2))

# Add an hidden layer with 32 neurons and Relu as the activation function
model_ao_do.add(Dense(units=32, activation='relu'))

# Add a dropout with ratio of 0.1
model_ao_do.add(Dropout(0.1))


# Add an output layer with 1 neuron and Sigmoid as the activation function
# as this is a binary classification problem
model_ao_do.add(Dense(units=1, activation='sigmoid'))
```

In [ ]:
```python
#Code to use Adam as the optimizer
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

#Choose Precision, Recall and F1 score as the metric
metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]

# Compile the model with loss function of binary cross entropy and F1score as the metric
model_ao_do.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

In [ ]:
```python
model_ao_do.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	768
dense_1 (Dense)	(None, 32)	2,080
dropout (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 32)	1,056
dropout_1 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 1)	33

Total params: 3,937 (15.38 KB)

Trainable params: 3,937 (15.38 KB)

Non-trainable params: 0 (0.00 B)

In [ ]:
```python
start = time.time()

history_ao_do = model_ao_do.fit(
X_train, y_train,
batch_size=32,
validation_data=(X_val,y_val),
epochs=100,
verbose=1
)

end=time.time()
```

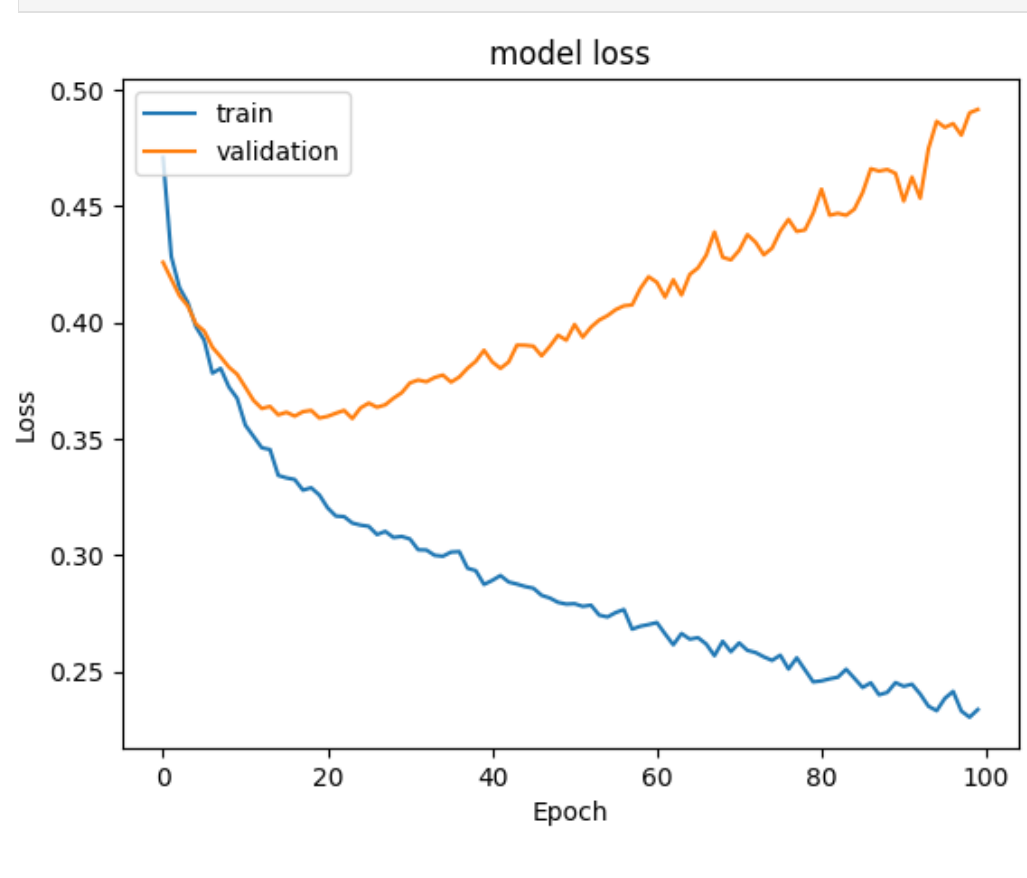
```
Epoch 1/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - f1_score: 0.3364 - loss: 0.5153 - precision: 0.3291 - recall: 0.0598 - val_f1_score: 0.3382 - val_loss: 0.4258 - val_precision: 0.6641 - val_recall: 0.2088
Epoch 2/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.3364 - loss: 0.4346 - precision: 0.7007 - recall: 0.2041 - val_f1_score: 0.3382 - val_loss: 0.4186 - val_precision: 0.6645 - val_recall: 0.2531
Epoch 3/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.4196 - precision: 0.6683 - recall: 0.2525 - val_f1_score: 0.3382 - val_loss: 0.4114 - val_precision: 0.6524 - val_recall: 0.2998
Epoch 4/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.3364 - loss: 0.4094 - precision: 0.6487 - recall: 0.3029 - val_f1_score: 0.3382 - val_loss: 0.4070 - val_precision: 0.6720 - val_recall: 0.3071
Epoch 5/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3997 - precision: 0.6930 - recall: 0.3401 - val_f1_score: 0.3382 - val_loss: 0.3993 - val_precision: 0.7015 - val_recall: 0.3464
Epoch 6/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3364 - loss: 0.3962 - precision: 0.6888 - recall: 0.3648 - val_f1_score: 0.3382 - val_loss: 0.3963 - val_precision: 0.7643 - val_recall: 0.2948
Epoch 7/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3364 - loss: 0.3787 - precision: 0.7012 - recall: 0.3619 - val_f1_score: 0.3382 - val_loss: 0.3893 - val_precision: 0.7143 - val_recall: 0.3563
Epoch 8/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3837 - precision: 0.7254 - recall: 0.3695 - val_f1_score: 0.3382 - val_loss: 0.3852 - val_precision: 0.7451 - val_recall: 0.3735
Epoch 9/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.3364 - loss: 0.3762 - precision: 0.7401 - recall: 0.3910 - val_f1_score: 0.3382 - val_loss: 0.3807 - val_precision: 0.7358 - val_recall: 0.3833
Epoch 10/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - f1_score: 0.3364 - loss: 0.3742 - precision: 0.7302 - recall: 0.3827 - val_f1_score: 0.3382 - val_loss: 0.3777 - val_precision: 0.7308 - val_recall: 0.3735
Epoch 11/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.3652 - precision: 0.7483 - recall: 0.4032 - val_f1_score: 0.3382 - val_loss: 0.3721 - val_precision: 0.7094 - val_recall: 0.4079
Epoch 12/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.3533 - precision: 0.7673 - recall: 0.4338 - val_f1_score: 0.3382 - val_loss: 0.3664 - val_precision: 0.7406 - val_recall: 0.3857
Epoch 13/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3518 - precision: 0.7711 - recall: 0.4409 - val_f1_score: 0.3382 - val_loss: 0.3630 - val_precision: 0.7477 - val_recall: 0.4005
Epoch 14/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3518 - precision: 0.7837 - recall: 0.4370 - val_f1_score: 0.3382 - val_loss: 0.3638 - val_precision: 0.7419 - val_recall: 0.3956
Epoch 15/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3392 - precision: 0.7805 - recall: 0.4465 - val_f1_score: 0.3382 - val_loss: 0.3602 - val_precision: 0.7558 - val_recall: 0.4029
Epoch 16/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - f1_score: 0.3364 - loss: 0.3377 - precision: 0.7892 - recall: 0.4610 - val_f1_score: 0.3382 - val_loss: 0.3614 - val_precision: 0.7617 - val_recall: 0.4005
Epoch 17/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.3371 - precision: 0.7729 - recall: 0.4470 - val_f1_score: 0.3382 - val_loss: 0.3597 - val_precision: 0.7703 - val_recall: 0.3956
Epoch 18/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.3331 - precision: 0.8014 - recall: 0.4563 - val_f1_score: 0.3382 - val_loss: 0.3616 - val_precision: 0.7830 - val_recall: 0.4079
Epoch 19/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3348 - precision: 0.7862 - recall: 0.4567 - val_f1_score: 0.3382 - val_loss: 0.3621 - val_precision: 0.7632 - val_recall: 0.4275
Epoch 20/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3307 - precision: 0.7858 - recall: 0.4544 - val_f1_score: 0.3382 - val_loss: 0.3589 - val_precision: 0.7470 - val_recall: 0.4644
Epoch 21/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3254 - precision: 0.7918 - recall: 0.4893 - val_f1_score: 0.3382 - val_loss: 0.3596 - val_precision: 0.7797 - val_recall: 0.4349
Epoch 22/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - f1_score: 0.3364 - loss: 0.3215 - precision: 0.7890 - recall: 0.4754 - val_f1_score: 0.3382 - val_loss: 0.3609 - val_precision: 0.7661 - val_recall: 0.4103
Epoch 23/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - f1_score: 0.3364 - loss: 0.3256 - precision: 0.7902 - recall: 0.4683 - val_f1_score: 0.3382 - val_loss: 0.3621 - val_precision: 0.7500 - val_recall: 0.4275
Epoch 24/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.3225 - precision: 0.8016 - recall: 0.4796 - val_f1_score: 0.3382 - val_loss: 0.3586 - val_precision: 0.7578 - val_recall: 0.4152
Epoch 25/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.3189 - precision: 0.7979 - recall: 0.4976 - val_f1_score: 0.3382 - val_loss: 0.3632 - val_precision: 0.7699 - val_recall: 0.4275
Epoch 26/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.3187 - precision: 0.7892 - recall: 0.4813 - val_f1_score: 0.3382 - val_loss: 0.3653 - val_precision: 0.7565 - val_recall: 0.4275
Epoch 27/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3147 - precision: 0.8003 - recall: 0.4872 - val_f1_score: 0.3382 - val_loss: 0.3635 - val_precision: 0.7562 - val_recall: 0.4496
Epoch 28/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.3162 - precision: 0.7794 - recall: 0.4837 - val_f1_score: 0.3382 - val_loss: 0.3645 - val_precision: 0.7636 - val_recall: 0.4128
Epoch 29/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.3364 - loss: 0.3168 - precision: 0.8036 - recall: 0.4914 - val_f1_score: 0.3382 - val_loss: 0.3674 - val_precision: 0.7357 - val_recall: 0.4103
Epoch 30/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.3098 - precision: 0.7934 - recall: 0.5038 - val_f1_score: 0.3382 - val_loss: 0.3697 - val_precision: 0.7448 - val_recall: 0.4373
Epoch 31/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - f1_score: 0.3364 - loss: 0.3152 - precision: 0.7691 - recall: 0.4886 - val_f1_score: 0.3382 - val_loss: 0.3739 - val_precision: 0.7479 - val_recall: 0.4447
Epoch 32/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3131 - precision: 0.7803 - recall: 0.5082 - val_f1_score: 0.3382 - val_loss: 0.3752 - val_precision: 0.7456 - val_recall: 0.4177
Epoch 33/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.3024 - precision: 0.7853 - recall: 0.5178 - val_f1_score: 0.3382 - val_loss: 0.3744 - val_precision: 0.7436 - val_recall: 0.4275
Epoch 34/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.3062 - precision: 0.7935 - recall: 0.5039 - val_f1_score: 0.3382 - val_loss: 0.3763 - val_precision: 0.7511 - val_recall: 0.4373
Epoch 35/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.3364 - loss: 0.3054 - precision: 0.8016 - recall: 0.5025 - val_f1_score: 0.3382 - val_loss: 0.3773 - val_precision: 0.7349 - val_recall: 0.4496
Epoch 36/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.3364 - loss: 0.3083 - precision: 0.7907 - recall: 0.4942 - val_f1_score: 0.3382 - val_loss: 0.3743 - val_precision: 0.7322 - val_recall: 0.4300
Epoch 37/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3080 - precision: 0.7749 - recall: 0.5159 - val_f1_score: 0.3382 - val_loss: 0.3765 - val_precision: 0.7500 - val_recall: 0.4275
Epoch 38/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2991 - precision: 0.8084 - recall: 0.4991 - val_f1_score: 0.3382 - val_loss: 0.3803 - val_precision: 0.7255 - val_recall: 0.4545
Epoch 39/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.3020 - precision: 0.7753 - recall: 0.5183 - val_f1_score: 0.3382 - val_loss: 0.3832 - val_precision: 0.7160 - val_recall: 0.4521
Epoch 40/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - f1_score: 0.3364 - loss: 0.2948 - precision: 0.7935 - recall: 0.5178 - val_f1_score: 0.3382 - val_loss: 0.3880 - val_precision: 0.7171 - val_recall: 0.4423
Epoch 41/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - f1_score: 0.3364 - loss: 0.2928 - precision: 0.8049 - recall: 0.5318 - val_f1_score: 0.3382 - val_loss: 0.3830 - val_precision: 0.7276 - val_recall: 0.4398
Epoch 42/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2968 - precision: 0.7944 - recall: 0.5353 - val_f1_score: 0.3382 - val_loss: 0.3802 - val_precision: 0.7183 - val_recall: 0.4447
Epoch 43/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2950 - precision: 0.8052 - recall: 0.5270 - val_f1_score: 0.3382 - val_loss: 0.3830 - val_precision: 0.7222 - val_recall: 0.4472
Epoch 44/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2914 - precision: 0.8152 - recall: 0.5395 - val_f1_score: 0.3382 - val_loss: 0.3902 - val_precision: 0.7214 - val_recall: 0.4644
Epoch 45/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2907 - precision: 0.8070 - recall: 0.5303 - val_f1_score: 0.3382 - val_loss: 0.3901 - val_precision: 0.6984 - val_recall: 0.4324
Epoch 46/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - f1_score: 0.3364 - loss: 0.2941 - precision: 0.7859 - recall: 0.5166 - val_f1_score: 0.3382 - val_loss: 0.3898 - val_precision: 0.7068 - val_recall: 0.4324
Epoch 47/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - f1_score: 0.3364 - loss: 0.2886 - precision: 0.8087 - recall: 0.5384 - val_f1_score: 0.3382 - val_loss: 0.3856 - val_precision: 0.7412 - val_recall: 0.4152
Epoch 48/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2897 - precision: 0.7994 - recall: 0.5354 - val_f1_score: 0.3382 - val_loss: 0.3897 - val_precision: 0.7004 - val_recall: 0.4595
Epoch 49/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2845 - precision: 0.8044 - recall: 0.5561 - val_f1_score: 0.3382 - val_loss: 0.3945 - val_precision: 0.6944 - val_recall: 0.4300
Epoch 50/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2861 - precision: 0.8087 - recall: 0.5676 - val_f1_score: 0.3382 - val_loss: 0.3923 - val_precision: 0.6914 - val_recall: 0.4349
Epoch 51/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2869 - precision: 0.7797 - recall: 0.5398 - val_f1_score: 0.3382 - val_loss: 0.3991 - val_precision: 0.7085 - val_recall: 0.4300
Epoch 52/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - f1_score: 0.3364 - loss: 0.2823 - precision: 0.7907 - recall: 0.5391 - val_f1_score: 0.3382 - val_loss: 0.3936 - val_precision: 0.6967 - val_recall: 0.4177
Epoch 53/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - f1_score: 0.3364 - loss: 0.2872 - precision: 0.8018 - recall: 0.5509 - val_f1_score: 0.3382 - val_loss: 0.3980 - val_precision: 0.6889 - val_recall: 0.4570
Epoch 54/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2831 - precision: 0.8011 - recall: 0.5621 - val_f1_score: 0.3382 - val_loss: 0.4010 - val_precision: 0.6935 - val_recall: 0.4447
Epoch 55/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2788 - precision: 0.8084 - recall: 0.5575 - val_f1_score: 0.3382 - val_loss: 0.4029 - val_precision: 0.6911 - val_recall: 0.4398
Epoch 56/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2799 - precision: 0.7841 - recall: 0.5614 - val_f1_score: 0.3382 - val_loss: 0.4055 - val_precision: 0.6917 - val_recall: 0.4521
Epoch 57/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2832 - precision: 0.7974 - recall: 0.5533 - val_f1_score: 0.3382 - val_loss: 0.4071 - val_precision: 0.7038 - val_recall: 0.4496
Epoch 58/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2687 - precision: 0.8083 - recall: 0.5765 - val_f1_score: 0.3382 - val_loss: 0.4075 - val_precision: 0.6903 - val_recall: 0.4545
Epoch 59/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.3364 - loss: 0.2721 - precision: 0.8063 - recall: 0.5782 - val_f1_score: 0.3382 - val_loss: 0.4146 - val_precision: 0.6859 - val_recall: 0.4668
Epoch 60/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - f1_score: 0.3364 - loss: 0.2767 - precision: 0.8207 - recall: 0.5719 - val_f1_score: 0.3382 - val_loss: 0.4196 - val_precision: 0.6621 - val_recall: 0.4767
Epoch 61/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2761 - precision: 0.8125 - recall: 0.5644 - val_f1_score: 0.3382 - val_loss: 0.4172 - val_precision: 0.6620 - val_recall: 0.4668
Epoch 62/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2682 - precision: 0.8065 - recall: 0.5903 - val_f1_score: 0.3382 - val_loss: 0.4108 - val_precision: 0.6788 - val_recall: 0.4570
Epoch 63/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2647 - precision: 0.8097 - recall: 0.5859 - val_f1_score: 0.3382 - val_loss: 0.4183 - val_precision: 0.6929 - val_recall: 0.4545
Epoch 64/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2715 - precision: 0.7849 - recall: 0.5676 - val_f1_score: 0.3382 - val_loss: 0.4117 - val_precision: 0.6773 - val_recall: 0.4693
Epoch 65/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2682 - precision: 0.8017 - recall: 0.5945 - val_f1_score: 0.3382 - val_loss: 0.4206 - val_precision: 0.6957 - val_recall: 0.4324
Epoch 66/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - f1_score: 0.3364 - loss: 0.2688 - precision: 0.8129 - recall: 0.5592 - val_f1_score: 0.3382 - val_loss: 0.4234 - val_precision: 0.6803 - val_recall: 0.4496
Epoch 67/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - f1_score: 0.3364 - loss: 0.2671 - precision: 0.8156 - recall: 0.5724 - val_f1_score: 0.3382 - val_loss: 0.4289 - val_precision: 0.6791 - val_recall: 0.4472
Epoch 68/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2595 - precision: 0.8201 - recall: 0.5849 - val_f1_score: 0.3382 - val_loss: 0.4388 - val_precision: 0.6606 - val_recall: 0.4496
Epoch 69/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2681 - precision: 0.8104 - recall: 0.5760 - val_f1_score: 0.3382 - val_loss: 0.4280 - val_precision: 0.6678 - val_recall: 0.4791
Epoch 70/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2592 - precision: 0.8084 - recall: 0.6068 - val_f1_score: 0.3382 - val_loss: 0.4268 - val_precision: 0.6714 - val_recall: 0.4668
Epoch 71/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2640 - precision: 0.8142 - recall: 0.6075 - val_f1_score: 0.3382 - val_loss: 0.4309 - val_precision: 0.6769 - val_recall: 0.4324
Epoch 72/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2622 - precision: 0.8203 - recall: 0.5946 - val_f1_score: 0.3382 - val_loss: 0.4377 - val_precision: 0.6741 - val_recall: 0.4472
Epoch 73/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - f1_score: 0.3364 - loss: 0.2639 - precision: 0.8214 - recall: 0.5934 - val_f1_score: 0.3382 - val_loss: 0.4344 - val_precision: 0.6929 - val_recall: 0.4324
Epoch 74/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2566 - precision: 0.8193 - recall: 0.6110 - val_f1_score: 0.3382 - val_loss: 0.4290 - val_precision: 0.6741 - val_recall: 0.4472
Epoch 75/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2559 - precision: 0.8172 - recall: 0.5951 - val_f1_score: 0.3382 - val_loss: 0.4319 - val_precision: 0.6965 - val_recall: 0.4398
Epoch 76/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2630 - precision: 0.8178 - recall: 0.6171 - val_f1_score: 0.3382 - val_loss: 0.4392 - val_precision: 0.6654 - val_recall: 0.4349
Epoch 77/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2532 - precision: 0.8239 - recall: 0.6020 - val_f1_score: 0.3382 - val_loss: 0.4443 - val_precision: 0.6870 - val_recall: 0.4152
Epoch 78/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2567 - precision: 0.8188 - recall: 0.5983 - val_f1_score: 0.3382 - val_loss: 0.4391 - val_precision: 0.7068 - val_recall: 0.4324
Epoch 79/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - f1_score: 0.3364 - loss: 0.2509 - precision: 0.8292 - recall: 0.6087 - val_f1_score: 0.3382 - val_loss: 0.4397 - val_precision: 0.6848 - val_recall: 0.4324
Epoch 80/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2454 - precision: 0.8370 - recall: 0.6175 - val_f1_score: 0.3382 - val_loss: 0.4470 - val_precision: 0.6784 - val_recall: 0.4251
Epoch 81/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2471 - precision: 0.8356 - recall: 0.6120 - val_f1_score: 0.3382 - val_loss: 0.4572 - val_precision: 0.6618 - val_recall: 0.4423
Epoch 82/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2500 - precision: 0.8031 - recall: 0.6010 - val_f1_score: 0.3382 - val_loss: 0.4460 - val_precision: 0.6729 - val_recall: 0.4398
Epoch 83/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2494 - precision: 0.8243 - recall: 0.6283 - val_f1_score: 0.3382 - val_loss: 0.4469 - val_precision: 0.6856 - val_recall: 0.4447
Epoch 84/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.3364 - loss: 0.2487 - precision: 0.8397 - recall: 0.6146 - val_f1_score: 0.3382 - val_loss: 0.4460 - val_precision: 0.6715 - val_recall: 0.4570
Epoch 85/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - f1_score: 0.3364 - loss: 0.2533 - precision: 0.8115 - recall: 0.6361 - val_f1_score: 0.3382 - val_loss: 0.4486 - val_precision: 0.6836 - val_recall: 0.4300
Epoch 86/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2470 - precision: 0.8380 - recall: 0.6310 - val_f1_score: 0.3382 - val_loss: 0.4557 - val_precision: 0.6679 - val_recall: 0.4447
Epoch 87/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2473 - precision: 0.8104 - recall: 0.6242 - val_f1_score: 0.3382 - val_loss: 0.4660 - val_precision: 0.6703 - val_recall: 0.4545
Epoch 88/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2452 - precision: 0.8265 - recall: 0.6474 - val_f1_score: 0.3382 - val_loss: 0.4650 - val_precision: 0.6679 - val_recall: 0.4398
Epoch 89/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2391 - precision: 0.8321 - recall: 0.6197 - val_f1_score: 0.3382 - val_loss: 0.4657 - val_precision: 0.6643 - val_recall: 0.4570
Epoch 90/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - f1_score: 0.3364 - loss: 0.2477 - precision: 0.8237 - recall: 0.6146 - val_f1_score: 0.3382 - val_loss: 0.4640 - val_precision: 0.6705 - val_recall: 0.4300
Epoch 91/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - f1_score: 0.3364 - loss: 0.2446 - precision: 0.8284 - recall: 0.6500 - val_f1_score: 0.3382 - val_loss: 0.4521 - val_precision: 0.6667 - val_recall: 0.4373
Epoch 92/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2432 - precision: 0.8325 - recall: 0.6322 - val_f1_score: 0.3382 - val_loss: 0.4624 - val_precision: 0.6965 - val_recall: 0.4398
Epoch 93/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.3364 - loss: 0.2473 - precision: 0.8236 - recall: 0.6343 - val_f1_score: 0.3382 - val_loss: 0.4534 - val_precision: 0.6830 - val_recall: 0.4447
Epoch 94/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - f1_score: 0.3364 - loss: 0.2350 - precision: 0.8394 - recall: 0.6336 - val_f1_score: 0.3382 - val_loss: 0.4746 - val_precision: 0.6704 - val_recall: 0.4447
Epoch 95/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2347 - precision: 0.8538 - recall: 0.6312 - val_f1_score: 0.3382 - val_loss: 0.4864 - val_precision: 0.6745 - val_recall: 0.4226
Epoch 96/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2454 - precision: 0.8244 - recall: 0.6384 - val_f1_score: 0.3382 - val_loss: 0.4838 - val_precision: 0.6806 - val_recall: 0.4398
Epoch 97/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - f1_score: 0.3364 - loss: 0.2491 - precision: 0.8192 - recall: 0.6275 - val_f1_score: 0.3382 - val_loss: 0.4854 - val_precision: 0.6870 - val_recall: 0.4423
Epoch 98/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - f1_score: 0.3364 - loss: 0.2337 - precision: 0.8376 - recall: 0.6392 - val_f1_score: 0.3382 - val_loss: 0.4804 - val_precision: 0.6840 - val_recall: 0.4201
Epoch 99/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2331 - precision: 0.8415 - recall: 0.6511 - val_f1_score: 0.3382 - val_loss: 0.4901 - val_precision: 0.6714 - val_recall: 0.4619
Epoch 100/100
188/188 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.3364 - loss: 0.2380 - precision: 0.8351 - recall: 0.6369 - val_f1_score: 0.3382 - val_loss: 0.4915 - val_precision: 0.6792 - val_recall: 0.4423
```

In [ ]:
```python
print("Time taken in seconds ",end-start)
```

```
Time taken in seconds  220.69838190078735
```

Plotting Training and Validation Loss

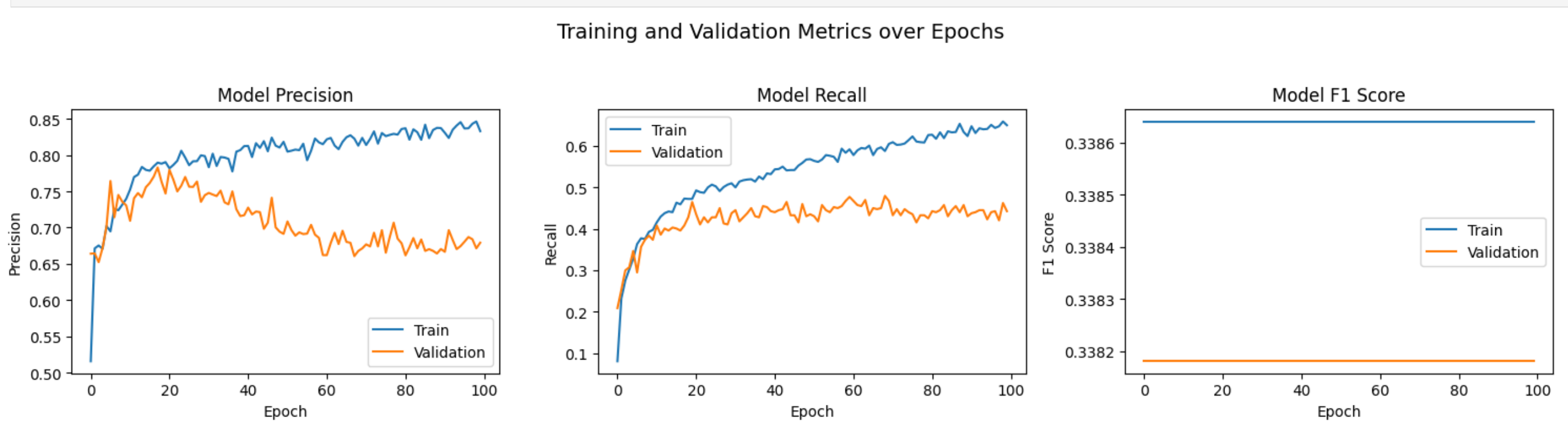
In [ ]:
```python
#Plotting Train Loss vs Validation Loss
plt.plot(history_ao_do.history['loss'])
plt.plot(history_ao_do.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```



- Training loss decreases consistently - indicating that the model is learning from the training data
- Validation loss initially follows training loss, but then flattens and increases gradually after about 30 epochs.
- Dropout has helped reduce overfitting compared to the previous model and the gap between training and validation loss is smaller.

Plotting the Precision, Recall and F1 Scores

In [ ]:
```python
plot_metrics(history_ao_do, metrics=["precision", "recall", "f1_score"])
```



Precision Metric - Overfitting

- Training precision increases steadily and levels of around 0.83, but, validation precision increases initially and then declines after about 30 epochs.
- This suggests overfitting and hence reducing generalizability

Recall Metric - Overfitting

- Training recall increases gradually and reaches approximately 0.65, while, validation recall improves initially and the plateaus to become noisy.
- This reflects that the model is learning to identify positives in the training data, but, struggles to generalize to unseen data.

F1 Metric - Minor improvement, Overfitting

- Both Training and Validation F1 scores are nearly flat, with sudden small spikes towards the end.
- The flat F1 score points towards the data imbalance.

In [ ]:
```python
#Predicting the results using best as a threshold for training data
y_train_pred = model_ao_do.predict(X_train)
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

```
188/188 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

Out[ ]:
```
array([[False],
       [False],
       [False],
       ...,
       [ True],
       [False],
       [ True]])
```

In [ ]:
```python
#Predicting the results using best as a threshold for validation data
y_val_pred = model_ao_do.predict(X_val)
y_val_pred = (y_val_pred > 0.5)
y_val_pred
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

Out[ ]:
```
array([[False],
       [False],
       [ True],
       ...,
       [False],
       [False],
       [False]])
```

In [ ]:
```python
model_name = "NN with Adam Dropout"
train_scores_df.loc[model_name] = recall_score(y_train, y_train_pred)
val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

Classification Report

In [ ]:
```python
#Training classification report
cr = classification_report(y_train, y_train_pred)
print(cr)
```

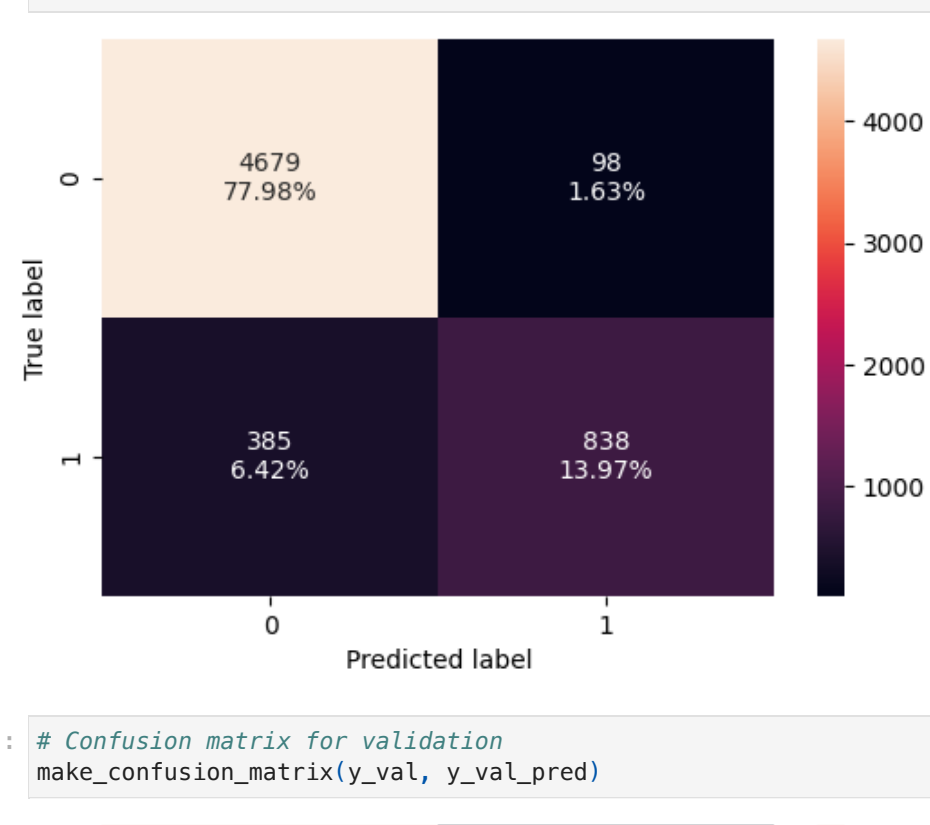
```
              precision    recall  f1-score   support

           0       0.92      0.98      0.95      4777
           1       0.90      0.69      0.78      1223

    accuracy                           0.92      6000
   macro avg       0.91      0.83      0.86      6000
weighted avg       0.92      0.92      0.92      6000
```

In [ ]:
```python
#Validation classification report
cr = classification_report(y_val, y_val_pred)
print(cr)
```

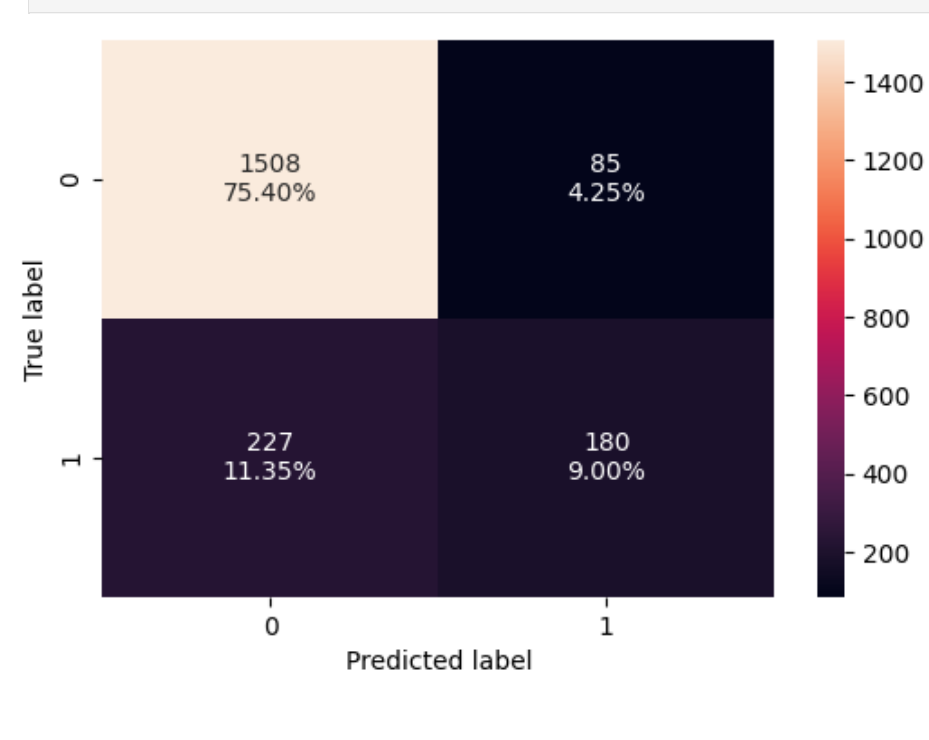
```
              precision    recall  f1-score   support

           0       0.87      0.95      0.91      1593
           1       0.68      0.44      0.54       407

    accuracy                           0.84      2000
   macro avg       0.77      0.69      0.72      2000
weighted avg       0.83      0.84      0.83      2000
```

Confusion Matrix

In [ ]:
```python
# Confusion matrix for training
make_confusion_matrix(y_train, y_train_pred)
```



In [ ]:
```python
# Confusion matrix for validation
make_confusion_matrix(y_val, y_val_pred)
```



- Validation metrics have slightly improved but still not great for Adam Optimizer with dropouts.
- The model is still learning training data well.

Neural Network with Balanced Data (by applying SMOTE) and SGD Optimizer

Applying SMOTE & Instantiate NN

In [ ]:
```python
sm = SMOTE(random_state=RANDOM_STATE)
X_train_sm, y_train_sm = sm.fit_resample(X_train, y_train)
print(X_train_sm.shape, y_train_sm.shape)
```

```
(9554, 11) (9554,)
```

In [ ]:
```python
# Initialization
backend.clear_session()
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
```

In [ ]:
```python
# Initialize the neural network with SGD Optimizer
model_sgd_sm = Sequential()

# Add an input layer with 128 neurons and Relu as the activation function
model_sgd_sm.add(Dense(units=128, activation='relu', input_shape=(X_train_sm.shape[1],)))

# Add an hidden layer with 64 neurons and Relu as the activation function
model_sgd_sm.add(Dense(units=64, activation='relu'))

# Add an hidden layer with 64 neurons and Relu as the activation function
model_sgd_sm.add(Dense(units=64, activation='relu'))

# Add an output layer with 1 neuron and Sigmoid as the activation function
# as this is a binary classification problem
model_sgd_sm.add(Dense(units=1, activation='sigmoid'))
```

In [ ]:
```python
#Code to use SGD as the optimizer.
optimizer = tf.keras.optimizers.SGD(0.001)

#Choose Precision, Recall and F1 score as the metric
metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]
```

In [ ]:
```python
# Compile the model with loss function of binary cross entropy and F1score as the metric
```

```
model_sgd_sm.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

```
In [ ]: model_sgd_sm.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,536
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 1)	65

Total params: 14,017 (54.75 KB)
Trainable params: 14,017 (54.75 KB)
Non-trainable params: 0 (0.00 B)

```
In [ ]: start = time.time()

        history_sgd_sm = model_sgd_sm.fit(
        X_train_sm, y_train_sm,
        batch_size=32, ## Code to specify the batch size to use
        epochs=100, ## Code to specify the number of epochs
        verbose=1,
        validation_data = (X_val,y_val)
        )

        end=time.time()
```

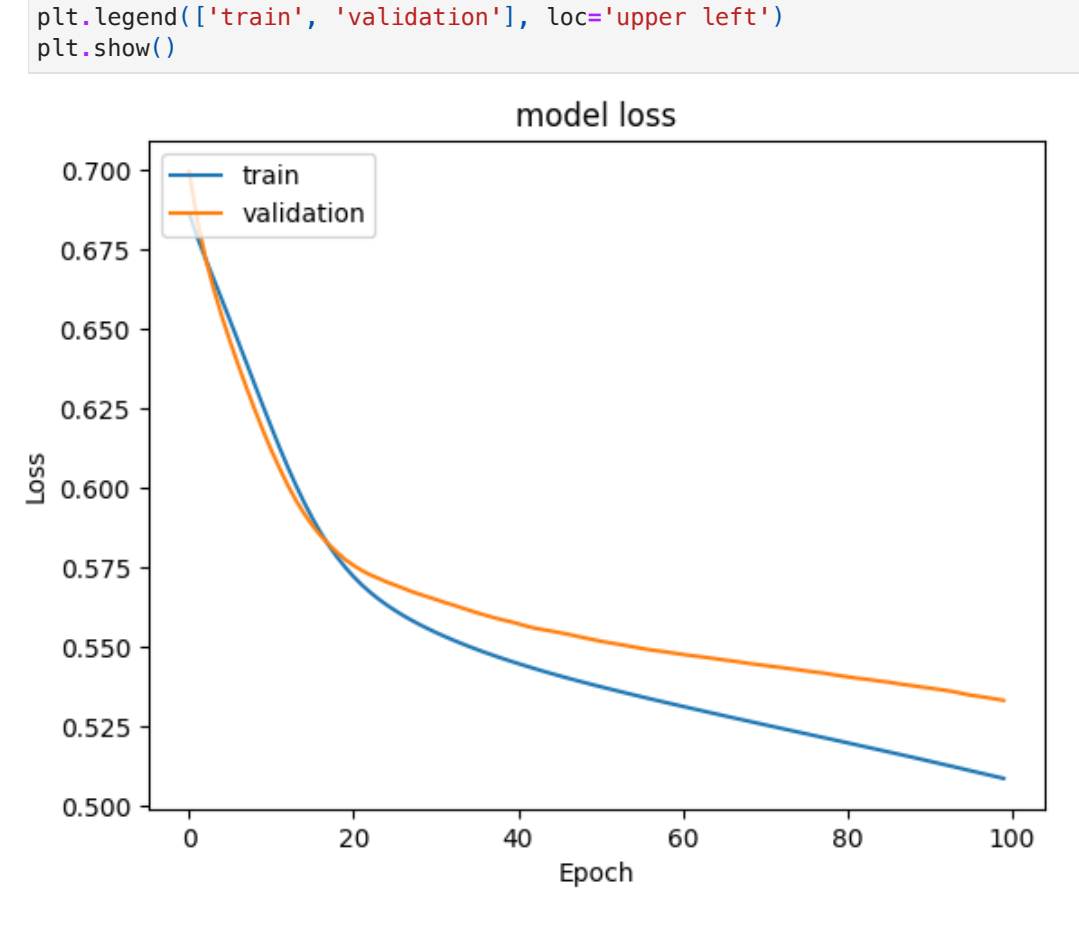
```
Epoch 1/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - f1_score: 0.6624 - loss: 0.6899 - precision: 0.5168 - recall: 0.8069 - val_f1_score: 0.3382 - val_loss: 0.6994 - val_precision: 0.2365 - val_recall: 0.7297
Epoch 2/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6821 - precision: 0.5615 - recall: 0.7367 - val_f1_score: 0.3382 - val_loss: 0.6840 - val_precision: 0.2676 - val_recall: 0.6634
Epoch 3/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6752 - precision: 0.6076 - recall: 0.6901 - val_f1_score: 0.3382 - val_loss: 0.6721 - val_precision: 0.3047 - val_recall: 0.6437
Epoch 4/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6688 - precision: 0.6357 - recall: 0.6602 - val_f1_score: 0.3382 - val_loss: 0.6623 - val_precision: 0.3333 - val_recall: 0.6413
Epoch 5/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6625 - precision: 0.6594 - recall: 0.6457 - val_f1_score: 0.3382 - val_loss: 0.6537 - val_precision: 0.3409 - val_recall: 0.6241
Epoch 6/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.6561 - precision: 0.6671 - recall: 0.6354 - val_f1_score: 0.3382 - val_loss: 0.6458 - val_precision: 0.3503 - val_recall: 0.6265
Epoch 7/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.6496 - precision: 0.6748 - recall: 0.6347 - val_f1_score: 0.3382 - val_loss: 0.6384 - val_precision: 0.3522 - val_recall: 0.6265
Epoch 8/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6624 - loss: 0.6431 - precision: 0.6781 - recall: 0.6389 - val_f1_score: 0.3382 - val_loss: 0.6312 - val_precision: 0.3581 - val_recall: 0.6388
Epoch 9/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6367 - precision: 0.6865 - recall: 0.6447 - val_f1_score: 0.3382 - val_loss: 0.6244 - val_precision: 0.3598 - val_recall: 0.6462
Epoch 10/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6302 - precision: 0.6893 - recall: 0.6513 - val_f1_score: 0.3382 - val_loss: 0.6179 - val_precision: 0.3628 - val_recall: 0.6462
Epoch 11/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.6240 - precision: 0.6914 - recall: 0.6586 - val_f1_score: 0.3382 - val_loss: 0.6117 - val_precision: 0.3654 - val_recall: 0.6437
Epoch 12/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.6179 - precision: 0.6931 - recall: 0.6628 - val_f1_score: 0.3382 - val_loss: 0.6061 - val_precision: 0.3654 - val_recall: 0.6437
Epoch 13/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6120 - precision: 0.6952 - recall: 0.6695 - val_f1_score: 0.3382 - val_loss: 0.6007 - val_precision: 0.3628 - val_recall: 0.6462
Epoch 14/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.6066 - precision: 0.6969 - recall: 0.6710 - val_f1_score: 0.3382 - val_loss: 0.5959 - val_precision: 0.3625 - val_recall: 0.6413
Epoch 15/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.6015 - precision: 0.6972 - recall: 0.6729 - val_f1_score: 0.3382 - val_loss: 0.5918 - val_precision: 0.3651 - val_recall: 0.6486
Epoch 16/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5968 - precision: 0.7000 - recall: 0.6808 - val_f1_score: 0.3382 - val_loss: 0.5880 - val_precision: 0.3643 - val_recall: 0.6462
Epoch 17/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.5926 - precision: 0.7012 - recall: 0.6850 - val_f1_score: 0.3382 - val_loss: 0.5848 - val_precision: 0.3639 - val_recall: 0.6437
Epoch 18/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.5888 - precision: 0.7043 - recall: 0.6894 - val_f1_score: 0.3382 - val_loss: 0.5821 - val_precision: 0.3668 - val_recall: 0.6462
Epoch 19/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5854 - precision: 0.7050 - recall: 0.6920 - val_f1_score: 0.3382 - val_loss: 0.5796 - val_precision: 0.3667 - val_recall: 0.6486
Epoch 20/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5824 - precision: 0.7059 - recall: 0.6936 - val_f1_score: 0.3382 - val_loss: 0.5773 - val_precision: 0.3672 - val_recall: 0.6486
Epoch 21/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5797 - precision: 0.7057 - recall: 0.6959 - val_f1_score: 0.3382 - val_loss: 0.5755 - val_precision: 0.3694 - val_recall: 0.6536
Epoch 22/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5774 - precision: 0.7089 - recall: 0.6988 - val_f1_score: 0.3382 - val_loss: 0.5739 - val_precision: 0.3712 - val_recall: 0.6585
Epoch 23/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5752 - precision: 0.7103 - recall: 0.6999 - val_f1_score: 0.3382 - val_loss: 0.5726 - val_precision: 0.3726 - val_recall: 0.6609
Epoch 24/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5733 - precision: 0.7107 - recall: 0.7020 - val_f1_score: 0.3382 - val_loss: 0.5715 - val_precision: 0.3715 - val_recall: 0.6609
Epoch 25/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.5716 - precision: 0.7124 - recall: 0.7063 - val_f1_score: 0.3382 - val_loss: 0.5703 - val_precision: 0.3726 - val_recall: 0.6609
Epoch 26/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5700 - precision: 0.7137 - recall: 0.7089 - val_f1_score: 0.3382 - val_loss: 0.5694 - val_precision: 0.3731 - val_recall: 0.6683
Epoch 27/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5685 - precision: 0.7146 - recall: 0.7090 - val_f1_score: 0.3382 - val_loss: 0.5684 - val_precision: 0.3723 - val_recall: 0.6658
Epoch 28/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5671 - precision: 0.7157 - recall: 0.7123 - val_f1_score: 0.3382 - val_loss: 0.5673 - val_precision: 0.3719 - val_recall: 0.6634
Epoch 29/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5658 - precision: 0.7163 - recall: 0.7129 - val_f1_score: 0.3382 - val_loss: 0.5665 - val_precision: 0.3733 - val_recall: 0.6658
Epoch 30/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.5645 - precision: 0.7174 - recall: 0.7150 - val_f1_score: 0.3382 - val_loss: 0.5656 - val_precision: 0.3743 - val_recall: 0.6658
Epoch 31/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.5633 - precision: 0.7177 - recall: 0.7168 - val_f1_score: 0.3382 - val_loss: 0.5648 - val_precision: 0.3771 - val_recall: 0.6708
Epoch 32/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5622 - precision: 0.7178 - recall: 0.7187 - val_f1_score: 0.3382 - val_loss: 0.5639 - val_precision: 0.3790 - val_recall: 0.6732
Epoch 33/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5611 - precision: 0.7190 - recall: 0.7223 - val_f1_score: 0.3382 - val_loss: 0.5632 - val_precision: 0.3781 - val_recall: 0.6708
Epoch 34/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5600 - precision: 0.7199 - recall: 0.7257 - val_f1_score: 0.3382 - val_loss: 0.5623 - val_precision: 0.3786 - val_recall: 0.6708
Epoch 35/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.5590 - precision: 0.7197 - recall: 0.7260 - val_f1_score: 0.3382 - val_loss: 0.5615 - val_precision: 0.3792 - val_recall: 0.6708
Epoch 36/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5580 - precision: 0.7210 - recall: 0.7275 - val_f1_score: 0.3382 - val_loss: 0.5606 - val_precision: 0.3783 - val_recall: 0.6683
Epoch 37/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.5571 - precision: 0.7216 - recall: 0.7283 - val_f1_score: 0.3382 - val_loss: 0.5599 - val_precision: 0.3773 - val_recall: 0.6683
Epoch 38/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - f1_score: 0.6624 - loss: 0.5561 - precision: 0.7229 - recall: 0.7302 - val_f1_score: 0.3382 - val_loss: 0.5591 - val_precision: 0.3759 - val_recall: 0.6658
Epoch 39/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5552 - precision: 0.7224 - recall: 0.7311 - val_f1_score: 0.3382 - val_loss: 0.5585 - val_precision: 0.3764 - val_recall: 0.6658
Epoch 40/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5544 - precision: 0.7228 - recall: 0.7330 - val_f1_score: 0.3382 - val_loss: 0.5579 - val_precision: 0.3778 - val_recall: 0.6683
Epoch 41/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5535 - precision: 0.7228 - recall: 0.7325 - val_f1_score: 0.3382 - val_loss: 0.5572 - val_precision: 0.3792 - val_recall: 0.6708
Epoch 42/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5527 - precision: 0.7239 - recall: 0.7327 - val_f1_score: 0.3382 - val_loss: 0.5564 - val_precision: 0.3786 - val_recall: 0.6708
Epoch 43/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5519 - precision: 0.7239 - recall: 0.7340 - val_f1_score: 0.3382 - val_loss: 0.5558 - val_precision: 0.3786 - val_recall: 0.6708
Epoch 44/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5511 - precision: 0.7240 - recall: 0.7336 - val_f1_score: 0.3382 - val_loss: 0.5553 - val_precision: 0.3792 - val_recall: 0.6708
Epoch 45/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5503 - precision: 0.7241 - recall: 0.7358 - val_f1_score: 0.3382 - val_loss: 0.5549 - val_precision: 0.3800 - val_recall: 0.6732
Epoch 46/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5496 - precision: 0.7238 - recall: 0.7360 - val_f1_score: 0.3382 - val_loss: 0.5544 - val_precision: 0.3814 - val_recall: 0.6757
Epoch 47/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5488 - precision: 0.7242 - recall: 0.7372 - val_f1_score: 0.3382 - val_loss: 0.5539 - val_precision: 0.3837 - val_recall: 0.6806
Epoch 48/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5481 - precision: 0.7254 - recall: 0.7393 - val_f1_score: 0.3382 - val_loss: 0.5533 - val_precision: 0.3853 - val_recall: 0.6806
Epoch 49/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - f1_score: 0.6624 - loss: 0.5474 - precision: 0.7259 - recall: 0.7399 - val_f1_score: 0.3382 - val_loss: 0.5528 - val_precision: 0.3853 - val_recall: 0.6806
Epoch 50/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - f1_score: 0.6624 - loss: 0.5467 - precision: 0.7257 - recall: 0.7387 - val_f1_score: 0.3382 - val_loss: 0.5522 - val_precision: 0.3853 - val_recall: 0.6806
Epoch 51/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5460 - precision: 0.7256 - recall: 0.7385 - val_f1_score: 0.3382 - val_loss: 0.5517 - val_precision: 0.3853 - val_recall: 0.6806
Epoch 52/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5453 - precision: 0.7257 - recall: 0.7390 - val_f1_score: 0.3382 - val_loss: 0.5512 - val_precision: 0.3858 - val_recall: 0.6806
Epoch 53/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.5446 - precision: 0.7258 - recall: 0.7392 - val_f1_score: 0.3382 - val_loss: 0.5508 - val_precision: 0.3841 - val_recall: 0.6757
Epoch 54/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5440 - precision: 0.7254 - recall: 0.7403 - val_f1_score: 0.3382 - val_loss: 0.5504 - val_precision: 0.3841 - val_recall: 0.6757
Epoch 55/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - f1_score: 0.6624 - loss: 0.5433 - precision: 0.7251 - recall: 0.7414 - val_f1_score: 0.3382 - val_loss: 0.5499 - val_precision: 0.3857 - val_recall: 0.6757
Epoch 56/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5427 - precision: 0.7255 - recall: 0.7427 - val_f1_score: 0.3382 - val_loss: 0.5494 - val_precision: 0.3862 - val_recall: 0.6757
Epoch 57/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5420 - precision: 0.7262 - recall: 0.7442 - val_f1_score: 0.3382 - val_loss: 0.5490 - val_precision: 0.3865 - val_recall: 0.6732
Epoch 58/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5414 - precision: 0.7263 - recall: 0.7448 - val_f1_score: 0.3382 - val_loss: 0.5486 - val_precision: 0.3865 - val_recall: 0.6732
Epoch 59/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.5408 - precision: 0.7264 - recall: 0.7467 - val_f1_score: 0.3382 - val_loss: 0.5483 - val_precision: 0.3865 - val_recall: 0.6732
Epoch 60/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5401 - precision: 0.7270 - recall: 0.7485 - val_f1_score: 0.3382 - val_loss: 0.5479 - val_precision: 0.3859 - val_recall: 0.6732
Epoch 61/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5395 - precision: 0.7274 - recall: 0.7500 - val_f1_score: 0.3382 - val_loss: 0.5475 - val_precision: 0.3854 - val_recall: 0.6732
Epoch 62/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5389 - precision: 0.7275 - recall: 0.7504 - val_f1_score: 0.3382 - val_loss: 0.5472 - val_precision: 0.3848 - val_recall: 0.6732
Epoch 63/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5383 - precision: 0.7287 - recall: 0.7515 - val_f1_score: 0.3382 - val_loss: 0.5469 - val_precision: 0.3852 - val_recall: 0.6757
Epoch 64/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5377 - precision: 0.7286 - recall: 0.7513 - val_f1_score: 0.3382 - val_loss: 0.5465 - val_precision: 0.3874 - val_recall: 0.6806
Epoch 65/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.5371 - precision: 0.7295 - recall: 0.7517 - val_f1_score: 0.3382 - val_loss: 0.5462 - val_precision: 0.3863 - val_recall: 0.6806
Epoch 66/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5365 - precision: 0.7297 - recall: 0.7527 - val_f1_score: 0.3382 - val_loss: 0.5458 - val_precision: 0.3863 - val_recall: 0.6806
Epoch 67/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5359 - precision: 0.7301 - recall: 0.7533 - val_f1_score: 0.3382 - val_loss: 0.5455 - val_precision: 0.3863 - val_recall: 0.6806
Epoch 68/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5353 - precision: 0.7305 - recall: 0.7537 - val_f1_score: 0.3382 - val_loss: 0.5451 - val_precision: 0.3869 - val_recall: 0.6806
Epoch 69/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5348 - precision: 0.7309 - recall: 0.7543 - val_f1_score: 0.3382 - val_loss: 0.5447 - val_precision: 0.3885 - val_recall: 0.6806
Epoch 70/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5342 - precision: 0.7308 - recall: 0.7543 - val_f1_score: 0.3382 - val_loss: 0.5444 - val_precision: 0.3913 - val_recall: 0.6855
Epoch 71/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.5336 - precision: 0.7310 - recall: 0.7539 - val_f1_score: 0.3382 - val_loss: 0.5440 - val_precision: 0.3916 - val_recall: 0.6880
Epoch 72/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5330 - precision: 0.7311 - recall: 0.7545 - val_f1_score: 0.3382 - val_loss: 0.5437 - val_precision: 0.3933 - val_recall: 0.6880
Epoch 73/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5324 - precision: 0.7311 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5434 - val_precision: 0.3947 - val_recall: 0.6904
Epoch 74/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5318 - precision: 0.7311 - recall: 0.7535 - val_f1_score: 0.3382 - val_loss: 0.5431 - val_precision: 0.3947 - val_recall: 0.6904
Epoch 75/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5313 - precision: 0.7321 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5427 - val_precision: 0.3952 - val_recall: 0.6904
Epoch 76/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5307 - precision: 0.7327 - recall: 0.7548 - val_f1_score: 0.3382 - val_loss: 0.5424 - val_precision: 0.3958 - val_recall: 0.6904
Epoch 77/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.5301 - precision: 0.7331 - recall: 0.7548 - val_f1_score: 0.3382 - val_loss: 0.5420 - val_precision: 0.3955 - val_recall: 0.6929
Epoch 78/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5295 - precision: 0.7334 - recall: 0.7557 - val_f1_score: 0.3382 - val_loss: 0.5417 - val_precision: 0.3958 - val_recall: 0.6904
Epoch 79/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5290 - precision: 0.7340 - recall: 0.7553 - val_f1_score: 0.3382 - val_loss: 0.5413 - val_precision: 0.3961 - val_recall: 0.6929
Epoch 80/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5284 - precision: 0.7350 - recall: 0.7550 - val_f1_score: 0.3382 - val_loss: 0.5409 - val_precision: 0.3966 - val_recall: 0.6929
Epoch 81/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5278 - precision: 0.7351 - recall: 0.7551 - val_f1_score: 0.3382 - val_loss: 0.5405 - val_precision: 0.3958 - val_recall: 0.6904
Epoch 82/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5272 - precision: 0.7359 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5401 - val_precision: 0.3960 - val_recall: 0.6880
Epoch 83/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.5266 - precision: 0.7356 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5398 - val_precision: 0.3949 - val_recall: 0.6880
Epoch 84/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5260 - precision: 0.7353 - recall: 0.7551 - val_f1_score: 0.3382 - val_loss: 0.5395 - val_precision: 0.3955 - val_recall: 0.6880
Epoch 85/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5254 - precision: 0.7361 - recall: 0.7551 - val_f1_score: 0.3382 - val_loss: 0.5391 - val_precision: 0.3941 - val_recall: 0.6855
Epoch 86/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - f1_score: 0.6624 - loss: 0.5249 - precision: 0.7359 - recall: 0.7540 - val_f1_score: 0.3382 - val_loss: 0.5388 - val_precision: 0.3941 - val_recall: 0.6855
Epoch 87/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5243 - precision: 0.7357 - recall: 0.7541 - val_f1_score: 0.3382 - val_loss: 0.5384 - val_precision: 0.3935 - val_recall: 0.6855
Epoch 88/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5237 - precision: 0.7364 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5380 - val_precision: 0.3952 - val_recall: 0.6855
Epoch 89/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.5231 - precision: 0.7371 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5377 - val_precision: 0.3952 - val_recall: 0.6855
Epoch 90/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5225 - precision: 0.7365 - recall: 0.7543 - val_f1_score: 0.3382 - val_loss: 0.5373 - val_precision: 0.3963 - val_recall: 0.6855
Epoch 91/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5219 - precision: 0.7367 - recall: 0.7538 - val_f1_score: 0.3382 - val_loss: 0.5370 - val_precision: 0.3963 - val_recall: 0.6855
Epoch 92/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5213 - precision: 0.7373 - recall: 0.7540 - val_f1_score: 0.3382 - val_loss: 0.5366 - val_precision: 0.3963 - val_recall: 0.6855
Epoch 93/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5207 - precision: 0.7376 - recall: 0.7548 - val_f1_score: 0.3382 - val_loss: 0.5362 - val_precision: 0.3952 - val_recall: 0.6855
Epoch 94/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5201 - precision: 0.7385 - recall: 0.7557 - val_f1_score: 0.3382 - val_loss: 0.5358 - val_precision: 0.3966 - val_recall: 0.6880
Epoch 95/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5195 - precision: 0.7385 - recall: 0.7556 - val_f1_score: 0.3382 - val_loss: 0.5352 - val_precision: 0.3952 - val_recall: 0.6855
Epoch 96/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5189 - precision: 0.7385 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5347 - val_precision: 0.3963 - val_recall: 0.6855
Epoch 97/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5183 - precision: 0.7391 - recall: 0.7550 - val_f1_score: 0.3382 - val_loss: 0.5344 - val_precision: 0.3960 - val_recall: 0.6830
Epoch 98/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - f1_score: 0.6624 - loss: 0.5177 - precision: 0.7399 - recall: 0.7548 - val_f1_score: 0.3382 - val_loss: 0.5340 - val_precision: 0.3986 - val_recall: 0.6855
Epoch 99/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - f1_score: 0.6624 - loss: 0.5171 - precision: 0.7406 - recall: 0.7546 - val_f1_score: 0.3382 - val_loss: 0.5335 - val_precision: 0.3986 - val_recall: 0.6855
Epoch 100/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5164 - precision: 0.7410 - recall: 0.7550 - val_f1_score: 0.3382 - val_loss: 0.5331 - val_precision: 0.3989 - val_recall: 0.6880
```

```
In [ ]: print("Time taken in seconds ",end-start)
```

```
Time taken in seconds  224.04781579971313
```

Plotting Training and Validation Loss

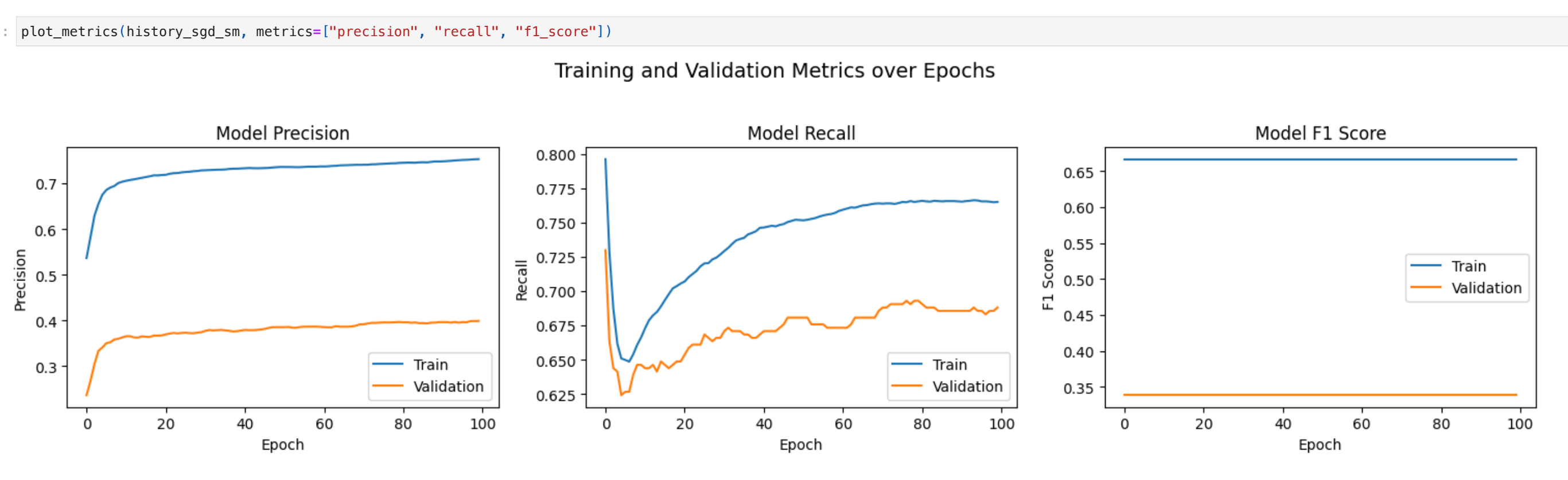
```
In [ ]: #Plotting Train Loss vs Validation Loss
        plt.plot(history_sgd_sm.history['loss'])
        plt.plot(history_sgd_sm.history['val_loss'])
        plt.title('model loss')
        plt.ylabel('Loss')
        plt.xlabel('Epoch')
        plt.legend(['train', 'validation'], loc='upper left')
        plt.show()
```



- Both Training and Validation losses are seen decreasing smoothly. This indicates that the model is improving its performance over time, without large fluctuations.
- The losses decreasing together suggest the model is learning general patterns.
- The widening gap indicates **overfitting**

Plotting the Precision, Recall and F1 scores

```
In [ ]: plot_metrics(history_sgd_sm, metrics=["precision", "recall", "f1_score"])
```



Precision Metric- Overfitting

- Training precision increases steadily though the training and ends around 0.75, while validation precision starts very low and reaches around 0.40.
- There is a large and continuously increasing gap between the two precision metrics.

Recall Metric - Overfitting

- Training recall starts high and has a sudden dip before recovering and increasing steadily. Validation recall follows the initial dip but rises to a lower values with more fluctuations.
- Similar to precision, a noticeable gap develops between the two recall metrics.

F1 Metric - Poor performance

- There is a huge gap between the two F1 scores.
- The model has failed to generalize and performs poorly on unseen data.

```
In [ ]: #Predicting the results using best as a threshold for training data
        y_train_pred = model_sgd_sm.predict(X_train_sm)
        y_train_pred = (y_train_pred > 0.5)
        y_train_pred
```

```
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

```
Out[ ]: array([[ True],
               [ True],
               [ True],
               ...,
               [ True],
               [False],
               [ True]])
```

```
In [ ]: #Predicting the results using best as a threshold for validation data
        y_val_pred = model_sgd_sm.predict(X_val)
        y_val_pred = (y_val_pred > 0.5)
        y_val_pred
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
Out[ ]: array([[False],
               [False],
               [ True],
               ...,
               [False],
               [False],
               [ True]])
```

```
In [ ]: model_name = "NN with SGD Smote"
        train_scores_df.loc[model_name] = recall_score(y_train_sm, y_train_pred)
        val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

Classification Report

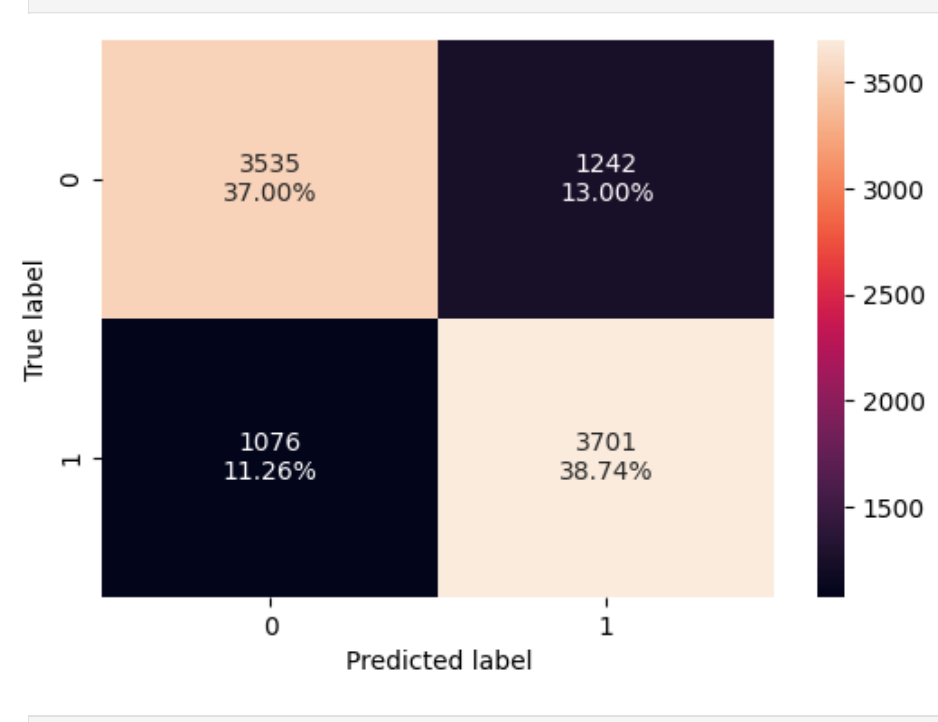
```
In [ ]: #Training classification report
        cr = classification_report(y_train_sm, y_train_pred)
        print(cr)
```

```
              precision    recall  f1-score   support

           0       0.77      0.74      0.75      4777
           1       0.75      0.77      0.76      4777

    accuracy                           0.76      9554
   macro avg       0.76      0.76      0.76      9554
weighted avg       0.76      0.76      0.76      9554
```

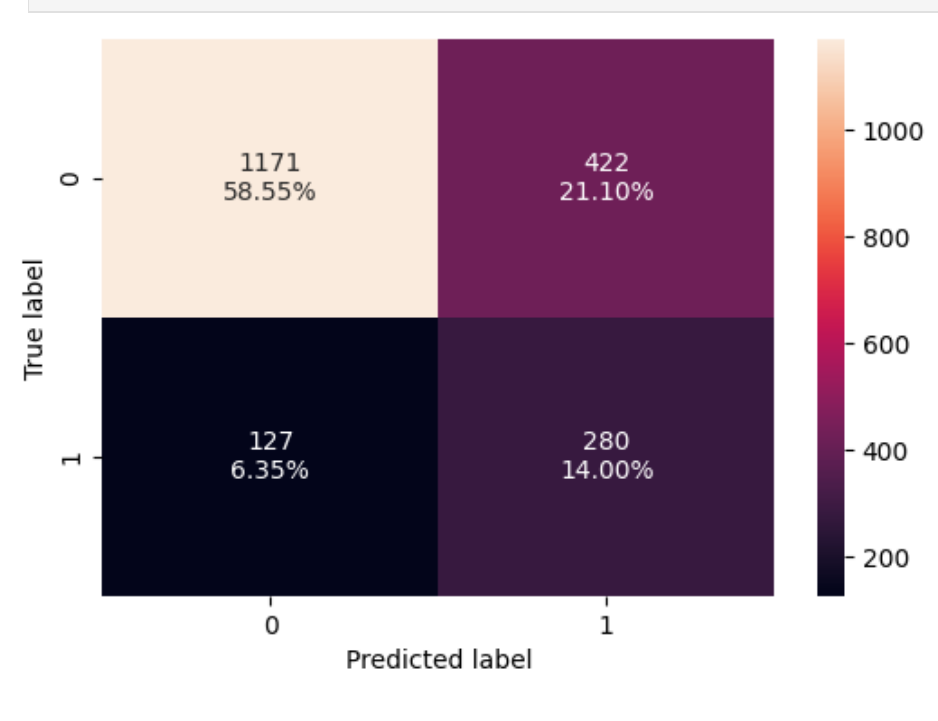
```
In [ ]: #Validation classification report
        cr = classification_report(y_val, y_val_pred)
        print(cr)
```

```
              precision    recall  f1-score   support

           0       0.90      0.74      0.81      1593
           1       0.40      0.69      0.50       407

    accuracy                           0.73      2000
   macro avg       0.65      0.71      0.66      2000
weighted avg       0.80      0.73      0.75      2000
```

```
In [ ]: # Confusion matrix for training
        make_confusion_matrix(y_train_sm, y_train_pred)
```



```
In [ ]: # Confusion matrix for validation
        make_confusion_matrix(y_val, y_val_pred)
```



- The model performs well and consistently on the majority class (high precision)
- The model is finding a substantial portion of the minority class (higher recall), but, the precision is very low (high number of false positives).
- The model performs poorly on unseen data and did not generalize well (confirms overfitting).

Neural Network with Balanced Data (by applying SMOTE) and Adam Optimizer

Instantiate NN

```
In [ ]: # Initialization
        backend.clear_session()
        np.random.seed(RANDOM_STATE)
        random.seed(RANDOM_STATE)
        tf.random.set_seed(RANDOM_STATE)
```

```
In [ ]: # Initialize the neural network with Adam Optimizer
        model_ao_sm = Sequential()

        # Add an input layer with 128 neurons and Relu as the activation function
        model_ao_sm.add(Dense(units=128, activation='relu', input_shape=(X_train_sm.shape[1],)))

        # Add an hidden layer with 64 neurons and Relu as the activation function
        model_ao_sm.add(Dense(units=64, activation='relu'))

        # Add an hidden layer with 64 neurons and Relu as the activation function
        model_ao_sm.add(Dense(units=64, activation='relu'))

        # Add an output layer with 1 neuron and Sigmoid as the activation function
        # as this is a binary classification problem
        model_ao_sm.add(Dense(units=1, activation='sigmoid'))
```

```
In [ ]: #Code to use Adam as the optimizer.
        optimizer = tf.keras.optimizers.Adam(0.001)

        #Choose Precision, Recall and F1 score as the metric
        metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]
```

```
In [ ]: # Compile the model with loss function of binary cross entropy and F1score as the metric
        model_ao_sm.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

```
In [ ]: model_ao_sm.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,536
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 1)	65

Total params: 14,017 (54.75 KB)
Trainable params: 14,017 (54.75 KB)
Non-trainable params: 0 (0.00 B)

```
In [ ]: start = time.time()

        history_ao_sm = model_ao_sm.fit(
        X_train_sm, y_train_sm,
        batch_size=32,
        epochs=100,
        verbose=1,
        validation_data = (X_val,y_val)
        )

        end = time.time()
```

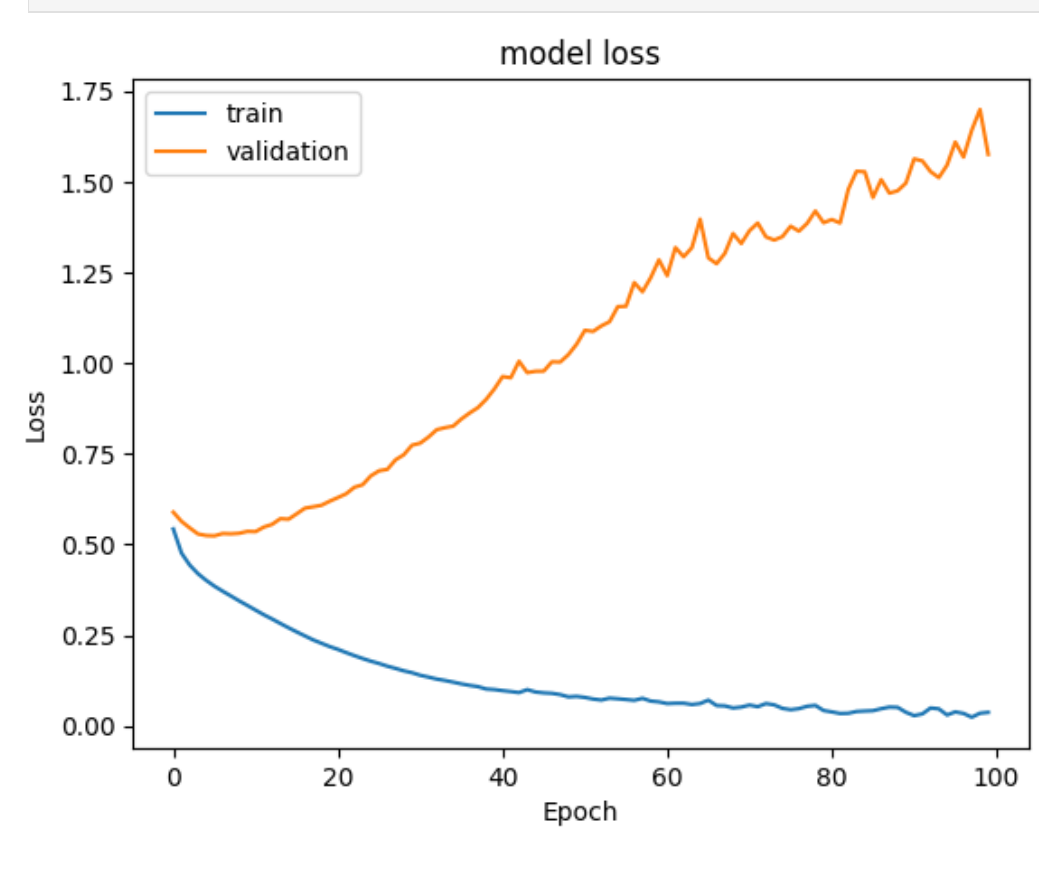
```
Epoch 1/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - f1_score: 0.6624 - loss: 0.5889 - precision: 0.6989 - recall: 0.6485 - val_f1_score: 0.3382 - val_loss: 0.5893 - val_precision: 0.3761 - val_recall: 0.7641
Epoch 2/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.6624 - loss: 0.4943 - precision: 0.7565 - recall: 0.7557 - val_f1_score: 0.3382 - val_loss: 0.5639 - val_precision: 0.4062 - val_recall: 0.7985
Epoch 3/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4599 - precision: 0.7786 - recall: 0.7805 - val_f1_score: 0.3382 - val_loss: 0.5460 - val_precision: 0.4225 - val_recall: 0.8108
Epoch 4/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4360 - precision: 0.7918 - recall: 0.7965 - val_f1_score: 0.3382 - val_loss: 0.5293 - val_precision: 0.4402 - val_recall: 0.8133
Epoch 5/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.6624 - loss: 0.4167 - precision: 0.7996 - recall: 0.8076 - val_f1_score: 0.3382 - val_loss: 0.5250 - val_precision: 0.4403 - val_recall: 0.8059
Epoch 6/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6624 - loss: 0.4010 - precision: 0.8075 - recall: 0.8209 - val_f1_score: 0.3382 - val_loss: 0.5242 - val_precision: 0.4529 - val_recall: 0.8157
Epoch 7/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.3869 - precision: 0.8166 - recall: 0.8292 - val_f1_score: 0.3382 - val_loss: 0.5305 - val_precision: 0.4501 - val_recall: 0.8084
Epoch 8/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.3737 - precision: 0.8235 - recall: 0.8341 - val_f1_score: 0.3382 - val_loss: 0.5297 - val_precision: 0.4540 - val_recall: 0.8010
Epoch 9/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.3601 - precision: 0.8299 - recall: 0.8485 - val_f1_score: 0.3382 - val_loss: 0.5311 - val_precision: 0.4531 - val_recall: 0.7961
Epoch 10/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.3474 - precision: 0.8329 - recall: 0.8557 - val_f1_score: 0.3382 - val_loss: 0.5364 - val_precision: 0.4526 - val_recall: 0.7862
Epoch 11/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6624 - loss: 0.3338 - precision: 0.8451 - recall: 0.8704 - val_f1_score: 0.3382 - val_loss: 0.5358 - val_precision: 0.4495 - val_recall: 0.7764
Epoch 12/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.3209 - precision: 0.8474 - recall: 0.8810 - val_f1_score: 0.3382 - val_loss: 0.5483 - val_precision: 0.4503 - val_recall: 0.7789
Epoch 13/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.3092 - precision: 0.8523 - recall: 0.8871 - val_f1_score: 0.3382 - val_loss: 0.5558 - val_precision: 0.4510 - val_recall: 0.7690
Epoch 14/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.2967 - precision: 0.8622 - recall: 0.8892 - val_f1_score: 0.3382 - val_loss: 0.5714 - val_precision: 0.4499 - val_recall: 0.7617
Epoch 15/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.2849 - precision: 0.8664 - recall: 0.8945 - val_f1_score: 0.3382 - val_loss: 0.5700 - val_precision: 0.4536 - val_recall: 0.7445
Epoch 16/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.2729 - precision: 0.8724 - recall: 0.8990 - val_f1_score: 0.3382 - val_loss: 0.5850 - val_precision: 0.4541 - val_recall: 0.7420
Epoch 17/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.2621 - precision: 0.8783 - recall: 0.9039 - val_f1_score: 0.3382 - val_loss: 0.6005 - val_precision: 0.4466 - val_recall: 0.7396
Epoch 18/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.2511 - precision: 0.8855 - recall: 0.9087 - val_f1_score: 0.3382 - val_loss: 0.6043 - val_precision: 0.4470 - val_recall: 0.7248
Epoch 19/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.2420 - precision: 0.8902 - recall: 0.9117 - val_f1_score: 0.3382 - val_loss: 0.6084 - val_precision: 0.4591 - val_recall: 0.7174
Epoch 20/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.2312 - precision: 0.8977 - recall: 0.9174 - val_f1_score: 0.3382 - val_loss: 0.6196 - val_precision: 0.4580 - val_recall: 0.7101
Epoch 21/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.2231 - precision: 0.9005 - recall: 0.9222 - val_f1_score: 0.3382 - val_loss: 0.6297 - val_precision: 0.4595 - val_recall: 0.7101
Epoch 22/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6625 - loss: 0.2128 - precision: 0.9089 - recall: 0.9279 - val_f1_score: 0.3383 - val_loss: 0.6401 - val_precision: 0.4634 - val_recall: 0.7002
Epoch 23/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6625 - loss: 0.2032 - precision: 0.9134 - recall: 0.9335 - val_f1_score: 0.3383 - val_loss: 0.6580 - val_precision: 0.4646 - val_recall: 0.6929
Epoch 24/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6625 - loss: 0.1954 - precision: 0.9159 - recall: 0.9384 - val_f1_score: 0.3383 - val_loss: 0.6650 - val_precision: 0.4679 - val_recall: 0.6978
Epoch 25/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6625 - loss: 0.1877 - precision: 0.9229 - recall: 0.9407 - val_f1_score: 0.3385 - val_loss: 0.6896 - val_precision: 0.4650 - val_recall: 0.7027
Epoch 26/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6626 - loss: 0.1818 - precision: 0.9247 - recall: 0.9421 - val_f1_score: 0.3385 - val_loss: 0.7035 - val_precision: 0.4624 - val_recall: 0.6953
Epoch 27/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6628 - loss: 0.1750 - precision: 0.9259 - recall: 0.9436 - val_f1_score: 0.3385 - val_loss: 0.7075 - val_precision: 0.4660 - val_recall: 0.6732
Epoch 28/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6628 - loss: 0.1676 - precision: 0.9295 - recall: 0.9440 - val_f1_score: 0.3387 - val_loss: 0.7336 - val_precision: 0.4644 - val_recall: 0.6732
Epoch 29/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6628 - loss: 0.1616 - precision: 0.9333 - recall: 0.9466 - val_f1_score: 0.3387 - val_loss: 0.7477 - val_precision: 0.4671 - val_recall: 0.6634
Epoch 30/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6628 - loss: 0.1551 - precision: 0.9343 - recall: 0.9505 - val_f1_score: 0.3387 - val_loss: 0.7740 - val_precision: 0.4613 - val_recall: 0.6732
Epoch 31/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6629 - loss: 0.1491 - precision: 0.9374 - recall: 0.9507 - val_f1_score: 0.3390 - val_loss: 0.7797 - val_precision: 0.4638 - val_recall: 0.6462
Epoch 32/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6630 - loss: 0.1428 - precision: 0.9385 - recall: 0.9524 - val_f1_score: 0.3390 - val_loss: 0.7965 - val_precision: 0.4663 - val_recall: 0.6462
Epoch 33/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6633 - loss: 0.1384 - precision: 0.9399 - recall: 0.9538 - val_f1_score: 0.3402 - val_loss: 0.8167 - val_precision: 0.4737 - val_recall: 0.6413
Epoch 34/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6636 - loss: 0.1349 - precision: 0.9423 - recall: 0.9541 - val_f1_score: 0.3409 - val_loss: 0.8223 - val_precision: 0.4848 - val_recall: 0.6290
Epoch 35/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6637 - loss: 0.1293 - precision: 0.9458 - recall: 0.9580 - val_f1_score: 0.3414 - val_loss: 0.8266 - val_precision: 0.4918 - val_recall: 0.5897
Epoch 36/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6640 - loss: 0.1220 - precision: 0.9490 - recall: 0.9618 - val_f1_score: 0.3417 - val_loss: 0.8463 - val_precision: 0.4921 - val_recall: 0.6118
Epoch 37/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6645 - loss: 0.1179 - precision: 0.9505 - recall: 0.9604 - val_f1_score: 0.3430 - val_loss: 0.8628 - val_precision: 0.4970 - val_recall: 0.6069
Epoch 38/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6651 - loss: 0.1138 - precision: 0.9551 - recall: 0.9649 - val_f1_score: 0.3433 - val_loss: 0.8775 - val_precision: 0.5107 - val_recall: 0.5848
Epoch 39/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6659 - loss: 0.1066 - precision: 0.9555 - recall: 0.9678 - val_f1_score: 0.3449 - val_loss: 0.9001 - val_precision: 0.5166 - val_recall: 0.5749
Epoch 40/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6663 - loss: 0.1054 - precision: 0.9559 - recall: 0.9658 - val_f1_score: 0.3458 - val_loss: 0.9289 - val_precision: 0.5293 - val_recall: 0.5332
Epoch 41/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6677 - loss: 0.1022 - precision: 0.9586 - recall: 0.9688 - val_f1_score: 0.3478 - val_loss: 0.9631 - val_precision: 0.5125 - val_recall: 0.5528
Epoch 42/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6687 - loss: 0.0993 - precision: 0.9603 - recall: 0.9684 - val_f1_score: 0.3494 - val_loss: 0.9599 - val_precision: 0.5182 - val_recall: 0.5602
Epoch 43/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6704 - loss: 0.0961 - precision: 0.9621 - recall: 0.9724 - val_f1_score: 0.3473 - val_loss: 1.0054 - val_precision: 0.4937 - val_recall: 0.5749
Epoch 44/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6704 - loss: 0.1047 - precision: 0.9589 - recall: 0.9656 - val_f1_score: 0.3464 - val_loss: 0.9740 - val_precision: 0.5022 - val_recall: 0.5602
Epoch 45/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6715 - loss: 0.1004 - precision: 0.9579 - recall: 0.9671 - val_f1_score: 0.3473 - val_loss: 0.9777 - val_precision: 0.5211 - val_recall: 0.5455
Epoch 46/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6720 - loss: 0.0964 - precision: 0.9642 - recall: 0.9714 - val_f1_score: 0.3493 - val_loss: 0.9782 - val_precision: 0.5386 - val_recall: 0.5479
Epoch 47/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6735 - loss: 0.0946 - precision: 0.9625 - recall: 0.9738 - val_f1_score: 0.3467 - val_loss: 1.0040 - val_precision: 0.5032 - val_recall: 0.5774
Epoch 48/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6734 - loss: 0.0928 - precision: 0.9639 - recall: 0.9720 - val_f1_score: 0.3471 - val_loss: 1.0031 - val_precision: 0.5078 - val_recall: 0.5577
Epoch 49/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6752 - loss: 0.0860 - precision: 0.9674 - recall: 0.9754 - val_f1_score: 0.3477 - val_loss: 1.0237 - val_precision: 0.5129 - val_recall: 0.5872
Epoch 50/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6760 - loss: 0.0876 - precision: 0.9661 - recall: 0.9728 - val_f1_score: 0.3498 - val_loss: 1.0525 - val_precision: 0.5022 - val_recall: 0.5504
Epoch 51/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6758 - loss: 0.0834 - precision: 0.9666 - recall: 0.9716 - val_f1_score: 0.3528 - val_loss: 1.0907 - val_precision: 0.5011 - val_recall: 0.5553
Epoch 52/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6776 - loss: 0.0818 - precision: 0.9692 - recall: 0.9769 - val_f1_score: 0.3552 - val_loss: 1.0877 - val_precision: 0.5112 - val_recall: 0.5602
Epoch 53/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6787 - loss: 0.0778 - precision: 0.9663 - recall: 0.9749 - val_f1_score: 0.3527 - val_loss: 1.1028 - val_precision: 0.5000 - val_recall: 0.5725
Epoch 54/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6800 - loss: 0.0838 - precision: 0.9634 - recall: 0.9761 - val_f1_score: 0.3581 - val_loss: 1.1143 - val_precision: 0.5134 - val_recall: 0.5651
Epoch 55/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6811 - loss: 0.0812 - precision: 0.9665 - recall: 0.9752 - val_f1_score: 0.3565 - val_loss: 1.1559 - val_precision: 0.4927 - val_recall: 0.5823
Epoch 56/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6817 - loss: 0.0793 - precision: 0.9647 - recall: 0.9745 - val_f1_score: 0.3580 - val_loss: 1.1569 - val_precision: 0.5065 - val_recall: 0.5749
Epoch 57/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6831 - loss: 0.0768 - precision: 0.9680 - recall: 0.9769 - val_f1_score: 0.3578 - val_loss: 1.2221 - val_precision: 0.4837 - val_recall: 0.5823
Epoch 58/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6840 - loss: 0.0802 - precision: 0.9664 - recall: 0.9705 - val_f1_score: 0.3602 - val_loss: 1.1969 - val_precision: 0.5000 - val_recall: 0.5995
Epoch 59/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6861 - loss: 0.0753 - precision: 0.9689 - recall: 0.9766 - val_f1_score: 0.3588 - val_loss: 1.2359 - val_precision: 0.5031 - val_recall: 0.6069
Epoch 60/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6865 - loss: 0.0736 - precision: 0.9692 - recall: 0.9730 - val_f1_score: 0.3590 - val_loss: 1.2848 - val_precision: 0.4731 - val_recall: 0.6044
Epoch 61/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6871 - loss: 0.0681 - precision: 0.9740 - recall: 0.9802 - val_f1_score: 0.3657 - val_loss: 1.2413 - val_precision: 0.5152 - val_recall: 0.5848
Epoch 62/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6878 - loss: 0.0657 - precision: 0.9737 - recall: 0.9769 - val_f1_score: 0.3634 - val_loss: 1.3187 - val_precision: 0.4931 - val_recall: 0.6118
Epoch 63/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6910 - loss: 0.0675 - precision: 0.9733 - recall: 0.9779 - val_f1_score: 0.3597 - val_loss: 1.2934 - val_precision: 0.4855 - val_recall: 0.6167
Epoch 64/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6915 - loss: 0.0630 - precision: 0.9748 - recall: 0.9801 - val_f1_score: 0.3637 - val_loss: 1.3184 - val_precision: 0.4823 - val_recall: 0.6020
Epoch 65/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6919 - loss: 0.0649 - precision: 0.9746 - recall: 0.9770 - val_f1_score: 0.3594 - val_loss: 1.3971 - val_precision: 0.4458 - val_recall: 0.6462
Epoch 66/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.6908 - loss: 0.0910 - precision: 0.9625 - recall: 0.9710 - val_f1_score: 0.3651 - val_loss: 1.2903 - val_precision: 0.5207 - val_recall: 0.5872
Epoch 67/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.6940 - loss: 0.0602 - precision: 0.9774 - recall: 0.9803 - val_f1_score: 0.3659 - val_loss: 1.2743 - val_precision: 0.5085 - val_recall: 0.5897
Epoch 68/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6953 - loss: 0.0572 - precision: 0.9783 - recall: 0.9798 - val_f1_score: 0.3667 - val_loss: 1.3026 - val_precision: 0.5139 - val_recall: 0.5921
Epoch 69/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6957 - loss: 0.0518 - precision: 0.9776 - recall: 0.9849 - val_f1_score: 0.3666 - val_loss: 1.3574 - val_precision: 0.5075 - val_recall: 0.5848
Epoch 70/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.6991 - loss: 0.0578 - precision: 0.9782 - recall: 0.9827 - val_f1_score: 0.3632 - val_loss: 1.3299 - val_precision: 0.5082 - val_recall: 0.6069
Epoch 71/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.6966 - loss: 0.0578 - precision: 0.9757 - recall: 0.9831 - val_f1_score: 0.3656 - val_loss: 1.3652 - val_precision: 0.5169 - val_recall: 0.5995
Epoch 72/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6990 - loss: 0.0489 - precision: 0.9817 - recall: 0.9835 - val_f1_score: 0.3724 - val_loss: 1.3864 - val_precision: 0.4872 - val_recall: 0.6093
Epoch 73/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6985 - loss: 0.0678 - precision: 0.9696 - recall: 0.9771 - val_f1_score: 0.3695 - val_loss: 1.3482 - val_precision: 0.5147 - val_recall: 0.6020
Epoch 74/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.7021 - loss: 0.0566 - precision: 0.9759 - recall: 0.9864 - val_f1_score: 0.3704 - val_loss: 1.3394 - val_precision: 0.5446 - val_recall: 0.5405
Epoch 75/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.7009 - loss: 0.0470 - precision: 0.9800 - recall: 0.9866 - val_f1_score: 0.3771 - val_loss: 1.3489 - val_precision: 0.5450 - val_recall: 0.5209
Epoch 76/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.7061 - loss: 0.0453 - precision: 0.9828 - recall: 0.9837 - val_f1_score: 0.3760 - val_loss: 1.3776 - val_precision: 0.5343 - val_recall: 0.5356
Epoch 77/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.7064 - loss: 0.0474 - precision: 0.9809 - recall: 0.9857 - val_f1_score: 0.3742 - val_loss: 1.3639 - val_precision: 0.5109 - val_recall: 0.5749
Epoch 78/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.7052 - loss: 0.0510 - precision: 0.9784 - recall: 0.9840 - val_f1_score: 0.3758 - val_loss: 1.3854 - val_precision: 0.5267 - val_recall: 0.5823
Epoch 79/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.7043 - loss: 0.0647 - precision: 0.9752 - recall: 0.9794 - val_f1_score: 0.3840 - val_loss: 1.4194 - val_precision: 0.5684 - val_recall: 0.5209
Epoch 80/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.7073 - loss: 0.0451 - precision: 0.9819 - recall: 0.9834 - val_f1_score: 0.3708 - val_loss: 1.3872 - val_precision: 0.4989 - val_recall: 0.5627
Epoch 81/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.7049 - loss: 0.0408 - precision: 0.9839 - recall: 0.9875 - val_f1_score: 0.3764 - val_loss: 1.3960 - val_precision: 0.5125 - val_recall: 0.5528
Epoch 82/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7118 - loss: 0.0391 - precision: 0.9845 - recall: 0.9882 - val_f1_score: 0.3844 - val_loss: 1.3864 - val_precision: 0.5257 - val_recall: 0.5528
Epoch 83/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.7098 - loss: 0.0371 - precision: 0.9841 - recall: 0.9879 - val_f1_score: 0.3791 - val_loss: 1.4791 - val_precision: 0.5075 - val_recall: 0.5799
Epoch 84/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.7127 - loss: 0.0416 - precision: 0.9839 - recall: 0.9899 - val_f1_score: 0.3956 - val_loss: 1.5290 - val_precision: 0.5642 - val_recall: 0.4963
Epoch 85/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.7141 - loss: 0.0457 - precision: 0.9832 - recall: 0.9857 - val_f1_score: 0.3820 - val_loss: 1.5277 - val_precision: 0.5092 - val_recall: 0.5430
Epoch 86/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7154 - loss: 0.0441 - precision: 0.9806 - recall: 0.9868 - val_f1_score: 0.3876 - val_loss: 1.4569 - val_precision: 0.5508 - val_recall: 0.5332
Epoch 87/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7152 - loss: 0.0449 - precision: 0.9814 - recall: 0.9844 - val_f1_score: 0.3860 - val_loss: 1.5060 - val_precision: 0.4883 - val_recall: 0.5651
Epoch 88/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.7129 - loss: 0.0488 - precision: 0.9799 - recall: 0.9841 - val_f1_score: 0.3853 - val_loss: 1.4687 - val_precision: 0.5368 - val_recall: 0.5012
Epoch 89/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.7159 - loss: 0.0570 - precision: 0.9809 - recall: 0.9810 - val_f1_score: 0.3904 - val_loss: 1.4753 - val_precision: 0.5500 - val_recall: 0.5405
Epoch 90/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.7141 - loss: 0.0363 - precision: 0.9859 - recall: 0.9883 - val_f1_score: 0.3785 - val_loss: 1.4965 - val_precision: 0.5093 - val_recall: 0.5381
Epoch 91/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7172 - loss: 0.0334 - precision: 0.9863 - recall: 0.9900 - val_f1_score: 0.3843 - val_loss: 1.5632 - val_precision: 0.4879 - val_recall: 0.5946
Epoch 92/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.7187 - loss: 0.0336 - precision: 0.9863 - recall: 0.9908 - val_f1_score: 0.3798 - val_loss: 1.5577 - val_precision: 0.4877 - val_recall: 0.5823
Epoch 93/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.7189 - loss: 0.0472 - precision: 0.9794 - recall: 0.9826 - val_f1_score: 0.3803 - val_loss: 1.5279 - val_precision: 0.5021 - val_recall: 0.5823
Epoch 94/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.7213 - loss: 0.0488 - precision: 0.9820 - recall: 0.9846 - val_f1_score: 0.3924 - val_loss: 1.5116 - val_precision: 0.5499 - val_recall: 0.5283
Epoch 95/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.7208 - loss: 0.0310 - precision: 0.9884 - recall: 0.9897 - val_f1_score: 0.3810 - val_loss: 1.5460 - val_precision: 0.5158 - val_recall: 0.5602
Epoch 96/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.7250 - loss: 0.0320 - precision: 0.9885 - recall: 0.9909 - val_f1_score: 0.3895 - val_loss: 1.6096 - val_precision: 0.5369 - val_recall: 0.5356
Epoch 97/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7233 - loss: 0.0358 - precision: 0.9849 - recall: 0.9886 - val_f1_score: 0.3954 - val_loss: 1.5681 - val_precision: 0.5601 - val_recall: 0.5037
Epoch 98/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7280 - loss: 0.0232 - precision: 0.9934 - recall: 0.9955 - val_f1_score: 0.3994 - val_loss: 1.6423 - val_precision: 0.5422 - val_recall: 0.5209
Epoch 99/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7288 - loss: 0.0311 - precision: 0.9864 - recall: 0.9899 - val_f1_score: 0.3925 - val_loss: 1.6993 - val_precision: 0.5043 - val_recall: 0.5823
Epoch 100/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.7294 - loss: 0.0394 - precision: 0.9820 - recall: 0.9871 - val_f1_score: 0.3987 - val_loss: 1.5756 - val_precision: 0.5497 - val_recall: 0.5160
```

```
In [ ]: print("Time taken in seconds ",end-start)
```

```
Time taken in seconds  252.97596740722656
```

Plotting Training and Validation Loss

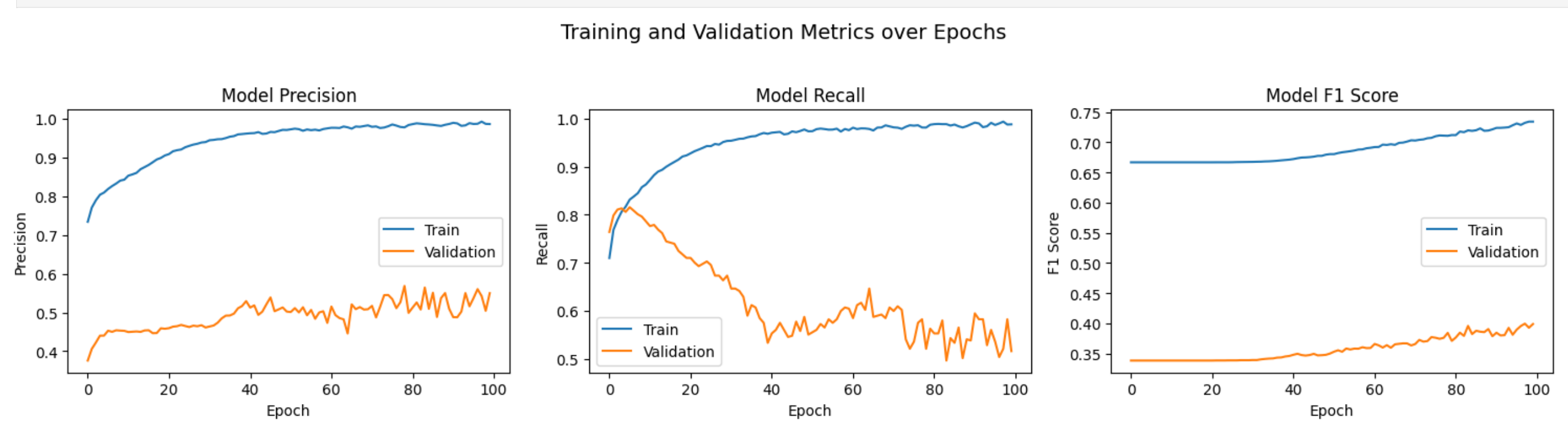
```
In [ ]: #Plotting Train Loss vs Validation Loss
        plt.plot(history_ao_sm.history['loss'])
        plt.plot(history_ao_sm.history['val_loss'])
        plt.title('model loss')
        plt.ylabel('Loss')
        plt.xlabel('Epoch')
        plt.legend(['train', 'validation'], loc='upper left')
        plt.show()
```



- Training model loss decreases rapidly and ends close to 0. This indicates that Adam Optimizer if effectively minimizing the loss function and fitting the training well.
- The Validation loss starts in a similar fashion and then starts increasing rapidly and widens the gap. This indicates overfitting.

Plotting the Precision, Recall and F1 Scores

```
In [ ]: plot_metrics(history_ao_sm, metrics=["precision", "recall", "f1_score"])
```



Precision Metric- Overfitting

- The Training precision increases rapidly and becomes very precise on the training data.
- The Validation precision increases initially and then plateaus to a relatively low value after fluctuations.
- High precision on training data does not translate to validation data.

Recall Metric- Too specific to training dataset

- Training recall increases steadily and plateaus near 1 - finding all relevant instances.
- Validation recall starts high and then decreases rapidly with fluctuations to a lower level.
- Huge gap has developed between training and validation recall.

F1 Metric- Poor performance

- A huge gap exists between the training and validation scores.
- The model is not generalizing well to unseen data.

```
In [ ]: #Predicting the results using best as a threshold for training data
        y_train_pred = model_ao_sm.predict(X_train_sm)
        y_train_pred = (y_train_pred > 0.5)
        y_train_pred
```

```
299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

```
Out[ ]: array([[False],
               [False],
               [False],
               ...,
               [ True],
               [ True],
               [ True]])
```

```
In [ ]: #Predicting the results using best as a threshold for validation data
        y_val_pred = model_ao_sm.predict(X_val)
        y_val_pred = (y_val_pred > 0.5)
        y_val_pred
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
Out[ ]: array([[False],
               [False],
               [False],
               ...,
               [False],
               [False],
               [False]])
```

```
In [ ]: model_name = "NN with Adam Smote"
        train_scores_df.loc[model_name] = recall_score(y_train_sm, y_train_pred)
        val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

```
In [ ]: #print(train_scores_df)
        #print(val_scores_df)
```

Classification Report

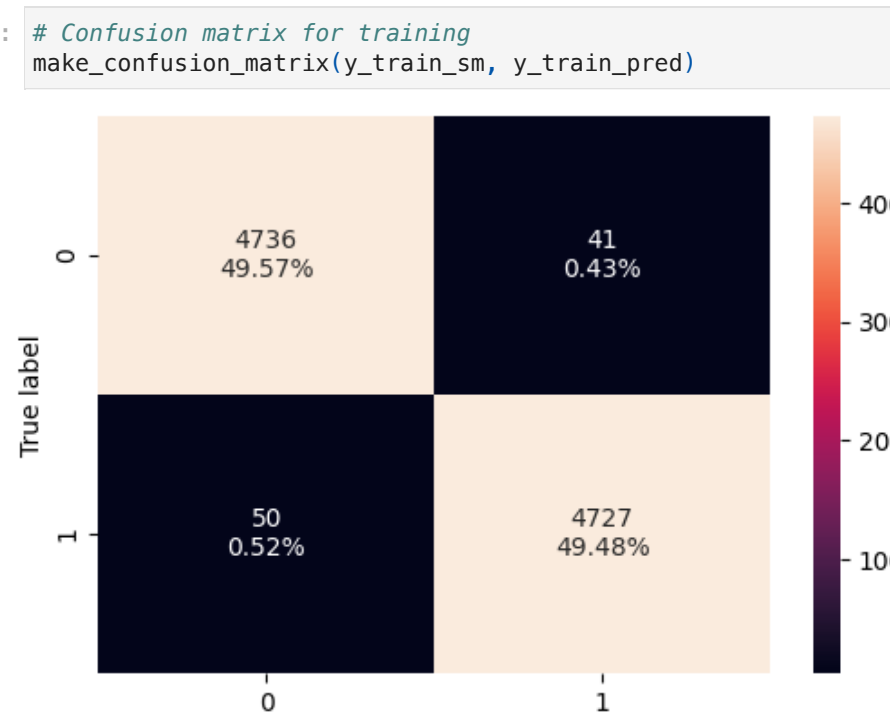
Classification Report

```
In [ ]: #Training classification report
        cr = classification_report(y_train_sm, y_train_pred)
        print(cr)

              precision    recall  f1-score   support

           0       0.99      0.99      0.99      4777
           1       0.99      0.99      0.99      4777

    accuracy                           0.99      9554
   macro avg       0.99      0.99      0.99      9554
weighted avg       0.99      0.99      0.99      9554
```

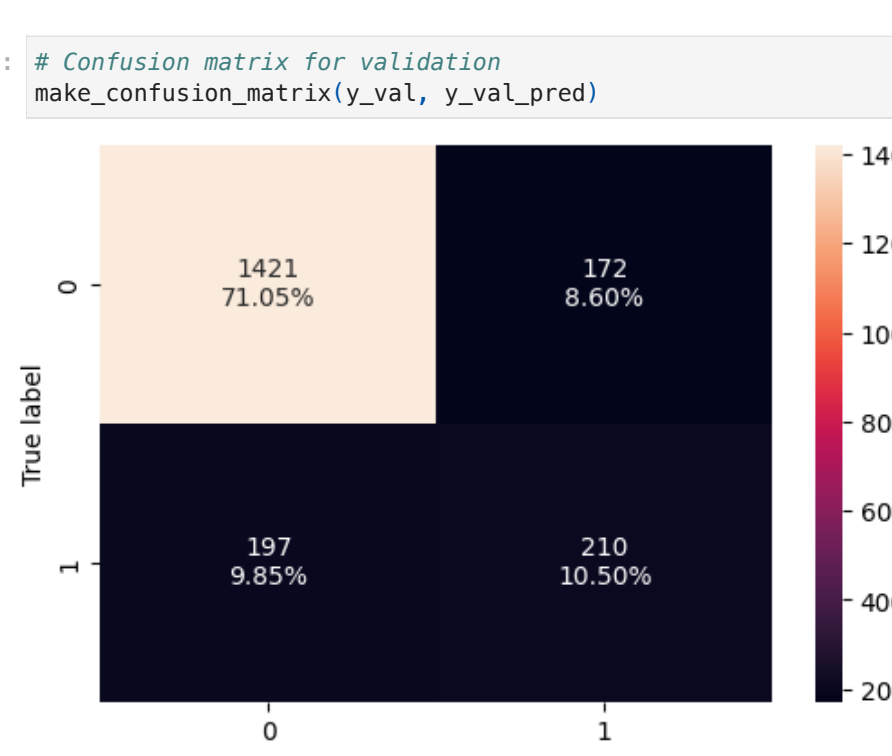
```
In [ ]: #Validation classification report
        cr = classification_report(y_val, y_val_pred)
        print(cr)

              precision    recall  f1-score   support

           0       0.88      0.89      0.89      1593
           1       0.55      0.52      0.53       407

    accuracy                           0.82      2000
   macro avg       0.71      0.70      0.71      2000
weighted avg       0.81      0.82      0.81      2000
```

Confusion Matrix

```
In [ ]: # Confusion matrix for training
        make_confusion_matrix(y_train_sm, y_train_pred)
```



```
In [ ]: # Confusion matrix for validation
        make_confusion_matrix(y_val, y_val_pred)
```



- The performance drop significantly on the validation set, especially for call "1" the minority class.
- The model learns the training data patterns well, but does not translate to unseen data.
- Overfitting is confirmed (as observed before)

Neural Network with Balanced Data (by applying SMOTE), Adam Optimizer, and Dropout

Instantiate NN

```
In [ ]: # Initialization
        backend.clear_session()
        np.random.seed(RANDOM_STATE)
        random.seed(RANDOM_STATE)
        tf.random.set_seed(RANDOM_STATE)
        DROPOUT = 0.5
```

```
In [ ]: # Initialize the neural network with Adam Optimizer
        model_ao_sm_do = Sequential()

        # Add an input layer with 64 neurons and Relu as the activation function
        model_ao_sm_do.add(Dense(units=64, activation='relu', input_shape=(X_train_sm.shape[1],)))

        # Add dropout rate
        model_ao_sm_do.add(Dropout(DROPOUT))

        # Add an hidden layer with 32 neurons and Relu as the activation function
        model_ao_sm_do.add(Dense(units=32, activation='relu'))

        # Add dropout rate
        model_ao_sm_do.add(Dropout(DROPOUT))

        # Add an hidden layer with 16 neurons and Relu as the activation function
        model_ao_sm_do.add(Dense(units=16, activation='relu'))

        # Add an output layer with 1 neuron and Sigmoid as the activation function
        # as this is a binary classification problem
        model_ao_sm_do.add(Dense(units=1, activation='sigmoid'))
```

```
In [ ]: # Use Adam as the optimizer
        optimizer = tf.keras.optimizers.Adam(0.001)

        #Choose Precision, Recall and F1 score as the metric
        metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]
```

```
In [ ]: # Compile the model with loss function of binary cross entropy and F1score as the metric
        model_ao_sm_do.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

```
In [ ]: model_ao_sm_do.summary()
```

```
Model: "sequential"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━┓
┃ Layer (type)                 ┃ Output Shape         ┃     Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━┩
│ dense (Dense)                │ (None, 64)           │         768 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dropout (Dropout)            │ (None, 64)           │           0 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dense_1 (Dense)              │ (None, 32)           │       2,080 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dropout_1 (Dropout)          │ (None, 32)           │           0 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dense_2 (Dense)              │ (None, 16)           │         528 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dense_3 (Dense)              │ (None, 1)            │          17 │
└──────────────────────────────┴──────────────────────┴─────────────┘
 Total params: 3,393 (13.25 KB)
 Trainable params: 3,393 (13.25 KB)
 Non-trainable params: 0 (0.00 B)
```

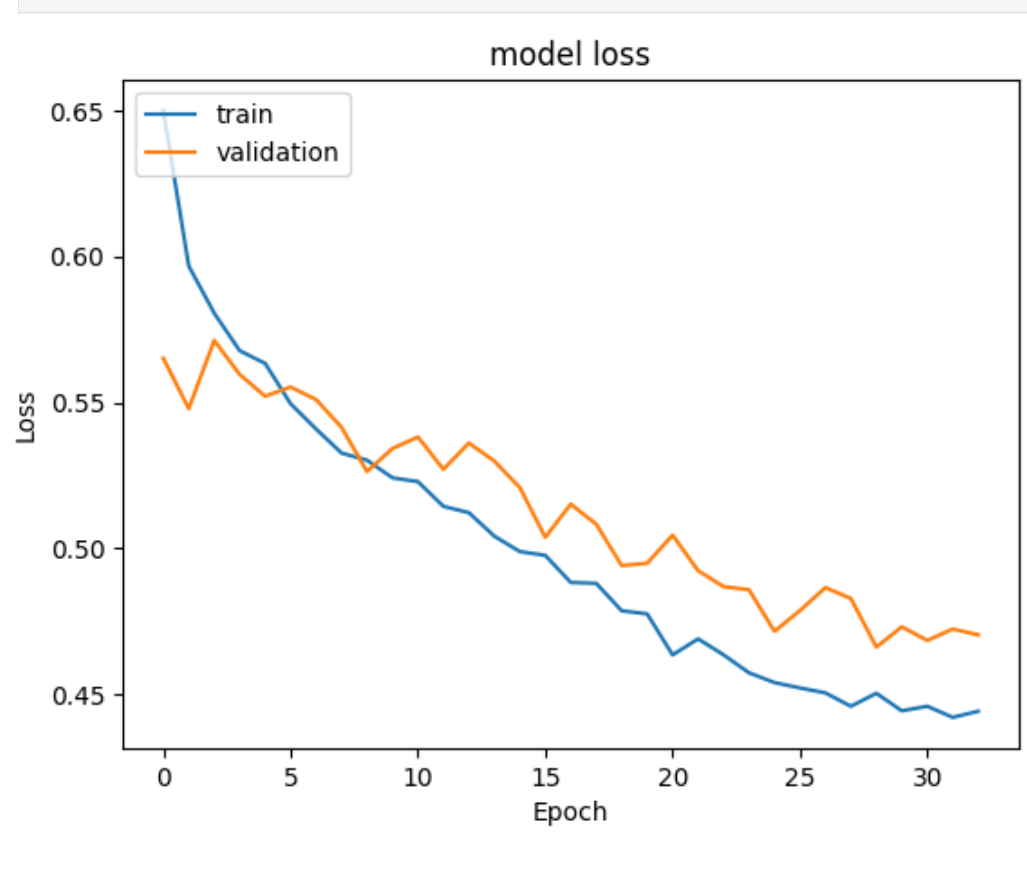
```
In [ ]: start = time.time()

        history_ao_sm_do = model_ao_sm_do.fit(
            X_train_sm, y_train_sm,
            batch_size=32,
            epochs=33,
            verbose=1,
            validation_data = (X_val,y_val)
        )

        end = time.time()
```

```
Epoch 1/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.6624 - loss: 0.6798 - precision: 0.5494 - recall: 0.6388 - val_f1_score: 0.3382 - val_loss: 0.5651 - val_precision: 0.3639 - val_recall: 0.6929
Epoch 2/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.6061 - precision: 0.6747 - recall: 0.6994 - val_f1_score: 0.3382 - val_loss: 0.5479 - val_precision: 0.3724 - val_recall: 0.7027
Epoch 3/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5904 - precision: 0.6829 - recall: 0.7228 - val_f1_score: 0.3382 - val_loss: 0.5712 - val_precision: 0.3690 - val_recall: 0.7199
Epoch 4/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5800 - precision: 0.6938 - recall: 0.7341 - val_f1_score: 0.3382 - val_loss: 0.5596 - val_precision: 0.3766 - val_recall: 0.7052
Epoch 5/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.5742 - precision: 0.7002 - recall: 0.7340 - val_f1_score: 0.3382 - val_loss: 0.5521 - val_precision: 0.3762 - val_recall: 0.6904
Epoch 6/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5574 - precision: 0.7139 - recall: 0.7351 - val_f1_score: 0.3382 - val_loss: 0.5553 - val_precision: 0.3820 - val_recall: 0.7076
Epoch 7/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5519 - precision: 0.7230 - recall: 0.7426 - val_f1_score: 0.3382 - val_loss: 0.5510 - val_precision: 0.3915 - val_recall: 0.7052
Epoch 8/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5436 - precision: 0.7234 - recall: 0.7385 - val_f1_score: 0.3382 - val_loss: 0.5415 - val_precision: 0.4110 - val_recall: 0.6978
Epoch 9/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5368 - precision: 0.7309 - recall: 0.7358 - val_f1_score: 0.3382 - val_loss: 0.5263 - val_precision: 0.4158 - val_recall: 0.6978
Epoch 10/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.5322 - precision: 0.7348 - recall: 0.7341 - val_f1_score: 0.3382 - val_loss: 0.5342 - val_precision: 0.4168 - val_recall: 0.6953
Epoch 11/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5269 - precision: 0.7444 - recall: 0.7304 - val_f1_score: 0.3382 - val_loss: 0.5382 - val_precision: 0.4204 - val_recall: 0.7076
Epoch 12/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5215 - precision: 0.7478 - recall: 0.7388 - val_f1_score: 0.3382 - val_loss: 0.5271 - val_precision: 0.4174 - val_recall: 0.7076
Epoch 13/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.5214 - precision: 0.7407 - recall: 0.7495 - val_f1_score: 0.3382 - val_loss: 0.5361 - val_precision: 0.4112 - val_recall: 0.7224
Epoch 14/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.5160 - precision: 0.7447 - recall: 0.7604 - val_f1_score: 0.3382 - val_loss: 0.5299 - val_precision: 0.4099 - val_recall: 0.7322
Epoch 15/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5089 - precision: 0.7536 - recall: 0.7567 - val_f1_score: 0.3382 - val_loss: 0.5208 - val_precision: 0.4207 - val_recall: 0.7297
Epoch 16/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5047 - precision: 0.7550 - recall: 0.7603 - val_f1_score: 0.3382 - val_loss: 0.5038 - val_precision: 0.4236 - val_recall: 0.7224
Epoch 17/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4962 - precision: 0.7634 - recall: 0.7667 - val_f1_score: 0.3382 - val_loss: 0.5152 - val_precision: 0.4288 - val_recall: 0.7396
Epoch 18/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4982 - precision: 0.7577 - recall: 0.7628 - val_f1_score: 0.3382 - val_loss: 0.5083 - val_precision: 0.4428 - val_recall: 0.7518
Epoch 19/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4824 - precision: 0.7692 - recall: 0.7646 - val_f1_score: 0.3382 - val_loss: 0.4941 - val_precision: 0.4441 - val_recall: 0.7518
Epoch 20/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4878 - precision: 0.7614 - recall: 0.7567 - val_f1_score: 0.3382 - val_loss: 0.4949 - val_precision: 0.4392 - val_recall: 0.7371
Epoch 21/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4731 - precision: 0.7778 - recall: 0.7727 - val_f1_score: 0.3382 - val_loss: 0.5046 - val_precision: 0.4358 - val_recall: 0.7666
Epoch 22/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4792 - precision: 0.7731 - recall: 0.7737 - val_f1_score: 0.3382 - val_loss: 0.4924 - val_precision: 0.4440 - val_recall: 0.7592
Epoch 23/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4674 - precision: 0.7841 - recall: 0.7696 - val_f1_score: 0.3382 - val_loss: 0.4869 - val_precision: 0.4391 - val_recall: 0.7617
Epoch 24/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4640 - precision: 0.7680 - recall: 0.7794 - val_f1_score: 0.3382 - val_loss: 0.4858 - val_precision: 0.4499 - val_recall: 0.7396
Epoch 25/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4618 - precision: 0.7804 - recall: 0.7730 - val_f1_score: 0.3382 - val_loss: 0.4716 - val_precision: 0.4748 - val_recall: 0.7420
Epoch 26/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4618 - precision: 0.7852 - recall: 0.7790 - val_f1_score: 0.3382 - val_loss: 0.4787 - val_precision: 0.4717 - val_recall: 0.7371
Epoch 27/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.4570 - precision: 0.7859 - recall: 0.7710 - val_f1_score: 0.3382 - val_loss: 0.4866 - val_precision: 0.4606 - val_recall: 0.7617
Epoch 28/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4549 - precision: 0.7898 - recall: 0.7919 - val_f1_score: 0.3382 - val_loss: 0.4829 - val_precision: 0.4726 - val_recall: 0.7641
Epoch 29/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4626 - precision: 0.7783 - recall: 0.7842 - val_f1_score: 0.3382 - val_loss: 0.4662 - val_precision: 0.4785 - val_recall: 0.7396
Epoch 30/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4500 - precision: 0.7885 - recall: 0.7751 - val_f1_score: 0.3382 - val_loss: 0.4731 - val_precision: 0.4747 - val_recall: 0.7592
Epoch 31/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4561 - precision: 0.7875 - recall: 0.7807 - val_f1_score: 0.3382 - val_loss: 0.4685 - val_precision: 0.4858 - val_recall: 0.7592
Epoch 32/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4518 - precision: 0.7960 - recall: 0.7852 - val_f1_score: 0.3382 - val_loss: 0.4724 - val_precision: 0.4824 - val_recall: 0.7740
Epoch 33/33
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.4497 - precision: 0.7953 - recall: 0.7839 - val_f1_score: 0.3382 - val_loss: 0.4704 - val_precision: 0.4845 - val_recall: 0.7666
```

```
In [ ]: print("Time taken in seconds ",end-start)

Time taken in seconds  76.79849982261658
```

Plotting Training and Validation Loss

```
In [ ]: #Plotting Train Loss vs Validation Loss
        plt.plot(history_ao_sm_do.history['loss'])
        plt.plot(history_ao_sm_do.history['val_loss'])
        plt.title('model loss')
        plt.ylabel('Loss')
        plt.xlabel('Epoch')
        plt.legend(['train', 'validation'], loc='upper left')
        plt.show()
```

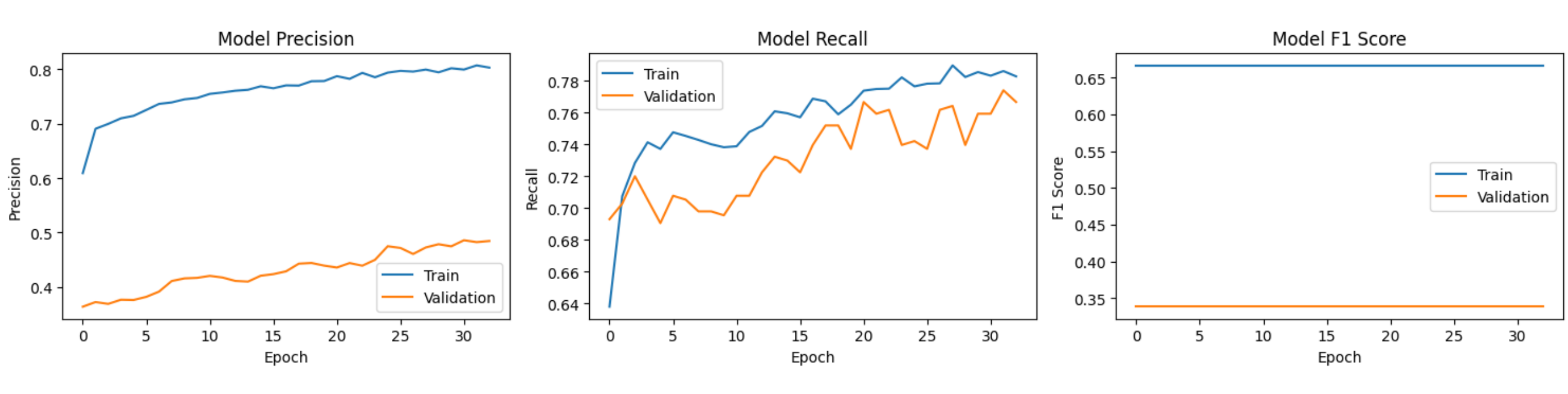


- Model is learning well on training data.
- It is not generalizing well on unseen data despite dropout regularization.
- SMOTE maybe introducing synthetic noise that the model is overfitting to.

Plotting the Precision, Recall and F1 scores

```
In [ ]: plot_metrics(history_ao_sm_do, metrics=["precision", "recall", "f1_score"])
```

Training and Validation Metrics over Epochs



Precision Metric -

- Training precision increases steadily
- Validation precision rises and plateaus at around 0.5

Recall Metric -

- Training recall is high and keep improving
- Validation recall improves initially and then flattens and dips
- Another sign of overfitting

F1 Metric -

- F1 scores confirm overfitting of data with a consistent difference between training and validation datasets.
- Validation score is low and flat, the model has poor generalization to the minority class despite using SMOTE and dropouts with Adam Optimizer.

```
In [ ]: #Predicting the results using best as a threshold for training data
        y_train_pred = model_ao_sm_do.predict(X_train_sm)
        y_train_pred = (y_train_pred > 0.5)
        y_train_pred

299/299 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

Out[ ]: array([[ True],
               [ True],
               [ True],
               ...,
               [ True],
               [False],
               [ True]])
```

```
In [ ]: #Predicting the results using best as a threshold for validation data
        y_val_pred = model_ao_sm_do.predict(X_val)
        y_val_pred = (y_val_pred > 0.5)
        y_val_pred

63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Out[ ]: array([[False],
               [False],
               [ True],
               ...,
               [False],
               [False],
               [ True]])
```

```
In [ ]: model_name = "NN with Adam Smote & Dropout"
        train_scores_df.loc[model_name] = recall_score(y_train_sm, y_train_pred)
        val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

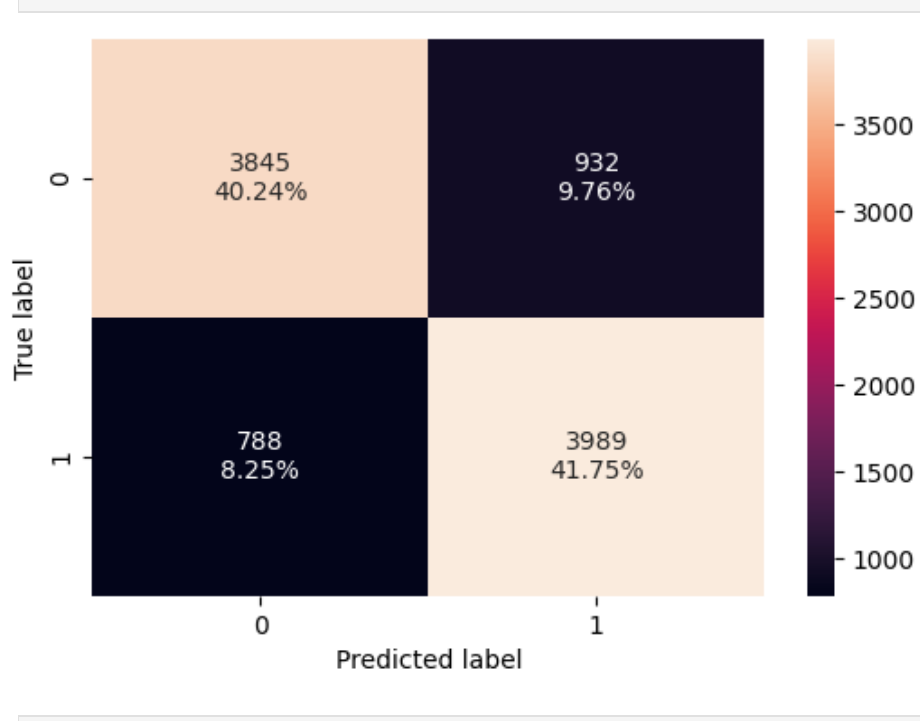
Classification Report

```
In [ ]: #Training classification report
        cr = classification_report(y_train_sm, y_train_pred)
        print(cr)

              precision    recall  f1-score   support

           0       0.83      0.80      0.82      4777
           1       0.81      0.84      0.82      4777

    accuracy                           0.82      9554
   macro avg       0.82      0.82      0.82      9554
weighted avg       0.82      0.82      0.82      9554
```

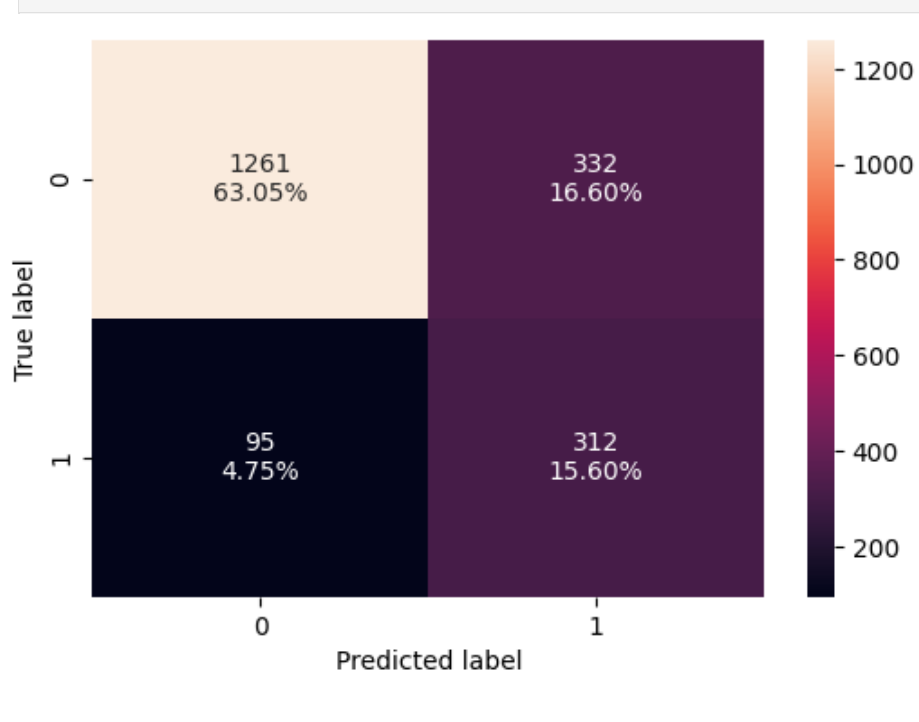
```
In [ ]: #Validation classification report
        cr = classification_report(y_val, y_val_pred)
        print(cr)

              precision    recall  f1-score   support

           0       0.93      0.79      0.86      1593
           1       0.48      0.77      0.59       407

    accuracy                           0.79      2000
   macro avg       0.71      0.78      0.72      2000
weighted avg       0.84      0.79      0.80      2000
```

Confusion Matrix

```
In [ ]: # Confusion matrix for training
        make_confusion_matrix(y_train_sm, y_train_pred)
```



```
In [ ]: # Confusion matrix for validation
        make_confusion_matrix(y_val, y_val_pred)
```



Neural Network with Balanced Data (by applying SMOTE), Adam Optimizer, Dropout and Early Stopping

Instantiate NN

```
In [ ]: # Initialization
        backend.clear_session()
        np.random.seed(RANDOM_STATE)
        random.seed(RANDOM_STATE)
        tf.random.set_seed(RANDOM_STATE)
        DROPOUT = 0.5
```

```
In [ ]: # Initialize the neural network with Adam Optimizer
        model_ao_sm_do_es = Sequential()

        # Add an input layer with 64 neurons and Relu as the activation function
        model_ao_sm_do_es.add(Dense(units=64, activation='relu', input_shape=(X_train_sm.shape[1],)))

        # Add dropout rate
        model_ao_sm_do_es.add(Dropout(DROPOUT))

        # Add an hidden layer with 32 neurons and Relu as the activation function
        model_ao_sm_do_es.add(Dense(units=32, activation='relu'))

        # Add dropout rate
        model_ao_sm_do_es.add(Dropout(DROPOUT))

        # Add an hidden layer with 16 neurons and Relu as the activation function
        model_ao_sm_do_es.add(Dense(units=16, activation='relu'))

        # Add an output layer with 1 neuron and Sigmoid as the activation function
        # as this is a binary classification problem
        model_ao_sm_do_es.add(Dense(units=1, activation='sigmoid'))
```

```
In [ ]: # Use Adam as the optimizer
        optimizer = tf.keras.optimizers.Adam(0.001)

        #Choose Precision, Recall and F1 score as the metric
        metric = [Precision(), Recall(), keras.metrics.F1Score(average = 'macro')]
```

```
In [ ]: # Compile the model with loss function of binary cross entropy and F1score as the metric
        model_ao_sm_do_es.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=metric)
```

```
In [ ]: # Early Stopping callback
        early_stopping = EarlyStopping(
            monitor='val_loss',
            verbose=1,
            patience=10,
            mode='min',
            restore_best_weights=True
        )
```

```
In [ ]: model_ao_sm_do.summary()
```

```
Model: "sequential"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━┓
┃ Layer (type)                 ┃ Output Shape         ┃     Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━┩
│ dense (Dense)                │ (None, 64)           │         768 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dropout (Dropout)            │ (None, 64)           │           0 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dense_1 (Dense)              │ (None, 32)           │       2,080 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dropout_1 (Dropout)          │ (None, 32)           │           0 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dense_2 (Dense)              │ (None, 16)           │         528 │
├──────────────────────────────┼──────────────────────┼─────────────┤
│ dense_3 (Dense)              │ (None, 1)            │          17 │
└──────────────────────────────┴──────────────────────┴─────────────┘
 Total params: 10,181 (39.77 KB)
 Trainable params: 3,393 (13.25 KB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 6,788 (26.52 KB)
```

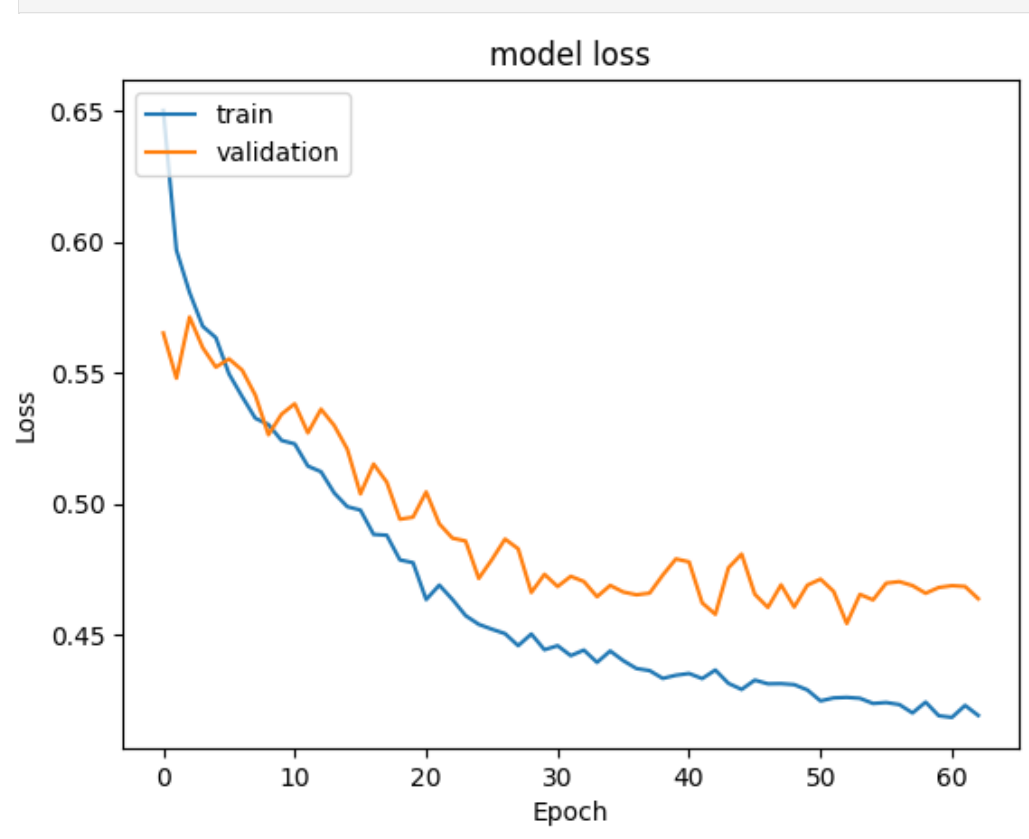
```
In [ ]: start = time.time()

        history_ao_sm_do_es = model_ao_sm_do_es.fit(
            X_train_sm, y_train_sm,
            batch_size=32,
            epochs=100,
            verbose=1,
            validation_data = (X_val,y_val),
            callbacks=[early_stopping]
        )

        end = time.time()
```

```
Epoch 1/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - f1_score: 0.6624 - loss: 0.6798 - precision: 0.5494 - recall: 0.6388 - val_f1_score: 0.3382 - val_loss: 0.5651 - val_precision: 0.3639 - val_recall: 0.6929
Epoch 2/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.6061 - precision: 0.6747 - recall: 0.6994 - val_f1_score: 0.3382 - val_loss: 0.5479 - val_precision: 0.3724 - val_recall: 0.7027
Epoch 3/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5904 - precision: 0.6829 - recall: 0.7228 - val_f1_score: 0.3382 - val_loss: 0.5712 - val_precision: 0.3690 - val_recall: 0.7199
Epoch 4/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.6624 - loss: 0.5800 - precision: 0.6938 - recall: 0.7341 - val_f1_score: 0.3382 - val_loss: 0.5596 - val_precision: 0.3766 - val_recall: 0.7052
Epoch 5/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5742 - precision: 0.7002 - recall: 0.7340 - val_f1_score: 0.3382 - val_loss: 0.5521 - val_precision: 0.3762 - val_recall: 0.6904
Epoch 6/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5574 - precision: 0.7139 - recall: 0.7351 - val_f1_score: 0.3382 - val_loss: 0.5553 - val_precision: 0.3820 - val_recall: 0.7076
Epoch 7/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5519 - precision: 0.7230 - recall: 0.7426 - val_f1_score: 0.3382 - val_loss: 0.5510 - val_precision: 0.3915 - val_recall: 0.7052
Epoch 8/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5436 - precision: 0.7234 - recall: 0.7385 - val_f1_score: 0.3382 - val_loss: 0.5415 - val_precision: 0.4110 - val_recall: 0.6978
Epoch 9/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - f1_score: 0.6624 - loss: 0.5368 - precision: 0.7309 - recall: 0.7358 - val_f1_score: 0.3382 - val_loss: 0.5263 - val_precision: 0.4158 - val_recall: 0.6978
Epoch 10/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - f1_score: 0.6624 - loss: 0.5322 - precision: 0.7348 - recall: 0.7341 - val_f1_score: 0.3382 - val_loss: 0.5342 - val_precision: 0.4168 - val_recall: 0.6953
Epoch 11/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.5269 - precision: 0.7444 - recall: 0.7304 - val_f1_score: 0.3382 - val_loss: 0.5382 - val_precision: 0.4204 - val_recall: 0.7076
Epoch 12/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5215 - precision: 0.7478 - recall: 0.7388 - val_f1_score: 0.3382 - val_loss: 0.5271 - val_precision: 0.4174 - val_recall: 0.7076
Epoch 13/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.5214 - precision: 0.7407 - recall: 0.7495 - val_f1_score: 0.3382 - val_loss: 0.5361 - val_precision: 0.4112 - val_recall: 0.7224
Epoch 14/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.6624 - loss: 0.5160 - precision: 0.7447 - recall: 0.7604 - val_f1_score: 0.3382 - val_loss: 0.5299 - val_precision: 0.4099 - val_recall: 0.7322
Epoch 15/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.5089 - precision: 0.7536 - recall: 0.7567 - val_f1_score: 0.3382 - val_loss: 0.5208 - val_precision: 0.4207 - val_recall: 0.7297
Epoch 16/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.5047 - precision: 0.7550 - recall: 0.7603 - val_f1_score: 0.3382 - val_loss: 0.5038 - val_precision: 0.4236 - val_recall: 0.7224
Epoch 17/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4962 - precision: 0.7634 - recall: 0.7667 - val_f1_score: 0.3382 - val_loss: 0.5152 - val_precision: 0.4288 - val_recall: 0.7396
Epoch 18/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4982 - precision: 0.7577 - recall: 0.7628 - val_f1_score: 0.3382 - val_loss: 0.5083 - val_precision: 0.4428 - val_recall: 0.7518
Epoch 19/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - f1_score: 0.6624 - loss: 0.4824 - precision: 0.7692 - recall: 0.7646 - val_f1_score: 0.3382 - val_loss: 0.4941 - val_precision: 0.4441 - val_recall: 0.7518
Epoch 20/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - f1_score: 0.6624 - loss: 0.4878 - precision: 0.7614 - recall: 0.7567 - val_f1_score: 0.3382 - val_loss: 0.4949 - val_precision: 0.4392 - val_recall: 0.7371
Epoch 21/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4731 - precision: 0.7778 - recall: 0.7727 - val_f1_score: 0.3382 - val_loss: 0.5046 - val_precision: 0.4358 - val_recall: 0.7666
Epoch 22/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4792 - precision: 0.7731 - recall: 0.7737 - val_f1_score: 0.3382 - val_loss: 0.4924 - val_precision: 0.4440 - val_recall: 0.7592
Epoch 23/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.4674 - precision: 0.7841 - recall: 0.7696 - val_f1_score: 0.3382 - val_loss: 0.4869 - val_precision: 0.4391 - val_recall: 0.7617
Epoch 24/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4640 - precision: 0.7680 - recall: 0.7794 - val_f1_score: 0.3382 - val_loss: 0.4858 - val_precision: 0.4499 - val_recall: 0.7396
Epoch 25/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4618 - precision: 0.7804 - recall: 0.7730 - val_f1_score: 0.3382 - val_loss: 0.4716 - val_precision: 0.4748 - val_recall: 0.7420
Epoch 26/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4618 - precision: 0.7852 - recall: 0.7790 - val_f1_score: 0.3382 - val_loss: 0.4787 - val_precision: 0.4717 - val_recall: 0.7371
Epoch 27/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4570 - precision: 0.7859 - recall: 0.7710 - val_f1_score: 0.3382 - val_loss: 0.4866 - val_precision: 0.4606 - val_recall: 0.7617
Epoch 28/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4549 - precision: 0.7898 - recall: 0.7919 - val_f1_score: 0.3382 - val_loss: 0.4829 - val_precision: 0.4726 - val_recall: 0.7641
Epoch 29/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - f1_score: 0.6624 - loss: 0.4626 - precision: 0.7783 - recall: 0.7842 - val_f1_score: 0.3382 - val_loss: 0.4662 - val_precision: 0.4785 - val_recall: 0.7396
Epoch 30/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4500 - precision: 0.7885 - recall: 0.7751 - val_f1_score: 0.3382 - val_loss: 0.4731 - val_precision: 0.4747 - val_recall: 0.7592
Epoch 31/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.4561 - precision: 0.7875 - recall: 0.7807 - val_f1_score: 0.3382 - val_loss: 0.4685 - val_precision: 0.4858 - val_recall: 0.7592
Epoch 32/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4518 - precision: 0.7960 - recall: 0.7852 - val_f1_score: 0.3382 - val_loss: 0.4724 - val_precision: 0.4824 - val_recall: 0.7740
Epoch 33/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.4497 - precision: 0.7953 - recall: 0.7839 - val_f1_score: 0.3382 - val_loss: 0.4704 - val_precision: 0.4845 - val_recall: 0.7666
Epoch 34/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.4470 - precision: 0.7976 - recall: 0.7902 - val_f1_score: 0.3382 - val_loss: 0.4646 - val_precision: 0.4918 - val_recall: 0.7396
Epoch 35/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - f1_score: 0.6624 - loss: 0.4519 - precision: 0.7937 - recall: 0.7817 - val_f1_score: 0.3382 - val_loss: 0.4689 - val_precision: 0.4773 - val_recall: 0.7494
Epoch 36/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4485 - precision: 0.7902 - recall: 0.7858 - val_f1_score: 0.3382 - val_loss: 0.4664 - val_precision: 0.4810 - val_recall: 0.7445
Epoch 37/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4479 - precision: 0.7917 - recall: 0.7871 - val_f1_score: 0.3382 - val_loss: 0.4653 - val_precision: 0.4856 - val_recall: 0.7445
Epoch 38/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4395 - precision: 0.7999 - recall: 0.7878 - val_f1_score: 0.3382 - val_loss: 0.4660 - val_precision: 0.4815 - val_recall: 0.7371
Epoch 39/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4401 - precision: 0.7875 - recall: 0.7874 - val_f1_score: 0.3382 - val_loss: 0.4729 - val_precision: 0.4815 - val_recall: 0.7346
Epoch 40/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.4422 - precision: 0.7936 - recall: 0.7793 - val_f1_score: 0.3382 - val_loss: 0.4790 - val_precision: 0.4767 - val_recall: 0.7543
Epoch 41/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6624 - loss: 0.4457 - precision: 0.7977 - recall: 0.7815 - val_f1_score: 0.3382 - val_loss: 0.4779 - val_precision: 0.4879 - val_recall: 0.7420
Epoch 42/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4421 - precision: 0.7987 - recall: 0.7828 - val_f1_score: 0.3382 - val_loss: 0.4623 - val_precision: 0.4958 - val_recall: 0.7297
Epoch 43/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4388 - precision: 0.7978 - recall: 0.7845 - val_f1_score: 0.3382 - val_loss: 0.4579 - val_precision: 0.4983 - val_recall: 0.7297
Epoch 44/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4402 - precision: 0.7959 - recall: 0.7911 - val_f1_score: 0.3382 - val_loss: 0.4756 - val_precision: 0.4888 - val_recall: 0.7494
Epoch 45/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.4388 - precision: 0.8053 - recall: 0.7790 - val_f1_score: 0.3382 - val_loss: 0.4808 - val_precision: 0.4810 - val_recall: 0.7469
Epoch 46/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6624 - loss: 0.4454 - precision: 0.7924 - recall: 0.7811 - val_f1_score: 0.3382 - val_loss: 0.4656 - val_precision: 0.4856 - val_recall: 0.7469
Epoch 47/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4405 - precision: 0.7964 - recall: 0.7896 - val_f1_score: 0.3382 - val_loss: 0.4605 - val_precision: 0.4824 - val_recall: 0.7396
Epoch 48/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4422 - precision: 0.8001 - recall: 0.7931 - val_f1_score: 0.3382 - val_loss: 0.4691 - val_precision: 0.4837 - val_recall: 0.7273
Epoch 49/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4418 - precision: 0.8063 - recall: 0.7928 - val_f1_score: 0.3382 - val_loss: 0.4607 - val_precision: 0.4974 - val_recall: 0.7076
Epoch 50/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.4391 - precision: 0.7976 - recall: 0.7878 - val_f1_score: 0.3382 - val_loss: 0.4690 - val_precision: 0.4839 - val_recall: 0.7396
Epoch 51/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4281 - precision: 0.8039 - recall: 0.7945 - val_f1_score: 0.3382 - val_loss: 0.4713 - val_precision: 0.4815 - val_recall: 0.7371
Epoch 52/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4319 - precision: 0.8047 - recall: 0.7876 - val_f1_score: 0.3382 - val_loss: 0.4666 - val_precision: 0.4871 - val_recall: 0.7396
Epoch 53/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4324 - precision: 0.7972 - recall: 0.7925 - val_f1_score: 0.3382 - val_loss: 0.4544 - val_precision: 0.4947 - val_recall: 0.6929
Epoch 54/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4301 - precision: 0.8067 - recall: 0.7966 - val_f1_score: 0.3382 - val_loss: 0.4655 - val_precision: 0.4861 - val_recall: 0.7297
Epoch 55/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4331 - precision: 0.8005 - recall: 0.7894 - val_f1_score: 0.3382 - val_loss: 0.4634 - val_precision: 0.4950 - val_recall: 0.7248
Epoch 56/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - f1_score: 0.6624 - loss: 0.4309 - precision: 0.8042 - recall: 0.7895 - val_f1_score: 0.3382 - val_loss: 0.4698 - val_precision: 0.4826 - val_recall: 0.7494
Epoch 57/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - f1_score: 0.6624 - loss: 0.4353 - precision: 0.8013 - recall: 0.7984 - val_f1_score: 0.3382 - val_loss: 0.4704 - val_precision: 0.4901 - val_recall: 0.7273
Epoch 58/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4272 - precision: 0.8052 - recall: 0.7876 - val_f1_score: 0.3382 - val_loss: 0.4688 - val_precision: 0.4722 - val_recall: 0.7518
Epoch 59/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - f1_score: 0.6624 - loss: 0.4323 - precision: 0.7933 - recall: 0.7874 - val_f1_score: 0.3382 - val_loss: 0.4659 - val_precision: 0.4902 - val_recall: 0.7346
Epoch 60/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - f1_score: 0.6624 - loss: 0.4279 - precision: 0.8061 - recall: 0.7877 - val_f1_score: 0.3382 - val_loss: 0.4681 - val_precision: 0.4902 - val_recall: 0.7396
Epoch 61/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - f1_score: 0.6624 - loss: 0.4273 - precision: 0.7990 - recall: 0.7984 - val_f1_score: 0.3382 - val_loss: 0.4688 - val_precision: 0.4834 - val_recall: 0.7494
Epoch 62/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - f1_score: 0.6624 - loss: 0.4326 - precision: 0.7988 - recall: 0.7962 - val_f1_score: 0.3382 - val_loss: 0.4685 - val_precision: 0.4715 - val_recall: 0.7322
Epoch 63/100
299/299 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - f1_score: 0.6624 - loss: 0.4280 - precision: 0.8001 - recall: 0.7954 - val_f1_score: 0.3382 - val_loss: 0.4639 - val_precision: 0.4858 - val_recall: 0.7125
Epoch 63: early stopping
Restoring model weights from the end of the best epoch: 53.
```

```
In [ ]: print("Time taken in seconds ",end-start)

Time taken in seconds  163.1056649684906
```

Plotting Training and Validation Loss

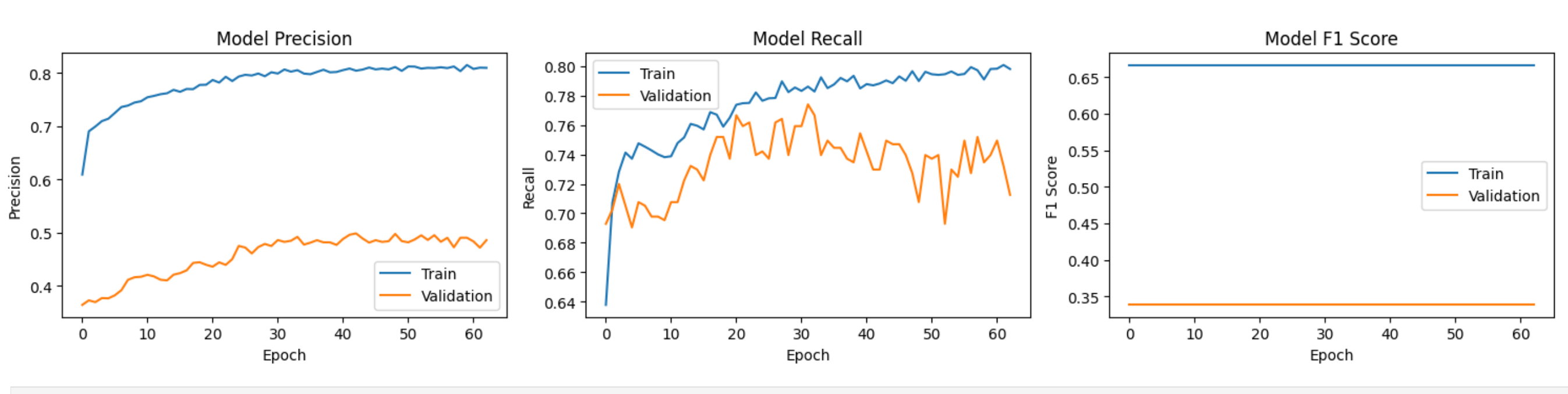
```
In [ ]: #Plotting Train Loss vs Validation Loss
        plt.plot(history_ao_sm_do_es.history['loss'])
        plt.plot(history_ao_sm_do_es.history['val_loss'])
        plt.title('model loss')
        plt.ylabel('Loss')
        plt.xlabel('Epoch')
        plt.legend(['train', 'validation'], loc='upper left')
        plt.show()
```



Plotting the Precision, Recall and F1 Scores

```
In [ ]: plot_metrics(history_ao_sm_do_es, metrics=["precision", "recall", "f1_score"])
```

Training and Validation Metrics over Epochs



```
In [ ]: #Predicting the results using best as a threshold for training data
        y_train_pred = model_ao_sm_do_es.predict(X_train_sm)
        y_train_pred = (y_train_pred > 0.5)
        y_train_pred

299/299 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

Out[ ]: array([[ True],
               [ True],
               [ True],
               ...,
               [ True],
               [False],
               [ True]])
```

```
In [ ]: #Predicting the results using best as a threshold for validation data
        y_val_pred = model_ao_sm_do_es.predict(X_val)
        y_val_pred = (y_val_pred > 0.5)
        y_val_pred

63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Out[ ]: array([[False],
               [False],
               [ True],
               ...,
               [False],
               [False],
               [ True]])
```

```
In [ ]: model_name = "NN with Adam Smote, Drop, Early Stop"

        train_scores_df.loc[model_name] = recall_score(y_train_sm, y_train_pred)
        val_scores_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

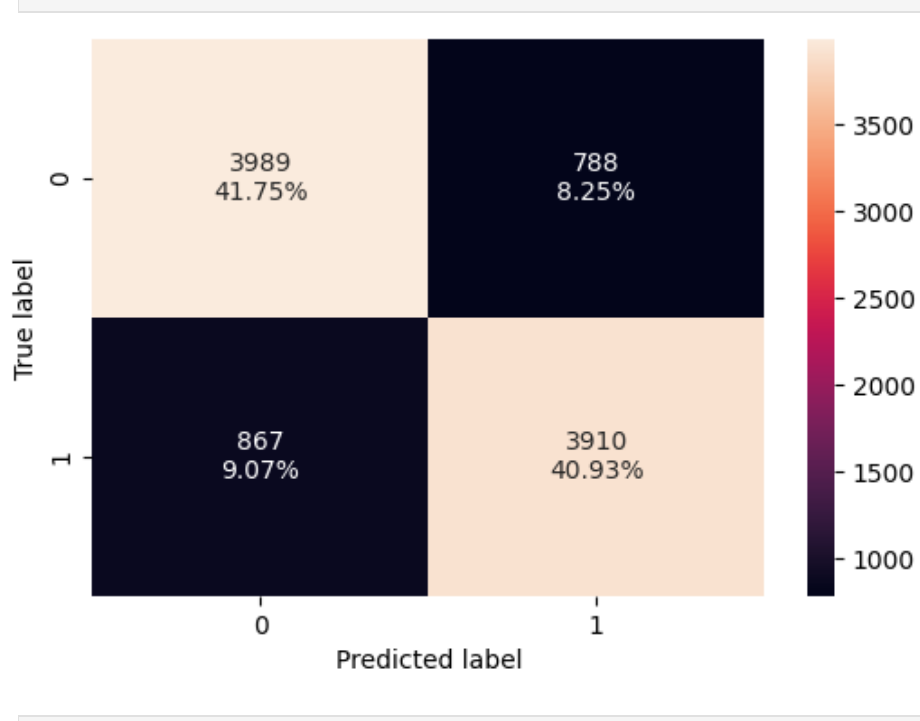
Classification Report

```
In [ ]: #Training classification report
        cr = classification_report(y_train_sm, y_train_pred)
        print(cr)

              precision    recall  f1-score   support

           0       0.82      0.84      0.83      4777
           1       0.83      0.82      0.83      4777

    accuracy                           0.83      9554
   macro avg       0.83      0.83      0.83      9554
weighted avg       0.83      0.83      0.83      9554
```

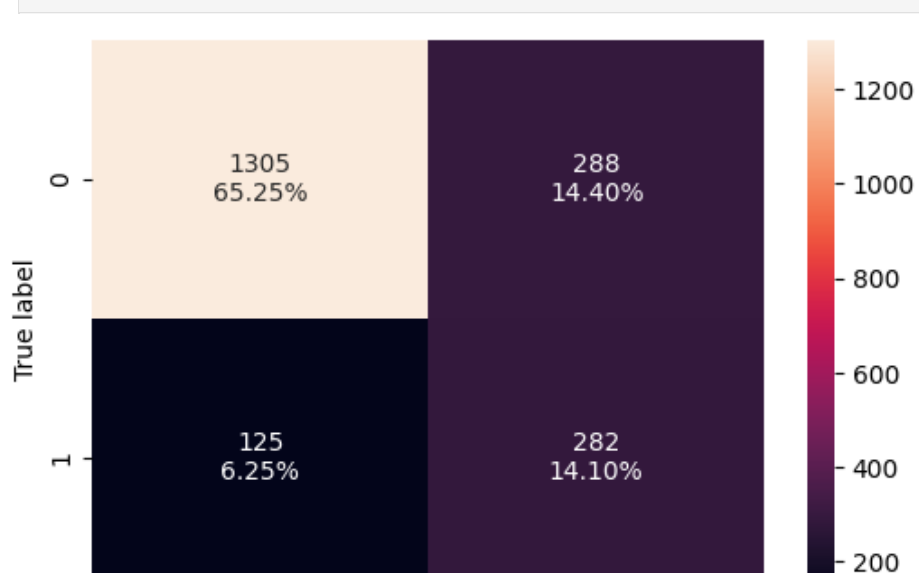
```
In [ ]: #Validation classification report
        cr = classification_report(y_val, y_val_pred)
        print(cr)

              precision    recall  f1-score   support

           0       0.91      0.82      0.86      1593
           1       0.49      0.69      0.58       407

    accuracy                           0.79      2000
   macro avg       0.70      0.76      0.72      2000
weighted avg       0.83      0.79      0.81      2000
```

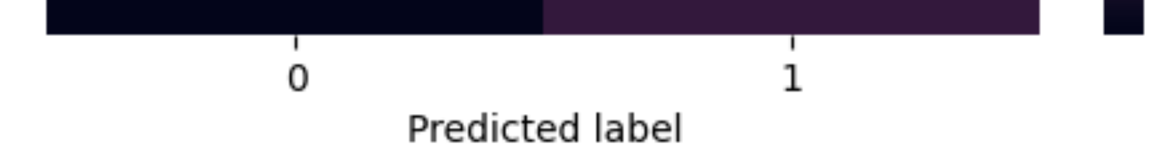
Confusion Matrix

```
In [ ]: # Confusion matrix for training
        make_confusion_matrix(y_train_sm, y_train_pred)
```



```
In [ ]: # Confusion matrix for validation
        make_confusion_matrix(y_val, y_val_pred)
```





Model Performance Comparison and Final Model Selection

```
In [ ]: df = pd.concat([train_scores_df, val_scores_df, (train_scores_df - val_scores_df)], axis=1)
        df.columns = ['Train Recall', 'Validation Recall', 'Delta']
        df
```

Out[ ]:

	Train Recall	Validation Recall	Delta
NN with SGD	0.206	0.187	0.019
NN with Adam	0.939	0.459	0.479
NN with Adam Dropout	0.685	0.442	0.243
NN with SGD Smote	0.775	0.688	0.087
NN with Adam Smote	0.990	0.516	0.474
NN with Adam Smote & Dropout	0.835	0.767	0.068
NN with Adam Smote, Drop, Early Stop	0.819	0.693	0.126

- We can choose the model with **Balanced Data (SMOTE), Adam Optimizer and Dropouts** based on the table above.

Model Performance on Test data

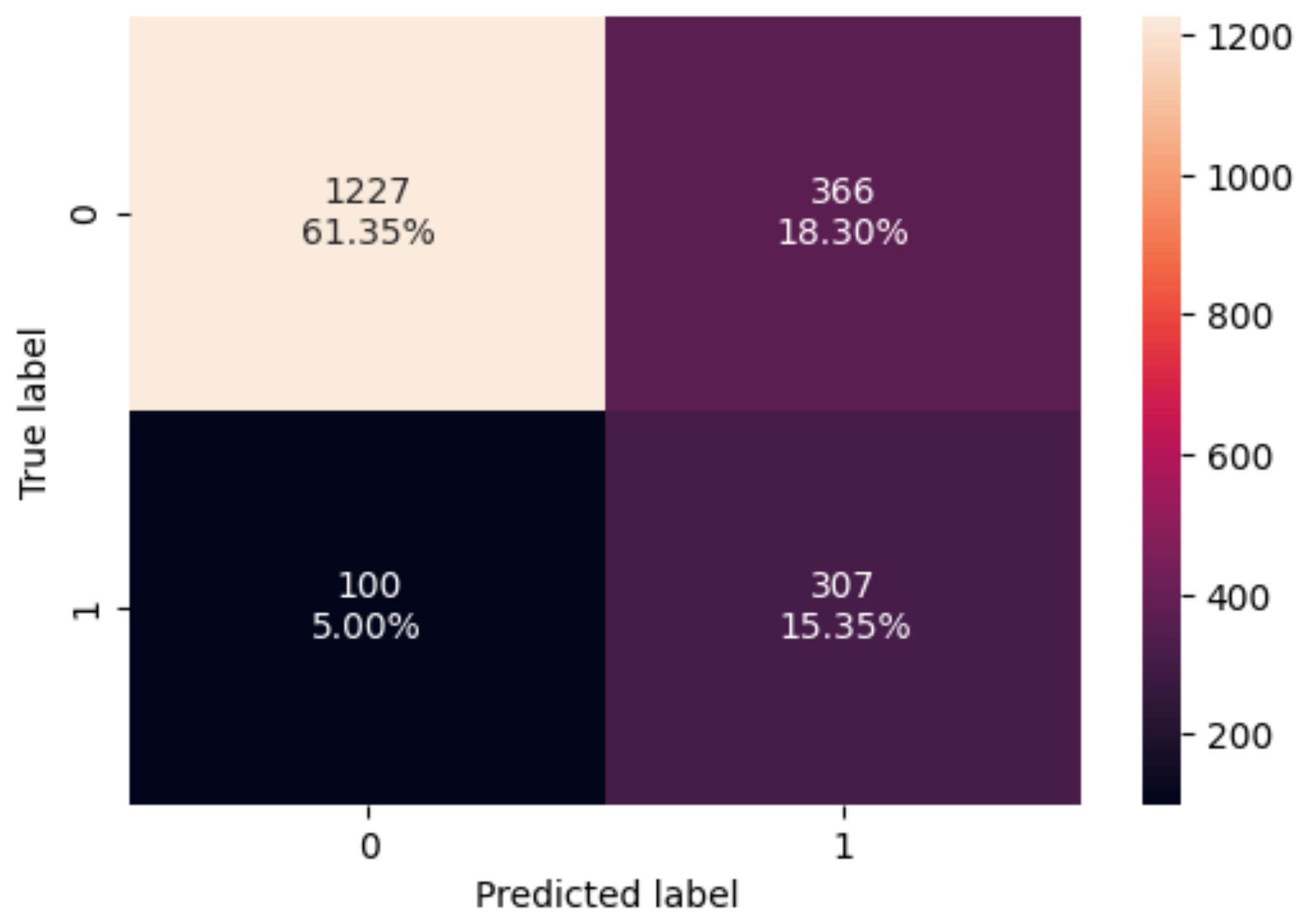
```
In [ ]: # Model chosen is model_ao_sm_do
        y_test_pred = model_ao_sm_do.predict(X_test)
        y_test_pred = (y_test_pred > 0.5)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
In [ ]: #Classification report
        cr = classification_report(y_test, y_test_pred)
        print(cr)
```

```
              precision    recall  f1-score   support

           0       0.92      0.77      0.84      1593
           1       0.46      0.75      0.57       407

    accuracy                           0.77      2000
   macro avg       0.69      0.76      0.70      2000
weighted avg       0.83      0.77      0.79      2000
```

```
In [ ]: #Confusion Matrix for test data
        make_confusion_matrix(y_test, y_test_pred)
```



- Good Minority Recall: The primary success of this model (maybe due to SMOTE) is its ability to find a large portion (77%) of the minority class instances (good Recall for Class 1). It misses relatively few actual Class 1 cases (only 93 FN).
- Poor Minority Precision: The major weakness is the high number of False Positives for Class 1. The model frequently misclassifies majority class instances (Class 0) as belonging to the minority class (Class 1), leading to very low precision (0.44) for Class 1.
- The model achieves high recall for the minority class at the significant cost of precision for that same class.
- Moderate Majority Class performance: The model performed decent on the majority class (Class 0), achieving high Precision (0.93) but only moderate Recall (0.75). The lower recall indicates that a noticeable portion (25%) of Class 0 instances were misclassified, primarily as Class 1 (contributing to Class 1's low precision).
- Overfitting is still observed with balanced training data. The learnings did not translate well to unseen imbalance data.
- Impact of SMOTE and Dropout: While SMOTE successfully forced the model to learn the minority class (leading to high recall), training on perfectly balanced, potentially synthetic data likely contributed heavily to the overfitting. The Dropout (0.5) used was insufficient to prevent this severe overfitting or to improve the poor precision on the imbalanced test set.

Actionable Insights and Business Recommendations

Key Findings/Actionable Insights

- Churn Risk Is Influenced by Age, Credit Score, and Account Activity
  - Older customers and those with lower credit scores are more likely to churn.
  - Customers marked as inactive (IsActiveMember = 0) show a significantly higher churn rate.
  - Geography plays a role — churn patterns vary across countries (e.g., higher churn in Germany).
- Balance and Products Held Are Not Strong Retention Factors Alone
  - Customers with high balances are still churning, indicating that balance alone is not a loyalty indicator.
  - Having only one product is common among churners - cross-selling may improve retention.
- Credit Card Ownership Shows Weak Correlation to Retention
  - The presence of a credit card does not appear to strongly affect churn.
  - Offering a card is not a strong enough hook — it should be combined with other benefits.
- Machine Learning Models Provide High Predictive Accuracy
  - Several models achieved high precision and recall, especially using the Adam optimizer.
  - With added regularization and dropout, overfitting was reduced, improving real-world generalization.

Business Recommendations

1. Proactively Engage High-Risk Segments

- Target older, inactive customers with acceptable credit scores for retention campaigns.
- Introduce personal finance coaching or customized offerings for these segments.

2. Geography Specific Strategies

- Analyze regional churn behavior deeper (e.g., why Germany sees higher churn).
- Create geography-personalized loyalty programs or local customer service improvements.

3. Enhance Product Bundling & Cross-Sell Offers

- Promote bundles (e.g., savings + investment products) to customers with only one product.
- Provide incentives for expanding their portfolio — perhaps gamified loyalty tiers.

4. Track and Act on Activity Signals

- Use IsActiveMember as a live signal for customer health scoring.
- Reach out early to inactive users before they churn — with nudges or benefit reminders.

5. Optimize Credit Scoring and Customer Education

- Educate customers on how improving their credit score can enhance their financial options.
- Offer free credit score monitoring within banking apps to increase stickiness.

Power Ahead ___